

UNIT STUDY NOTES

Data Analysis with Pandas

Pandas turns a table into something you can interrogate. This unit covers Series and DataFrame construction, selection and alignment, missing data, grouping and aggregation, reshaping, merges and joins, and time-aware indexes.

 9 source notebooks 239 topic sections extracted 7 study sections

Unit Overview

Up to this point, every program you have written has been built from the fundamentals: variables, lists, dictionaries, functions, conditionals, loops, and finally NumPy arrays for fast numerical work. Now we climb the next rung of the data science ladder: pandas. If NumPy provides raw speed and arrays, pandas provides meaning. It wraps data in a structure that knows both the values AND the labels attached to them. Pandas is, in its own words, a fast, powerful, flexible and easy-to-use open source data analysis and manipulation tool built on top of the Python programming language.

The two most important structures in pandas are the **Series** and the **DataFrame**. Think of an Excel sheet: a single column, with a header on top and row numbers on the left, is essentially a Series. The whole sheet, with many columns sharing the same row labels, is a DataFrame. This session focuses completely on the Series — the one-dimensional labeled array — because almost every pandas operation you will ever run eventually reduces to working on a Series.

Here is the roadmap for the session:

1. What pandas is and how to import it.
2. Creating a Series from a list, with an automatic index that runs from 0 to n minus 1.
3. Giving a Series a custom index and a name.
4. Building a Series directly from a dictionary, where keys become labels.
5. Inspecting a Series with attributes such as size, dtype, name, is_unique, index, and values.
6. Loading a single-column CSV straight into a Series with read_csv.
7. Copying data safely so that editing a subset never mutates the original by accident.
8. Previewing and sorting with head, tail, sample, value_counts, and both sort methods.
9. Using maths methods such as count, sum, mean, median, mode, std, var, and describe.
10. Indexing, slicing, and editing values, including adding brand-new labels.
11. Connecting a Series to everyday Python: len, dict, membership, and broadcasting.
12. Boolean filtering to answer real questions such as: how many fifties did a batsman score?
13. Plotting a Series in a single line.
14. A toolbox of important methods: isin, apply, astype, between, clip, and the families that handle duplicates and missing values.

By the end you will think of a Series not as a plain list but as labeled data — values plus names — and you will be able to load, inspect, filter, compute on, and plot it using only a handful of expressive commands.

In the previous session you mastered the Series — a one-dimensional labeled array that behaves like a single Excel column with row numbers on the left and values on the right. Now the picture widens to its natural companion: the **DataFrame**, the two-dimensional labeled table that surveys every data science project in this course. If a Series is one column, a DataFrame is the whole spreadsheet: many columns side by side, all sharing the same row labels. Under the hood each column of a DataFrame is itself a Series, so everything you learned about Series — labels, dtypes, methods, filtering — carries straight over, column by column.

The DataFrame is also the bridge between pandas and the real world. Most data arrives as CSV files, and `pd.read_csv(...)` turns any such file into a DataFrame in one line. From there you can inspect its shape, its column types, its missing values, and its summary statistics, then select columns, select rows, and combine the two to cut out exactly the window of data you need.

Here is the roadmap for this session:

1. Creating a DataFrame from a list of lists, from a dictionary of lists, and directly from a CSV file.
2. Inspecting a DataFrame with shape, dtypes, index, columns, and values, plus the `info()` and `describe()` summaries.
3. Running maths across rows and columns with the axis parameter — the idea that sums and means can go down OR across.
4. Selecting one column into a Series, or several columns into a new DataFrame.
5. Selecting rows by integer position with `iloc` and by index label with `loc`.
6. Selecting rows and columns at the same time with two-dimensional indexing.
7. Filtering rows with boolean conditions — the pandas equivalent of SQL WHERE.
8. Adding brand-new columns, both as constants and as values derived from existing columns.
9. Optimizing memory by converting column dtypes, including the compact category dtype.
10. Exploring categorical columns quickly with `value_counts` and plots.

By the end of the session you will be able to load a real dataset, take its pulse in seconds, and reach in to drag out exactly the rows and columns your question needs. The DataFrame is where pandas earns its reputation, and everything you learn here will be used — and extended — in every later pandas session, including grouping and merging.

In the last two sessions you met the Series and the DataFrame and learned to build, inspect, select, and filter them. This session is the tool drawer: a deliberate, method-by-method tour of the DataFrame functions you will reach for in almost every real analysis. Each method is demonstrated on small toy frames first, so the behaviour is crystal clear, and then on real datasets — movies, IPL matches, and student records — so you can see the method solving an actual problem.

The session follows a natural pipeline. It opens with frequency: which values repeat, and how often — that is `valuecounts`. *Then it moves the data around: sorting by values or by index, and re-labelling with `setindex` and `resetindex`. Along the way you will compute ranks for leaderboards, rename columns, count unique values, and then enter the world of missing data — detecting it with `isnull` and `notnull`, deleting it with `dropna`, and patching*

it with fillna. Duplicates get their own pair of tools, dropduplicates and duplicated, and whole rows or columns can be removed with drop. Custom logic arrives through apply, membership checks through isin, and relationships between numeric columns through corr. The session finishes with quick top-N extraction via nlargest and nsmallest, positional column addition via insert, and safe copying with copy.

Here is the one-line mental map to hold onto: most pandas methods return a NEW object and leave the original untouched — if you want the change to be permanent, pass inplace=True where it is supported. Throughout the session, keep asking two questions of every method: what does it operate on (Series, DataFrame, or both?), and what does it return? Answer those two questions and the entire set of methods becomes predictable.

This session is dense by design. Do not rush — work the code cells in order, predict each output, and let the repetition across toy data and real data build the muscle memory you will carry into grouping and merging next.

Core ideas

DataFrame is tabular (rows/cols), labeled axes. Series is 1D.

Why

Facilitates selection, filtering, aggregation, joins, I/O.

AI relevance

Preprocessing/EDA.

In earlier sessions you learned to build, inspect, filter, and clean Series and DataFrames. Everything you did there operated on the whole table at once. Real questions, however, are almost never about the whole table — they are about its parts. Who is the top earner among action movies? Which director gets the most votes on average? How many movies did each actor lead? Every one of these questions splits the data into groups, computes something inside each group, and joins the answers back together. That pattern is so fundamental that pandas ships a dedicated engine for it: the GroupBy object, built on the split-apply-combine strategy.

Split-apply-combine is the mental model for this whole session. SPLIT: pandas divides the rows according to the distinct values of one or more columns — a group per genre, per director, per actor. APPLY: a function runs inside each group — sum, mean, count, or any function you write yourself. COMBINE: all the per-group results are collected back into one Series or DataFrame, indexed by the group keys. If you have ever used SQL's GROUP BY or a pivot table in Excel, you already know this concept; this session gives you the pandas spelling of it.

The session takes you through the GroupBy toolkit step by step. You start by creating a GroupBy object and confirming that it is lazy — grouping alone does nothing until you apply an operation. Then you run the built-in aggregations: sum, mean, std, and count across every group, chained to sortvalues and head for leaderboards. Next come the attributes that inspect the grouping itself: len and size for group counts, getgroup to pull out one group, groups to see the row-index mapping, and describe, nunique, sample, and nth for group-wise summaries.

From there the session moves to richer tools. The `agg` method runs several different aggregations in a single call using a dictionary or a list. Looping shows how to iterate (`groupname`, `groupdataframe`) pairs manually. `apply` is the full-power escape hatch that runs any custom function on every group — used here for within-group rankings and normalisation. Finally, `GroupBy` on multiple columns builds nested groups whose keys are tuples and whose results carry a `MultiIndex`. The notebook closes with a cricket exercise on ball-by-ball deliveries, where all these patterns come together: most runs, most sixes, death-over hitters, and a reusable function for any batsman's highest score.

Work every code cell in order. `GroupBy` is one of those tools you can only master by running — predict the output first, then check it against the line below.

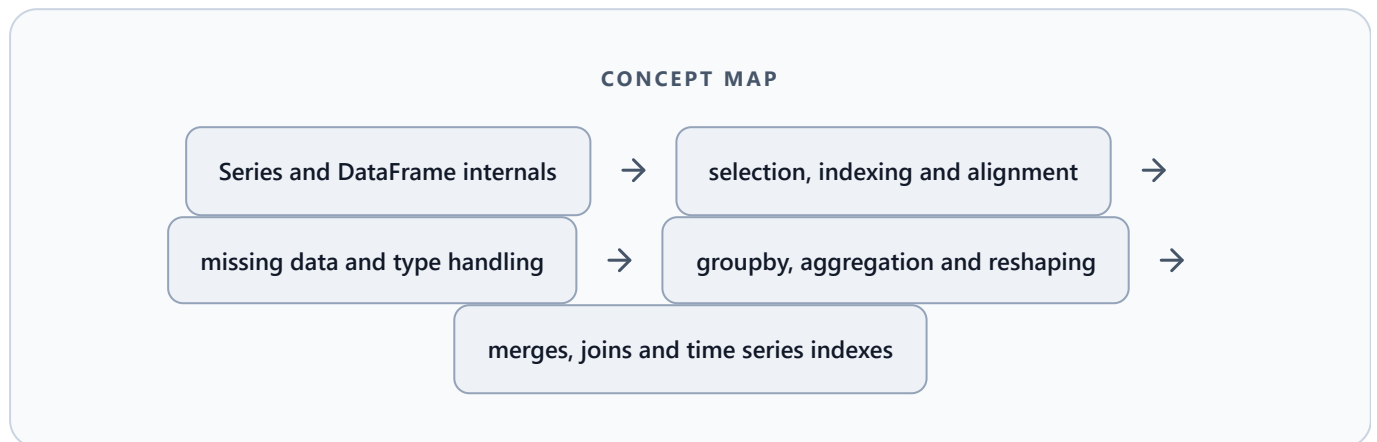
Prerequisites

→ NumPy

What you will be able to do

- ✓ Build and inspect a `DataFrame` with correct dtypes
- ✓ Select data with `loc`, `iloc` and boolean masks
- ✓ Handle missing data deliberately rather than silently
- ✓ Group, aggregate, pivot and reshape with confidence
- ✓ Join datasets without corrupting row counts

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

Pandas Series

1. Introducing Pandas

Pandas fundamentals

Labeled axes, alignment, indexing, grouping, missing data, I/O. Critical for EDA/preprocessing.

AI/ML relevance

Shapes, dtypes, vectorization, indexing, and clean tabular ops are critical for reliable data pipelines.

1.1 Importing pandas

Pandas is built on top of NumPy, so in almost every notebook that uses pandas you will see the two imports that open this session:

The `import pandas as pd` line creates the pandas module under a short alias, so that every time you need a pandas function you write `pd.` instead of the longer full name. The `import numpy as np` line is just as important because pandas stores its underlying numeric data in NumPy arrays, and a lot of pandas behaviour — vectorized operations, broadcasting, and dtype handling — is really NumPy behaviour wearing a pandas coat.

Why an alias at all? Consider how often you will type `pd.Series(...)` or `pd.read_csv(...)` in a single analysis. Heavy modules are almost always imported with a short, conventional alias: `np` for NumPy and `pd` for pandas. If you see code that says `from pandas import Series`, it works too, but the `pd.` prefix is far more common and it clearly advertises where each function lives.

One mental model that will help throughout this session: a pandas Series is a NumPy array plus a set of labels. The labels are the index, and together they give the data meaning. Everything you loved about fast, vectorized NumPy still works under the hood, but now each element can also be found by a name.

PYTHON

```
import numpy as np
import pandas as pd
```

PYTHON

```
import numpy as np
import pandas as pd
```

2. Series from Lists

Pandas fundamentals

Labeled axes, alignment, indexing, grouping, missing data, I/O. Critical for EDA/preprocessing.

2.1 The labeled array

The very first thing the code does is wrap an ordinary Python list in `pd.Series(...)`:

When you print such a Series, pandas shows you two things side by side. On the left is the **index** — the labels — and on the right is each **value**. Because we did not supply labels, pandas creates a default index automatically: `0, 1, 2, 3, 4`. This is called a RangeIndex, and it behaves much like list positions. The vertical arrangement matters: row 0 of the index holds `'India'`, row 1 holds `'Pakistan'`, and so on.

Second, notice the bottom line: `dtype: object`. Since the list contains text, pandas stores it as the general object dtype rather than a numeric one. If the list had contained integers, you would see `dtype: int64` instead. So a Series can hold data of any type — strings, integers, floats, and even Python objects — which is exactly why pandas describes a Series as a one-dimensional array that can hold data of any type.

Think of a Series as an Excel column: the row numbers on the striped bar are the index, the cells are the values, and the column header (which we meet later as the Series name) sits at the top. Everything pandas does from here on — slicing, filtering, grouping, plotting — assumes this labeled column as its basic unit.

PYTHON

```
country = ['India', 'Pakistan', 'USA', 'Nepal', 'Srilanka']
pd.Series(country)
```

2.2 Dtype inference on creation

Run the string example and then the integer example and compare what pandas reports:

The first prints `dtype: object`; the second prints `dtype: int64`. Pandas looks at every value in the list and picks the narrowest dtype that can represent all of them. This is called dtype inference. A list of integers becomes int64, a list of floats or a mixture containing a float becomes float64, and text becomes object.

Why care? Your dtype decides which operations are legal and how much memory the Series consumes. Integer and float dtypes let you sum, average, and compare the values with fast vectorized NumPy routines. An object dtype mostly forces slower, element-by-element handling. This is the first hint of a theme that will return in `astype` and the category dtype: the right dtype makes your analysis correct AND fast.

There is also a small but important detail hiding in this section: the Series that holds runs is assigned to a variable, `runs_ser`. The string Series was created inline and never stored. In later cells you will rely on `runs_ser`, so storing the result of `pd.Series(...)` is the normal way to keep data around. An inline creation is useful for a quick peek; a named variable is what you need for real work.

PYTHON

```
# string
country = ['India', 'Pakistan', 'USA', 'Nepal', 'Srilanka']

pd.Series(country)
```

PYTHON

```
# integers
runs = [13,24,56,78,100]

runs_ser = pd.Series(runs)
```

3. Custom Index and Name

3.1 Why labels matter

So far the Series used a default 0-to-n index. But the whole point of a labeled array is that the labels can carry meaning. Here the code attaches subject names to marks and a title to the whole column:

The `index=` parameter maps one number to each label: `67` belongs to `'maths'`, `57` to `'english'`, and so on. The lists must have the same length — pandas raises a `ValueError` otherwise, because it would not know which label belongs to which value.

The `name=` parameter gives the whole Series a title. It appears at the bottom of the printed output, like `Name: Nitish ke marks, dtype: int64`, and it becomes the column header if you later place this Series inside a `DataFrame`. A descriptive name is free documentation for you and for anyone reading your output.

Once labels exist, they are not just decoration. You can fetch a single value by its label with `marks_series['maths']`, exactly the way you would look up a dictionary key. And later in the course, when two Series interact, pandas will align them BY INDEX rather than by position — two Series sharing labels automatically line up their values. That is the superpower of labeled data, and it is why the Series keeps its index alongside the values at all times.

PYTHON

```
# custom index
marks = [67,57,89,100]
subjects = ['maths','english','science','hindi']

pd.Series(marks,index=subjects)
```

PYTHON

```
# setting a name
marks = pd.Series(marks,index=subjects,name='Nitish ke marks')
marks
```

4.1 Dict keys become the index

If a Series is a labeled array, then a Python dictionary — keys and values — is a very natural way to describe it:

When you pass a dict to `pd.Series(...)`, the dict keys become the index labels and the dict values become the Series values. No separate index list is needed, because the keys already ARE the labels. This is the most Pythonic of the creation routes: you write a label-value mapping once and pandas turns it into a labeled column.

A few practical details are worth storing. First, order is preserved. Since Python 3.7, dicts remember insertion order, and pandas builds the Series in that same order — mathematics, then english, science, hindi, in the exact order the dict was written.

The dtype is inferred from the values, exactly as in the list case. Here all four values are integers, so the Series is int64.

Second, this pattern composes beautifully with the real world. Configuration data, lookup tables, and small metadata maps are usually stored as dictionaries, and a single `pd.Series(dict)` turns any of them into an analysis-ready column.

Finally, note that you can still pass a `name=` even when building from a dictionary. The name lives on the Series itself, independent of the index, and it will be printed at the bottom of the output as the title of the column.

PYTHON

```
marks = {
    'maths':67,
    'english':57,
    'science':89,
    'hindi':100
}

marks_series = pd.Series(marks,name='nitish ke marks')
marks_series
```

PYTHON

```
marks = {'maths': 67, 'english': 57, 'science': 89, 'hindi': 100}
pd.Series(marks, name='nitish ke marks')
```

5.1 Introspecting a Series

Every pandas object carries metadata that describes its own structure. The attributes you will use constantly are:

ATTRIBUTE	MEANING
<code>size</code>	total number of elements
<code>dtype</code>	data type of the values
<code>name</code>	name of the series
<code>is_unique</code>	True when all values are distinct
<code>index</code>	the index (labels) object
<code>values</code>	the raw values as a NumPy array

The cells run each one against `marks_series`: `size` returns 4, `dtype` returns `int64`, `name` returns the title string, and `is_unique` returns `True` because all the marks differ. The second `is_unique` example builds `pd.Series([1, 1, 2, 3, 4, 5])`, which contains a repeated 1, and correctly reports `False`.

`index` and `values` deserve special attention. `marks_series.index` returns an `Index` object holding the labels: `Index(['maths', 'english', 'science', 'hindi'], dtype='object')`. For the runs Series created without custom labels, `runs_ser.index` returns a `RangeIndex(start=0, stop=5, step=1)` — the compact representation of the labels 0 through 4.

`values`, by contrast, drops the labels entirely and returns the underlying NumPy array — `array([67, 57, 89, 100])`. If you ever need pure numbers for a fast library function that knows nothing about pandas, `series.values` is the door out of the pandas world.

5.2 Reading attributes in practice

Attributes are your first response when you meet an unfamiliar Series. The pattern is: create a Series, then interrogate it.

`marks_series.size` and `len(marks_series)` answer the same question, but `size` arrives through the pandas layer and is slightly cheaper on large arrays. `marks_series.dtype` tells you what kind of numbers or text you are holding and, as we saw, which operations will be fast. `name` tells you what the column represents — always worth printing before a long analysis. `is_unique` is the start of duplicate detection: if it is `False`, some values appear more than once and you may want to investigate.

The `Index` object returned by `.index` can hold custom labels (like subject names), and pandas makes no assumption that the labels are numbers. When labels are numbers 0 through `n-1`, pandas still stores them efficiently as a `RangeIndex` rather than as a list of separate integers.

Two extra attributes appear in the notes for this section and are worth knowing even though they are not run as separate cells. `.shape` returns the dimensions as a tuple — for a Series it is `(n,)`, one element followed by a comma — and `.hasnans` returns `True` if any value is missing. Together with `size`, `dtype`, and `name`, these form the metadata toolkit you will use at the start of every exploration: check the shape, check the type, check the name, check for missing values. Almost every data science workflow begins with a one-line structural report of the data you just loaded.

PYTHON

```
# size
marks_series.size
```

PYTHON

```
# dtype
marks_series.dtype
```

6.1 readcsv, squeeze, and indexcol

Most real data does not begin as a list or a dict; it begins as a file. The standard entry point is `pd.read_csv(...)`. When the CSV holds a single meaningful column, pandas can deliver it directly as a Series:

The `subs.csv` file contains one data column, so with `squeeze=True` pandas turns the result into a Series instead of a one-column DataFrame; its name is the column header, `Subscribers gained`. Printing it shows the default RangeIndex 0 to 364 and dtype int64 — 365 daily values.

The `kohli_ip1.csv` file has two columns, a match number and a runs value. The trick is `index_col='match_no'`: that column becomes the index, and the remaining runs column becomes the Series values. The output then shows scores labeled by match number, with the Series named `runs`.

The `bollywood.csv` file associates each movie title with a lead actor. `index_col='movie'` makes the title the index, leaving the actor column as the values, and the Series is named `lead`.

This is the closest pandas gets to a dedicated 'Series from CSV' constructor: use `squeeze=True` when there is one column, and add `index_col=...` when a column should become the label axis. The result is data that arrives already labeled and ready for the methods in the rest of this session.

PYTHON

```
# with one col
subs = pd.read_csv('/content/subs.csv', squeeze=True)
subs
```

PYTHON

```
subs = pd.read_csv('/content/subs.csv', squeeze=True)
vk = pd.read_csv('/content/kohli_ip1.csv', index_col='match_no', squeeze=True)
movies = pd.read_csv('/content/bollywood.csv', index_col='movie', squeeze=True)
```

7. Copy versus Views

7.1 The trap of shared data

Pandas frequently avoids copying data. Instead, a derived object may be a **view** that points at the same underlying memory as its source. The cells demonstrate the trap with `vk.head()`:

`vk.head()` takes the first five rows of the batting runs Series. Assigning `new[1] = 1` looks harmless — a local change to a small preview. But because `new` is a view, the write travels straight back into the original `vk`, silently changing the original's second value. A beginner reads this code and expects the original to be untouched; pandas, for efficiency, shares the underlying data instead of copying it.

The rule is simple: when you plan to MODIFY a subset of data, make an independent copy first with `.copy()`:

Now `new` owns its own storage. Setting `new[1] = 100` changes only the copy, and the printed original `vk` still shows its untouched values. This is also the seed of a warning you will meet in DataFrames: chained assignment, where the selection and the write happen in separate steps, triggers pandas' `SettingWithCopyWarning` precisely because it is impossible to tell at a glance whether you are editing a view or a copy. Internalize the habit now — copy before you modify — and you will avoid a whole class of silent bugs.

PYTHON

```
new = vk.head()
new[1] = 1
```

PYTHON

```
new = vk.head().copy()
new[1] = 100
```

7.2 Making safe copies

When should you reach for `.copy()`? Whenever the object you are about to edit is derived from another object by filtering, slicing, or a chain of methods such as `head()` or `tail()`.

The safest workflow used by experienced pandas users is:

1. Read the data.
2. Take a preview with `head()` or `tail()` to verify it looks right.
3. If the preview is only for looking, keep it as is.
4. The moment a subset must be changed, call `.copy()` on it first.

The distinction between the two examples in this notebook is the entire lesson:

The first block mutates the original silently; the second block is fully contained. The same semantics apply to `.loc` and `.iloc` selections, which you will learn later: a slice without `.copy()` is a window onto the original data.

This copy-versus-view behaviour is a classic interview topic, but more importantly it is a daily safety habit. Data cleaning pipelines often reload the same dataset many times, and an accidental view-write can corrupt a DataFrame you thought was unchanged. One `.copy()` call guarantees that your edits stay local, and the tiny memory cost is far cheaper than debugging a mystery mutation.

PYTHON

```
# with 2 cols
vk = pd.read_csv('/content/kohli_ipl.csv', index_col='match_no', squeeze=True)
vk
```

PYTHON

```
movies = pd.read_csv('/content/bollywood.csv', index_col='movie', squeeze=True)
movies
```

8.1 head, tail, sample, and value_counts

After loading real data, the first moves are preview and counting. The cells apply four workhorse methods.

`subs.head()` shows the first five rows, and `vk.head(3)` shows the first three. `vk.tail(10)` shows the last ten. Both take an optional count parameter and default to five. They are cheap ways to peek at the top and bottom of data and confirm the file loaded as expected.

`movies.sample(5)` returns five random rows — useful for a quick sanity check when the data has no natural top or bottom, or when you want a taste of the full range without bias. Each call picks different rows.

`movies.value_counts()` deserves special attention. It counts how many times each distinct value appears and returns a new Series whose index is the unique value and whose values are the frequencies, sorted descending. The output answers an immediate question: `Akshay Kumar` stars in 48 movies, `Amitabh Bachchan` in 45, `Ajay Devgn` in 38, then Salman Khan, Sanjay Dutt, and so on down to the actors who appear only once. It is the fastest possible way to understand the distribution of a categorical column.

Together these four methods form the standard greeting committee for any dataset: look at the head, glance at the tail, sample a few random rows, and count how common each category is.

8.2 sortvalues and sortindex

Sorting brings order to a column. Two methods handle it: `sort_values` sorts by the VALUES, while `sort_index` sorts by the LABELS.

The first call sorts the runs Series from highest to lowest with `ascending=False`. Reading the output you can immediately see the top scores: match 128 with 113 runs, match 126 with 109, then 108, 100, 100, and so on down to the many zero-run matches. Confirming the top value is as easy as chaining `.head(1)`, which the notebook does to report the maximum, 113.

`sort_index(ascending=False)` rearranges by label instead — the bollywood Series becomes ordered by movie title from Z backwards to A. `movies.sort_index(ascending=False, inplace=True)` is special because `inplace=True` mutates the original Series: pandas sorts `movies` in place and returns nothing. The later `movies` prints confirm the titles now run from Zor Lagaa Ke...Haiya! downward.

The final cells show the complement: `vk.sort_values(inplace=True)` permanently sorts the runs from smallest to largest, so the zeros come first and the 113 appears last.

The keyword trio to remember for both methods is: `ascending` controls direction, `inplace` decides whether the original is modified, and `na_position` (seen later) decides where missing values go. Sorting plus a preview is how you extract top performers in one line.

PYTHON

```
# head and tail
subs.head()
```

PYTHON

```
vk.head(3)
```

9.1 count, sum, mean, median, mode, std, var

Aggregate maths on a Series is one line — no loops, no explicit iteration:

Each method collapses the whole column down to a single number. `vk.count()` reports 215, the number of valid (non-missing) values in the runs Series. `subs.sum()` gives 49510 — the total daily subscribers gained across the year. `subs.mean()` is the average, `vk.median()` is 24.0, and `movies.mode()` is the most frequent

value, `Akshay Kumar`, `subs.std()` and `vk.var()` measure how spread out the values are: variance is the average squared deviation from the mean, and the standard deviation is its square root, which keeps the size in the original units.

All of these are vectorized: pandas delegates the heavy arithmetic to speedy compiled routines and returns the scalar answer. If you had computed them with a manual Python loop, the code would be longer and far slower.

One subtlety worth internalizing: `count` EXCLUDES missing values. If a Series had NaNs, `count` would report only the valid entries, while `size` (seen earlier) counts everything. That detail matters the moment a real dataset arrives with holes in it.

PYTHON

```
vk.count()
subs.sum()
subs.mean()
print(vk.median())
print(movies.mode())
print(subs.std())
print(vk.var())
```

9.2 describe and the statistical toolkit

Instead of calling six methods to characterize a column, you can call one:

`describe()` prints the summary statistics for the Series in one compact table: count, mean, standard deviation, minimum, the 25%, 50%, and 75% percentiles, and the maximum. For the subscribers data you get the full picture at a glance — 365 days, an average of about 135.6, a standard deviation around 62.7, a minimum of 33, and a maximum of 396. The 50% row is the median, which tells you that half the days stayed below 123 daily subscribers.

This is the classic first-glance report of exploratory data analysis. Before building any charts or models, ask `describe()` what your columns look like: their centre, their spread, their range. If the mean and median are far apart, the data is probably skewed. If the minimum is bizarre, there may be an outlier or a coding error.

Two supporting ideas connect these methods to what lies ahead. First, they all ignore missing values by default, treating NaN as absent rather than as data. Second, when you move to DataFrames in the next session, the same statistics can run down columns or across rows using an axis parameter — but on a one-dimensional Series the axis choice simply does not arise. Statistical summaries are always the fastest way to build intuition about a new column.

PYTHON

```
# count
vk.count()
```

PYTHON

```
# sum -> product
subs.sum()
```

10.1 Positional and label access

A Series supports two very different ways to ask for values, and this section demonstrates both:

Because `x` uses the default integer index, `x[-1]` works positionally and returns the last element. Negative indexing is fully supported, so `x[-1]` behaves like a Python list. The label-indexed Series follow the same idea: `movies['2 States (2014 film)']` returns the single value for that exact label — in the output, `'Arjun Kapoor'` — because for a Series with string labels, square brackets with a string mean label lookup, like a dictionary key.

Where pandas can get confusing is when the index itself is numeric. If a Series was created with default integer labels, square brackets behave positionally; if it was created with meaningful labels, they behave by label. The safe rule for this section: look at the index type first. A `RangeIndex` invites positional access, and a custom `Index` invites label access.

Slices behave positionally, exactly like NumPy. `vk[5:16]` returns rows 5 through 15 (stop exclusive), `vk[-5:]` grabs the last five rows, and `movies[:,2]` steps through every second movie. Fancy indexing with a list — `vk[[1, 3, 4, 5]]` — returns the rows at those positions in one call. All of these are quick, expressive ways to grab subsets without writing a loop.

PYTHON

```
# integer indexing
x = pd.Series([12,13,14,35,46,57,58,79,9])
x
```

PYTHON

```
x = pd.Series([12, 13, 14, 35, 46, 57, 58, 79, 9])
```

10.2 Fancy indexing and step slices

Fancy indexing means passing a list of positions inside the square brackets, and the code demonstrates it with `vk[[1, 3, 4, 5]]`:

The Series rows at match numbers 1, 3, 4, and 5 are returned as a new Series: scores 1, 13, 12, and 1. The order of the returned rows follows the order of the requested indices, not the original data order — you could ask for `vk[[5, 1]]` and get the rows in that order.

The slice examples in this section cover the three classic slice shapes:

- `vk[5:16]` — a normal start:stop slice, stop exclusive, giving eleven rows (5 through 15).
- `vk[-5:]` — a negative-start slice, taking the last five rows (matches 211 to 215).
- `movies[:,2]` — a start:stop:step slice that takes every second movie title.

All three work because pandas slices a Series the way you slice a Python list or a NumPy vector, and both positive and negative indices are legal.

The last example is label access: `movies['2 States (2014 film)']`. When the index holds strings, a bare string inside brackets is a label lookup, and the result is the single value for that key — here, the lead actor `'Arjun Kapoor'`.

The practical advice pandas veterans repeat: when the index type makes access ambiguous, prefer the explicit `.loc` and `.iloc` accessors that arrive in the DataFrame session. They make the intent — label versus position — impossible to misread, and they clear up most of the confusion around 'why did pandas return the wrong row?'.

PYTHON

```
# negative indexing
x[-1]
```

PYTHON

```
movies
```

11.1 Assignment by position and label

A Series is mutable: you can change existing values, add new ones, and edit whole chunks in one go. All the indexing styles from the previous section have a matching assignment form.

`marks_series[1] = 100` replaces the value at position 1 of the Series. After the assignment, the printed output shows english at 100 instead of 57. This is positional editing — the index label stays the same, only the value changes.

The slice and fancy forms extend the same idea to whole ranges at once. `runs_ser[2:4] = [100, 100]` sets two consecutive positions in one statement, and because the runs Series was given a default index, the assignment is positional. `runs_ser[[0, 3, 4]] = [0, 0, 0]` is fancy assignment: a list of positions on the left, a list of new values on the right, matched one to one.

Label-based editing works exactly like a dictionary write. `movies['2 States (2014 film)'] = 'Alia Bhatt'` looks up the row whose label is that movie title and swaps its value.

Because a Series holds its values in one homogeneous array, an assignment may also change the dtype when the new value forces it — assigning a float into an integer Series silently upgrades it. Keep that in mind as a possible side effect of any bulk edit.

11.2 Adding new labels

Editing is not limited to labels that already exist. Assign to a label that is missing from the index, and pandas appends a brand-new element:

The label `'evs'` did not exist, so pandas adds a new row to the Series with that label and the value 100. When the Series is printed, the new subject appears at the bottom — the output shows maths, english, science, hindi, sst, evs in the order they were inserted. The earlier cells had set sst to 90 and english to 100, so the final Series holds a mix of original and edited values.

This combination — update existing labels, append missing ones, and mutate slices — is exactly the behaviour a real data-cleaning workflow needs. Booleans, placeholders, corrected values, and new categories all funnel into the Series through the same assignment syntax.

Three habits keep this kind of work safe. First, when speed matters for a single element, `.at[label]` and `.iat[position]` are faster than the general bracket syntax because they skip pandas' ambiguity checks. Second, remember the copy-versus-view lesson: if you edit a derived subset, copy it first so the mutation stays local. And third, be aware that values are aligned by index on assignment — pandas matches labels, not row order, so a renamed Series can surprise you if its labels no longer line up.

PYTHON

```
# using indexing
marks_series[1] = 100
marks_series
```

PYTHON

```
# what if an index does not exist
marks_series['evs'] = 100
```

12. Copy and Views in Practice

12.1 When edits leak into the original

This section returns to the copy-versus-view theme and demonstrates it with the exact batting data you have been working with:

Watch the order of operations carefully. First, `new = vk.head()` builds a preview of the first five rows. Then `new[1] = 1` edits it. Because `head()` returned a view that shares memory with `vk`, that edit FALLS THROUGH into the original Series — the number at match 1 is overwritten where it lives in `vk`.

Then the code rebuilds the subset the safe way: `new = vk.head().copy()`. This time `new` owns its own data. `new[1] = 100` writes 100 into the copy, and when the notebook prints both, the copy shows 100 at its first row while `vk` is unchanged — its match number 1 still holds that earlier, view-written value.

The lesson is visual and permanent: a bare `head()`, `tail()`, or slice is a window onto the original; add `.copy()` and it becomes an independent snapshot. Remember the rule — modify only copies, never raw views — and you will dodge the class of bugs where an experiment on a small preview silently rewrites the master dataset.

PYTHON

```
new = vk.head()
new[1] = 1
new = vk.head().copy()
new[1] = 100
```

13.1 len, type, list, dict, membership

A Series is a pandas object, but it also behaves like an ordinary Python sequence, and this section proves how much interoperability survives:

`len(subs)` returns 365, the element count — same as `subs.size`. `type(subs)` reports `pandas.core.series.Series`, the real class behind the friendly name. `list(marks_series)` produces a plain Python list of the values, while `dict(marks_series)` rebuilds the original label-to-value dictionary. These conversions are the bridge back to classic Python whenever you need it.

Membership deserves a careful read because pandas splits the question into two. `'2 States (2014 film)' in movies` checks the INDEX: is that label present? It returns True. `'Alia Bhatt' in movies.values` checks the VALUES: is that actor any recorded lead? Also True. If you drop `.values` and write `'Alia Bhatt' in movies`, pandas looks only at the labels and returns False. The default `in` is an index test, exactly like a dictionary's key test, and `.values` explicitly switches the search to the raw data.

Note the `sorted(subs)`, `min(subs)`, and `max(subs)` calls from the first cell as well: these plain-Python functions iterate a Series the same way they iterate a list, so they work without any pandas-specific ritual.

PYTHON

```
len(subs)
type(subs)
list(marks_series)
dict(marks_series)
'2 States (2014 film)' in movies
'Alia Bhatt' in movies.values
```

13.2 Broadcasting and relational operators

Two NumPy-style behaviours make a Series behave like vectorized Python rather than a dumb column.

The addition is called BROADCASTING. A scalar meets a Series, and the operation applies to EVERY element at once: add 100 to 67 and get 167, 100 to 100 and get 200, and so on. The result is a whole new Series with the same index, and the original `marks_series` is never touched. Broadcast arithmetic turns a per-element loop into a single expression.

The comparison `vk >= 50` produces a BOOLEAN Series: for each run, pandas evaluates whether the score is at least 50 and stores True or False. The output shows the match numbers with mostly False values and an occasional True — match 213, with 73 runs, earns a True. A boolean Series is the raw material of filtering, which is precisely what the next section builds on top of it.

Three operators matter beyond arithmetic: `+`, `-`, `*`, `/` for maths, and the full family `>`, `>=`, `<`, `<=`, `==`, `!=` for comparison. Since pandas keeps the labels attached to every result, you can broadcast, compare, and then filter without ever losing track of which row you are talking about. The mental model — one expression, applied everywhere, labels preserved — is the reason pandas code reads so much like English questions about the data.

PYTHON

```
# Len/type/dir/sorted/max/min
print(len(subs))
print(type(subs))
print(dir(subs))
print(sorted(subs))
print(min(subs))
print(max(subs))
```

PYTHON

```
# type conversion
list(marks_series)
```

14. Boolean Filtering

Functional tools

map/filter apply functions; reduce aggregates. Often paired with lambdas.

14.1 Masks select matching values

The relational operators from the previous section now do the real work. The pattern is

```
series[boolean_series] :
```

Each condition builds a boolean Series of True and False values — one entry per row. When you place that boolean Series inside the square brackets, pandas keeps only the True rows and discards the False ones. The result is a new Series holding just the values that matched the condition, and `.size` counts them.

The three examples turn plain conditions into real cricket questions:

- `vk[vk >= 50]` keeps every innings of 50 runs or more, and `.size` reports 50 — that is the number of half-centuries plus centuries in this dataset.
- `vk[vk == 0]` keeps the ducks, and `.size` reports 9 — nine innings where he was out for a duck.
- `subs[subs > 200]` keeps the days with more than 200 new subscribers, and there were 59 of them.

A full loop could produce the same counts, but the mask expression is shorter, faster, and far easier to read. This single idiom — build a condition, wrap it in square brackets — is one of the most frequently used lines in all of pandas.

PYTHON

```
vk[vk >= 50].size
vk[vk == 0].size
subs[subs > 200].size
```

14.2 Filtering value_counts results

Boolean masks are not limited to original columns; they also filter the OUTPUT of other operations.

`movies.value_counts()` first counts how many movies each actor leads, producing a Series whose index is the actor and whose values are the counts. Then `num_movies[num_movies > 20]` applies a mask to that result, keeping only the actors with more than 20 movies.

The answer is the top of the bollywood pile: Akshay Kumar with 48, Amitabh Bachchan with 45, Ajay Devgn with 38, Salman Khan with 31, Sanjay Dutt with 26, Shah Rukh Khan with 22, and Emraan Hashmi with 21. Asking who appears often is now a two-step pipeline: count, then threshold.

Two refinements make masks stronger. You can COMBINE conditions with the operators `&` (and), `|` (or), and `~` (not) — pandas fits them elementwise, one True/False per row — and each condition must be wrapped in parentheses for the operator precedence to hold. You can also chain a mask directly onto a method and filter in one expression, though naming the intermediate Series is much more readable.

Finally, remember that a comparison with NaN is always False — a missing value satisfies no greater-than test. Keep that in mind when your data has holes, and pair masks with the missing-value tools if you need to include or exclude them deliberately.

PYTHON

```
# Find no of 50's and 100's scored by kohli
vk[vk >= 50].size
```

PYTHON

```
# find number of ducks
vk[vk == 0].size
```

15.1 plot() in one line

A Series carries its own plotting method, so a quick visual check costs a single call:

By default `plot()` produces a LINE chart: subscriber counts on the vertical axis against the index on the horizontal axis. Because `subs` is indexed 0 through 364, the line graph tells the story of the whole year at a glance — you can see the daily rhythm of subscriber growth and spot where the numbers spike.

Under the hood pandas delegates the drawing to matplotlib, the standard Python plotting library. The cell output shows the matplotlib axes object together with a small default figure, confirming that a chart was created. You do not need to call matplotlib directly for a basic plot — the `.plot()` method wires everything up for you.

The `kind=` parameter swaps the chart style:

A bar chart suits a Series where each label deserves its own column. A pie chart suits a frequency Series — which is exactly what the bollywood example builds. Plotting is the fastest sanity check in exploratory data analysis: one line from a column, and the shape of the data appears on screen.

PYTHON

```
subs.plot()
```

PYTHON

```
vk.plot(kind='bar')
movies.value_counts().head(20).plot(kind='pie')
```

15.2 Pie charts from value_counts

The pie plot neatly combines three ideas from this session:

Read the chain from left to right. First, `movies.value_counts()` turns the actor column into a frequency Series — how many movies each actor leads. Next, `.head(20)` cuts the list down to the top twenty actors, because a pie chart with over 500 slices (one per unique actor) would be unreadable. Finally, `.plot(kind='pie')` renders the top twenty as a pie.

Each slice corresponds to one actor, and pandas labels the slices with the index values — the actor names. The slice sizes are proportional to the counts, so the eye immediately lands on the biggest contributors.

A few plot styles are available through the same `kind` argument: `'line'`, `'bar'`, `'pie'`, `'hist'`, `'area'`, and `'box'` among others. The ones you will use most in this course are the line plot for trends, the bar plot for category comparisons, the histogram for numeric distributions, and the pie for shares of a whole.

For now, treat plotting as a one-line superpower. You do not need to memorize matplotlib's finer controls; the Series method produces a good default chart, and later sessions will refine titles, colours, and axis labels when a polished figure is required.

PYTHON

```
subs.plot()
```

PYTHON

```
movies.value_counts().head(20).plot(kind='pie')
```

16.1 isin and apply

Two methods cover a huge share of day-to-day selection and transformation.

`isin` answers membership for an entire Series at once:

The condition `vk.isin([49, 99])` returns a boolean Series that is True wherever a value equals 49 or 99. Wrapped in brackets, it filters to exactly the near-misses — match 82 with 99 runs and match 86 with 49 runs. Notice how this replaces a manual OR chain: the notebook first writes the long form `vk[(vk == 49) | (vk == 99)]` and then shows the compact `isin` equivalent. For a long whitelist, `isin` is much cleaner than a string of equalities.

`apply` runs a function on every element:

The first function takes each actor's full name, splits it on spaces, takes the first token, and uppercases it — so `Vicky Kaushal` becomes `VICKY`. The result is a new Series with the SAME index. The second function classifies each day: above the average subscriber count yields `'good day'`, otherwise `'bad day'`.

Transformations like these would otherwise require an explicit loop; `apply` turns them into a single expressive expression.

PYTHON

```
vk[vk.isin([49, 99])]
```

PYTHON

```
movies.apply(lambda x: x.split()[0].upper())
subs.apply(lambda x: 'good day' if x > subs.mean() else 'bad day')
```

16.2 Duplicates and missing values

The cleaning workflow begins here, and the code runs two complementary families on small demo Series.

`drop_duplicates(keep='last')` removes repeated values, keeping the LAST occurrence of each — the output keeps the second 1, the second 2, and so on. `temp.duplicated()` returns a boolean Series that marks every repeat, and `.sum()` counts the True values: 4 duplicates in the demo, and 137 in the real batting data.

Then the code builds a Series with three missing entries:

`temp.size` reports all 10 elements, but `temp.count()` reports 7 — size includes missing values, count skips them. `temp.isnull().sum()` reports exactly 3, the number of NaNs. `temp.dropna()` removes the missing rows entirely, and `temp.fillna(temp.mean())` replaces each NaN with the mean of the valid values, here 5.0.

The takeaway is a pipeline: detect gaps with `isnull`, delete them with `dropna`, or patch them with `fillna`, and always remember that size and count answer different questions about the same column.

PYTHON

```
temp = pd.Series([1, 1, 2, 2, 3, 3, 4, 4])
temp.drop_duplicates(keep='last')
temp.duplicated().sum()
```

PYTHON

```
temp = pd.Series([1, 2, 3, np.nan, 5, 6, np.nan, 8, np.nan, 10])
```

16.3 astype, between, clip

This trio manages the values' type, range, and bounds.

`astype` converts the dtype:

The example measures memory with the standard `sys` module and shows the payoff of a smaller integer type: the original `int64` Series occupies 3456 bytes, and the `int16` version only 2166. Downcasting is a real optimization when a column's values fit a narrower range.

`between` creates an inclusive range mask:

`vk.between(51, 99)` is True for every score from 51 through 99 inclusive, and the filter counts 43 such innings — the classic between-50-and-100 analysis, written in one word instead of two comparisons.

`clip` clamps the values inside a window:

Every value below 100 becomes 100, and every value above 200 becomes 200; the rest pass through untouched. The output shows the earliest days pinned to 100 and the late-year spikes pinned to 200. Clipping is the standard way to tame outliers before analysis.

Remember one caution with `astype`: narrowing a type can lose precision, so only downcast when the data genuinely fits. These three methods, plus the duplicate and NaN tools, are the raw building blocks of every data-cleaning pipeline you will write.

PYTHON

```
sys.getsizeof(vk)
sys.getsizeof(vk.astype('int16'))
```

PYTHON

```
vk[vk.between(51, 99)].size
```

16.4 `isin` and `apply` in practice

Revisit the notebook's real-data examples and the pattern becomes clear.

`vk[vk.isin([49, 99])]` filters the batting runs to 49 and 99, the annoying near-hundreds. The phrasing reads like English: keep the values that are IN this list. With `and` and `or` equivalents — the `&` and `|` operators — you can fold `isin` into bigger conditions, though every sub-expression needs parentheses.

The actor transformation chains `.split()` and `.upper()` inside a lambda, and it produces a new Series preserving the movie-title index. This is a transformation: same labels, changed values. The classification example shows `apply` hitting a different purpose — turning continuous numbers into categories by comparing each day's value to the overall mean.

A performance note is worth remembering even at this stage: `apply` invokes a Python function once per element, so it is slower than a vectorized operation that pandas can execute natively. Use `isin` and comparisons for filtering whenever they express the question; reach for `apply` when the logic genuinely needs a custom Python function, such as parsing text or combining several pieces of a row. The two methods are complementary — `isin` decides which rows to keep, `apply` decides what each kept value becomes.

16.5 A cleanup pipeline for missing values

The missing-value demo Series is a miniature of real data: integers, gaps, and a decision to make. The tools arrive in order of aggression.

`drop_duplicates` and `deduplicated` deal with repeats. The demo keeps all four distinct values and drops the extra copies, and `vk.duplicated().sum()` quantifies how many rows repeat in the batting data.

`keep='first'`, `keep='last'`, and `keep=False` (remove ALL copies) control which occurrence survives.

Missing values get a family of their own. `isnull()` returns True for each missing slot, and chaining `.sum()` turns that into a count of holes. `dropna()` deletes every row containing a NaN. `fillna(value)` patches holes with a chosen value — a constant, the mean, the median, or even a neighbour's value via methods like `bfill` (backfill, copying the next valid value) and `ffill` (forward fill, copying the previous one).

The distinction between `size` and `count` is the hidden trap: size reports everything including NaNs, count reports only valid values. When someone shows you 10 rows but count says 7, the difference of 3 is your missing-value report.

For the subscriber and cricket datasets in this notebook, the strategy is simple: inspect with `isnull().sum()`, then decide — delete the holes if they are few and unimportant, fill them with a statistic if the column is essential. Writing the cleanup as a pipeline keeps it readable and repeatable for future loads of the same file.

16.6 Memory and range control

This final trio is about keeping numeric data honest.

The memory comparison in the cells is worth re-reading line by line. `sys.getsizeof(vk)` measures the Python object overhead of the int64 Series at 3456 bytes; `sys.getsizeof(vk.astype('int16'))` measures the int16 version at 2166 bytes. Each element of int16 needs half the bytes of int32 and a quarter of int64, so on wide datasets dtype-tuning produces dramatic savings.

`between(51, 99)` is a filter that needs no comparison operators: it is inclusive on both ends, so a score of exactly 51 or 99 matches. Work through the equivalence once: `vk[vk.between(51, 99)]` means exactly the same as `vk[(vk >= 51) & (vk <= 99)]`. When you see `between` in someone's code, you are seeing the friendly face of two joined comparisons.

`clip(100, 200)` bounds every value into the interval. Below-trend days become 100, extreme spikes become 200, and everything in between stays untouched. It differs from `between` in a crucial way: clip CHANGES the values, whereas between only SELECTS rows. Filter the data with between; clamp the data with clip.

One warning accompanies downcasting: narrowed types can overflow or lose precision if a value exceeds the range, so profile the data first (min and max) before shrinking the dtype. With that check in place, `astype`, `between`, and `clip` give you precise control over type, range, and bounds.

PYTHON

```
# astype
# between
# clip
# drop_duplicates
# isnull
# dropna
# fillna
# isin
# apply
# copy
```

PYTHON

```
import numpy as np
import pandas as pd
```

Summary

In this session you built the Series from the ground up: a one-dimensional labeled array — a single column with values on the right and index labels on the left. You created Series from lists, custom index and name pairs, dictionaries, and CSV files via `readcsv` with *squeeze* and *indexcol*, and inspected them with `size`, `dtype`, `name`, `is_unique`, `index`, and `values`.

You computed the statistical toolkit of `count`, `sum`, `mean`, `median`, `mode`, `std`, `var`, and `describe`; indexed, sliced, and edited values by position and label; and used `len`, `dict`, `membership`, and `broadcasting`. Boolean masks answered questions such as how many fifties and ducks a batsman scored, and one-line plots turned numbers into charts. The important methods — `isin`, `apply`, `astype`, `between`, `clip`, `drop_duplicates`, `isnull`, `dropna`, `fillna`, and `copy` — formed a complete cleanup pipeline. The Series is now your basic labeled unit.

In this session you turned the one-dimensional Series into the two-dimensional DataFrame. You built tables from lists of lists, from dictionaries of lists, and directly from CSV files with `read_csv`, and you inspected them with `shape`, `dtypes`, `index`, `columns`, `values`, `info`, and `describe` - adding `isnull`, `uplicated`, and `rename` to the health check.

You ran maths across rows and columns with the `axis` parameter, selected single columns into Series and several into DataFrames, and selected rows by position with `iloc` and by label with `loc`, including two-dimensional windows. Boolean masks filtered rows with combined conditions - the pandas WHERE clause. New columns arrived as constants or derived values, `astype` and `category` slimmed memory, and `value_counts` produced instant frequency reports.

This session opened your DataFrame toolkit method by method. *valuecounts built instant frequency tables, sortvalues and sortindex rearranged rows, rank produced leaderboards, and setindex, resetindex, and rename managed the labels. unique and nunique summarised distinct values, while isnull, notnull, and hasnans pinpointed missing data — removed with dropna and patched with fillna, and duplicates handled by distinct and dropduplicates.*

`drop` removed unwanted rows and columns, `apply` ran custom functions element-wise or row-wise, `isin` filtered against a whitelist, and `corr` measured the relationship between numeric columns. `nlargest` and `nsmallest` extracted the top and bottom performers, `insert` placed a column at any position, and `copy` guaranteed safe, independent snapshots. The through-line to remember: most methods return new objects and leave originals untouched; pass `inplace` where permanence is wanted. With this toolbox on hand, you are ready to split data into groups and recombine it — the GroupBy session is next.

This session turned GroupBy — split-apply-combine — into a tool you can reach for without thinking. You split a DataFrame by one column with `movies.groupby('Genre')` and met the lazy GroupBy object, then applied the built-in aggregations `sum`, `mean`, `std`, and `count`, chained with *sortvalues and head to build leaderboards. Attributes and methods such as len, size, nunique, nth, getgroup, groups, describe, and sample* let you inspect the grouping structure itself. The `agg` method ran many aggregations in one call using dictionary and list syntax, and looping unpacked (`groupname`, `groupdataframe`) pairs for manual work. `apply` ran custom functions per group — counting titles, ranking movies inside their genre, and normalising ratings. Grouping

on multiple columns produced tuple keys and MultiIndex results, answering director-actor and actor-genre questions. The cricket exercise tied it all together: top run-scorers, sixes, death-over hitters, per-opponent records, and a reusable highest-score function. GroupBy is now your default answer for every 'how many / top N / average per category' question.

This session added the combining tools to your pandas toolkit. `pd.concat` stacked the two monthly registration files into one with `ignore_index=True`, recorded their origin with `keys=`, and even placed tables side by side with `axis=1`. `merge` then joined tables on common keys in all four flavours — inner, left, right, and outer — including the *lefton/righton* spelling for differently-named keys and the self-join for pairing partner records. Merging into groupby pipelines answered the business questions directly: total revenue by joining registrations to prices, month-by-month revenue by combining concat with groupby, courses with no enrolments and students with none via `setdiff1d` and `isin`, and top-N leaderboards by joined groupby counts. The IPL case study closed the loop, merging ball-by-ball deliveries with match summaries to rank stadiums by sixes per match and to find every season's orange cap holder. The rule to carry forward: merge when you need another row's information, concat when you need more rows or columns.

This session taught you to treat indexes and columns as buildable hierarchies, not fixed labels. A **MultiIndex** — created with `from_tuples` or `from_product` — gives a Series or DataFrame multiple key levels, so `s['cse']` returns a whole sub-series and `(branch, year)` pairs label rows precisely. `unstack` unfolds an inner index level into columns (Series → DataFrame), while `stack` folds columns back into the index. MultiIndex **DataFrames** extend the idea to both axes, and selection follows naturally: `loc` with tuples and level-slices, `iloc` with pure positions, `sort_index(level=...)` for ordering, `transpose` and `swaplevel` for rotating the hierarchy. Finally, **wide vs long** shapes were reconciled: `melt` flattens wide attribute columns into tidy rows (demonstrated on COVID case data), and `pivot_table` re-groups data into row × column summary grids with customisable `aggfunc` and `margins` — turning a raw expense log into plotted month-by-category insights.

This session turned text wrangling from a slow, crash-prone loop into fast, vectorised expressions. You saw why NumPy-style vectorisation wins: one operation over the whole column instead of a Python loop, and you saw the classic failure — `None.startswith(...)` raises `TypeError` — that vanilla Python cannot survive. Pandas' **.str accessor** solved it: `s.str.startswith('c')` and `s.str.upper()` run across the column and merely propagate missing values. Using Titanic names you converted case with `upper/lower/capitalize/title`, measured length with `.str.len()`, stripped whitespace, exploded `Lastname, Title, Firstname` into separate columns via `split(...).str.get(...)` and `expand=True`, and counted the extracted titles. Cleaning continued with `str.replace` (merging Ms./Mlle. into Miss.), and filtering joined the toolkit through `startswith`, `endswith`, and `isdigit` boolean masks. Finally, **regular expressions** arrived: `contains('john', case=False)`, alternation with `|`, anchored consonant-consonant surname patterns, and element-wise string slicing with `[::-1]` — a complete vocabulary for searching and tidying any text column.

This session built Pandas' complete time stack from a single moment up to plotted trends. A **Timestamp** is a single point in time, parsing strings like `'5th January 2023 9:21AM'` and exposing `.year`, `.month`, `.day`, `.hour`, and more. You learned *why* `datetime` is not enough: Python objects are slow and bulky, while NumPy's `datetime64` stores dates as compact integers and adds days with `date + np.arange(12)`; Pandas' Timestamp fuses the two. Grouping timestamps produced a **DatetimeIndex**, which serves as time row-labels on Series and DataFrames. `date_range` generates whole calendars — daily (`D`), business (`B`), weekly (`W-`

`THU`), hourly (`6H`), and month/year-end (`M`, `MS`, `A`) — while `to_datetime` converts messy string columns safely, with `errors='coerce'` turning undecipherable entries into `NaT`. Finally the `.dt` accessor extracted month names, day names, and quarter flags from an expense column, and `groupby` on those labels produced weekday and monthly spend charts — a clean, reusable template for any date-stamped dataset.

This case study tied the whole Pandas time toolkit into one workflow. A **Timestamp** captures a single moment, a **DatetimeIndex** turns dates into row labels, `date_range` fabricates date sequences, and `to_datetime` parses messy string columns. **Timedelta** expresses the duration between moments — used here to compute realistic delivery times. Setting the date as the index enabled **partial indexing** (`yt.loc['2022']`) and derived labels like month/day names and quarters, which fed straight into **visualization** with automatic time axes and bar summaries. **asfreq** forced data onto a regular calendar grid, **resample** aggregated daily values into periods (down) or interpolated new points (up), **rolling** smoothed volatility with a moving average, and **shift** created lag features. You turned two mundane CSV files into a story about seasonal patterns, trends, and behaviour — exactly the kind of transformation real analysts perform every day.

Pandas DataFrame

1.1 From lists, dicts, and CSV

The very first cells build a table three different ways, and each tells you something about what a DataFrame really is.

The list-of-lists route treats each inner list as a row. `pd.DataFrame(...)` wraps them into a table with the automatic 0-to-n row index, and the `columns=` parameter names each column — iq, marks, package. Without it, the columns would be anonymous 0, 1, 2.

The dictionary route reads more like a real table:

Here every key becomes a column name and the list under it becomes that column's values. This is the preferred construction style because the column names are written right next to their data. Notice that `students.set_index('name', inplace=True)` then promotes the name column into the row index, which is what the notebook output shows — students are later looked up by their names exactly like dictionary keys.

Finally, the CSV route is the one you will use most in practice: `movies = pd.read_csv('movies.csv')` and `ipl = pd.read_csv('ipl-matches.csv')` load entire files into DataFrames in a single call, headers and all.

PYTHON

```
student_data = [[100, 80, 10], [90, 70, 7], [120, 100, 14], [80, 50, 2]]
pd.DataFrame(student_data, columns=['iq', 'marks', 'package'])
```

PYTHON

```
student_dict = {'name': [...], 'iq': [...], 'marks': [...], 'package': [...]}
students = pd.DataFrame(student_dict)
```

1.2 Deep dive — creating tables

Read the three construction styles as three levels of control over your table.

A list of lists gives you full positional control: you decide column order through the `columns=` list, and each inner list is one row. It is verbose to type but perfect when data comes from an algorithm or a parsed structure. The printed table shows the columns you named and rows indexed 0, 1, 2, 3.

A dictionary of lists is the most readable: the words `iq`, `marks`, and `package` appear once beside their data, not once per row. The code can then attach a meaningful row index with `set_index('name', inplace=True)`, converting the name column into the label axis. That single line turns the DataFrame into a lookup table where `students.loc['nitish']` retrieves a whole row by name. Because the operation runs `inplace=True`, the change is permanent on the `students` object — the printed output shows the name column as the left-hand index.

A list of file paths — `pd.read_csv('movies.csv')` and `pd.read_csv('ipl-matches.csv')` — is the real world. The movies file loads 1629 rows with 18 columns; the IPL file loads 950 match rows with 20 columns. In every case the result is the same structure: a two-dimensional table of rows and columns where each column is a Series sharing one index.

Whatever the source, the mental model never changes: a DataFrame is rows on one axis, columns on the other, and a name attached to every column. All the power of pandas flows from that simple contract.

PYTHON

```
# using lists
student_data = [
    [100,80,10],
    [90,70,7],
    [120,100,14],
    [80,50,2]
]

pd.DataFrame(student_data,columns=['iq','marks','package'])
```

PYTHON

```
# using dicts

student_dict = {
    'name':['nitish','ankit','rupesh','rishabh','amit','ankita'],
    'iq':[100,90,120,80,0,0],
    'marks':[80,70,100,50,0,0],
    'package':[10,7,14,2,0,0]
}

students = pd.DataFrame(student_dict)
students.set_index('name',inplace=True)
students
```

2.1 shape, dtypes, index, columns, values

When a fresh DataFrame lands in memory, the first reflex is to interrogate its structure. The attributes in this section answer five basic questions.

`shape` returns the dimensions as a `(rows, columns)` tuple. `movies.shape` gives `(950, 20)` — wait, that is the IPL frame's shape; the movies frame reports (1629, 18). Try not to fixate on the numbers themselves and instead make shape your habitual opening move: rows first, columns second.

`dtypes` prints the dtype of every column in one go. The IPL output shows ID as int64 and nearly everything else — City, Date, Season, MatchNumber, Team1, Team2, Venue — as object, because these columns hold text. Spotting an object column where numbers should be is the fastest way to catch a parsing problem.

`index` and `columns` report the two label axes. The default index is a RangeIndex(start=0, stop=950, step=1), and `columns` returns the Index of names — `Index(['iq', 'marks', 'package'], dtype='object')` in the student frame. Knowing your columns is a prerequisite to selecting them.

`values` returns the entire table as a two-dimensional NumPy array with the labels stripped away. The IPL output shows rows of raw data — match IDs, team names, umpire names — which is exactly what you pass to a numpy-only library when you need it.

2.2 info, describe, isnull, duplicated, rename

Attributes report structure; methods report health. This section runs the standard doctors' checklist.

`movies.info()` prints a compact summary: the class, the RangeIndex bounds, the total column count, and a per-column table of non-null counts and dtypes. The movies frame reports 1629 non-null values in most columns and a handful of columns with gaps — a perfect first warning that cleaning will be needed.

`ipl.info()` does the same for the cricket data and reveals, for example, that City has 899 non-null entries out of 950.

`movies.describe()` computes the statistical summary for every numeric column at once — count, mean, std, min, percentiles, max. The output for *yearofrelease*, *imdbrating*, and *imdbvotes* gives an instant sense of the data's centre and spread.

`movies.isnull().sum()` counts the missing values column by column, and `movies.duplicated().sum()` counts the duplicate rows — 0 for movies, but 1 for the students frame, meaning one student row is a full repeat.

`students.rename(columns={'marks': 'percent', 'package': 'lpa'}, inplace=True)` relabels columns before later cells use the friendlier names percent and lpa. Preview with head, tail, and sample, check health with info, describe, isnull, and duplicated, and relabel with rename — that is the entire inspection ritual, front to back.

PYTHON

```
# shape
movies.shape
ipl.shape
```

PYTHON

```
# dtypes
movies.dtypes
ipl.dtypes
```

3.1 The axis parameter

The maths methods learned for Series return a single scalar, because a Series has only one direction to collapse. A DataFrame has two, and the `axis` parameter chooses which one.

`axis=0` works DOWN the rows. The sum is computed per column, collapsing each column into one number, and the result is a Series indexed by column name. In the output, `sum(axis=0)` returns iq 390, percent 300, lpa 33 — three column totals. This is the default, so `students.var()` without an axis also works column-wise and returns one variance per column.

`axis=1` works ACROSS the columns. The mean is computed per row, collapsing each row into one number, and the result is a Series indexed by the row labels. Output shows a per-student average: row 0 near 63.33, row 1 near 55.67, row 2 at 78.0, and so on.

A solid memory hook: `axis=0` is vertical, so it drops the rows and leaves per-column results; `axis=1` is horizontal, so it drops the columns and leaves per-row results. Think of axis as the direction along which pandas moves, adding up everything it passes.

PYTHON

```
students.sum(axis=0)
students.mean(axis=1)
students.var()
```

3.2 sum, mean, and var in context

Run the three computations against the actual output and the axis idea becomes concrete.

`sum(axis=0)` totals each column: the iq column adds to 390, percent to 300, lpa to 33. You can read the result as 'the whole-class totals for every attribute'.

`mean(axis=1)` averages each row: student 0 (100, 80, 10) averages to 63.33, student 2 (120, 100, 14) to 78.0, and the two students with all-zero rows average to 0.0. This is 'the average attribute score for each student' — a per-person summary.

`var()` (default axis 0) measures spread per column: iq has variance 2710.0, percent 1760.0, lpa 33.5. The high iq variance tells you class intelligence scores vary widely, while package variance is comparatively small.

Every aggregate — sum, mean, min, max, var, std — follows identical axis rules, and pandas automatically operates only on numeric columns when you run them, skipping text. Missing values are skipped by default (skipna), so a NaN in a column does not poison the total unless you ask for it. Sums for totals, means for centres, variances for spread: you now have the vocabulary to summarise any numeric part of a table.

PYTHON

```
# sum -> axis argument
students.sum(axis=0)
```

PYTHON

```
students.mean(axis=1)
```

4.1 One column vs many

Column selection is the most common DataFrame operation, and pandas splits it into two distinct syntaxes.

Passing a single string in square brackets returns ONE column as a Series. The output shows the movie titles vertically with their index on the left, or the IPL venues — the Series keeps the DataFrame's row index, so you can still tell which row each value came from.

Passing a LIST of strings returns a new DataFrame containing exactly those columns, in the order you listed them. Nothing else is filtered — every row is present, and the result keeps the same index.

The rule of thumb: single brackets + string = Series; double brackets + list = DataFrame. The difference is the same as the difference between asking for one column of a table and asking for a smaller table. This distinction matters because a Series and a DataFrame support different subsequent operations, so knowing what you got back shapes what you can do next.

PYTHON

```
movies['title_x']  
ipl['Venue']
```

PYTHON

```
movies[['year_of_release', 'actors', 'title_x']]  
ipl[['Team1', 'Team2', 'WinningTeam']]
```

4.2 Deep dive on column selection

The examples in the notebook give you the full column-selection toolkit.

`movies['title_x']` returns the movie titles as a Series — one value per row, labels intact. `ipl['Venue']` does the same for the stadium names, and its output stretches over all 950 matches.

The multi-column examples build smaller DataFrames: `movies[['year_of_release', 'actors', 'title_x']]` gathers the year, the actor list, and the title side by side; `ipl[['Team1', 'Team2', 'WinningTeam']]` prepares a compact result frame. Column ORDER follows your list, and missing column names raise a `KeyError` — a helpful guard against typos.

Three extra facts complete the picture. First, the dot notation `ipl.Venue` also selects a column, but it can collide with DataFrame's own attribute names and reserved words, so brackets are preferred. Second, column selection is how you prepare inputs for analysis: grab the columns your question needs, then work on the slice. Third, creating brand-new columns uses the same bracket syntax on the left-hand side — `df['new'] = ...` — which the session demonstrates with the Country and lead actor columns. If you can comfortably choose one column or many, you have unlocked the door to every more advanced pandas pattern that follows.

PYTHON

```
# single cols  
movies['title_x']
```

PYTHON

```
ipl['Venue']
```

5.1 Position with `iloc`, label with `loc`

Rows are selected with two accessors that make the POSITION-vs-LABEL choice explicit.

`iloc` works by integer POSITION, exactly like list indexing. `movies.iloc[5]` returns the whole 6th row (position 5) as a Series with columns as its index. `movies.iloc[:5]` slices the first five rows into a DataFrame, and `movies.iloc[[0, 4, 5]]` grabs rows at positions 0, 4, and 5 — fancy indexing. Positions are 0-based and the slice's stop is exclusive, matching NumPy and Python lists.

`loc` works by INDEX LABEL. `students.loc['nitish']` pulls out the row whose name label is nitish. `students.loc['nitish':'rishabh':2]` slices from the nitish label to the rishabh label, stepping by two, and — crucially — label slices are INCLUSIVE of the end label, unlike positional slices. `students.loc[['nitish', 'ankita', 'rupesh']]` selects three rows by their labels in one go.

The decision rule: use `iloc` when you know the row number, `loc` when you know the row's name.

PYTHON

```
movies.iloc[5]
movies.iloc[:5]
movies.iloc[[0, 4, 5]]
```

PYTHON

```
students.loc['nitish']
students.loc['nitish':'rishabh':2]
students.loc[['nitish', 'ankita', 'rupesh']]
```

5.2 Deep dive on row access

Run through the actual outputs to cement the two accessors.

`movies.iloc[5]` returns a Series titled `Soni (film)`: `titlex`, `imdbid`, `poster_path`, and every other column becomes a label in that Series, with that movie's values under them. A single-row `iloc/loc` call therefore returns a Series; slices and lists return DataFrames.

`movies.iloc[:5]` and `movies.iloc[[0, 4, 5]]` return the DataFrame rows 0-4 and rows 0, 4, 5 respectively. `students.iloc[[0, 3, 4]]` shows the positional way to grab nitish, rishabh, and amit — those rows happen to sit at positions 0, 3, 4.

The `loc` examples use the name-based index set earlier. `students.loc['nitish']` shows the single student's `iq`, `marks`, and `package`. `students.loc['nitish':'rishabh':2]` keeps only the labels nitish and rupesh because stepping by 2 over four names skips ankit and rishabh. `students.loc[['nitish', 'ankita', 'rupesh']]` reorders to exactly the three requested.

Two professional habits close the section. When you need ONE cell fast, `.at[label]` and `.iat[position]` skip pandas' ambiguity checks entirely. And never mix the two styles in one expression — a label looked up positionally, or vice versa, is the classic source of silent wrong answers.

PYTHON

```
# single row
movies.iloc[5]
```

PYTHON

```
# multiple row
movies.iloc[:5]
```

6.1 Two-dimensional selection

The two accessors combine in a single expression when you need a rectangular window: rows first, then columns.

`movies.iloc[0:3, 0:3]` reads as 'rows at positions 0 to 2, columns at positions 0 to 2'. The result is the top-left 3 by 3 corner of the table: the titles, imdb ids, and poster paths of the first three movies. Positional slicing remains half-open, so 0:3 means positions 0, 1, and 2.

`movies.loc[0:2, 'title_x':'poster_path']` uses labels on BOTH axes. Rows 0 to 2 inclusive, columns from *title_x* up to and including *poster_path* — label slices are inclusive at both ends, which is why *poster_path* makes it into this result even though it sits three columns to the right of *title_x*.

The comma is the whole trick: everything before it selects rows, everything after it selects columns. Since the `movies` frame keeps its original `RangeIndex`, both the positional and label styles happen to pick the same window here — a coincidence that disappears the moment you set a custom index. The result of either call is a small `DataFrame` containing exactly the intersecting cells.

PYTHON

```
movies.iloc[0:3, 0:3]
movies.loc[0:2, 'title_x':'poster_path']
```

6.2 Deep dive on two-dimensional selection

These two calls look nearly identical and land on the same window — and that is precisely why they are a great teaching pair.

`iloc[0:3, 0:3]` is pure position. The code does not care what the columns are called; it takes the first three rows and the first three columns (positions 0 to 2). If the column order ever changes, this slice silently grabs different columns.

`loc[0:2, 'title_x':'poster_path']` is pure label. It starts from whatever label any integer 0, 1, 2 actually names in the index, and it walks the columns by NAME from *title_x* to *poster_path*. If you reordered the columns, the same labels would be found wherever they moved.

Both styles accept the full range of selectors you already know: single values, slices, step slices, and lists. So `movies.iloc[[0, 5], [1, 3]]` (positional lists) and `movies.loc[0:2, ['title_x', 'imdb_id']]` (label list) work exactly as expected.

Chopping a precise two-dimensional window out of a table is the heart of data preparation — feed the window to a chart, a model, or another analysis step. Choosing `iloc` or `loc` comes down to one question: do you trust positions or names? Use whichever the data makes trustworthy, and the same rectangular-window logic will serve you through the whole course.

PYTHON

```
movies.iloc[0:3,0:3]
```

PYTHON

```
movies.loc[0:2,'title_x':'poster_path']
```

7.1 Boolean masks

Column selection picks columns; filtering picks ROWS by a condition. The building block is the boolean mask.

`ipl['MatchNumber'] == 'Final'` compares every row's MatchNumber against the string 'Final' and produces a boolean Series — True where the match was a final, False otherwise. Passing that mask inside the brackets keeps only the True rows. The subsequent line, `new_df[['Season', 'WinningTeam']]`, selects two columns from the filtered frame, and the combined output lists every final with its season and winner.

The same one-liner style covers multiple conditions with the boolean operators `&` (and), `|` (or), and `~` (not):

Parentheses around EVERY condition are mandatory — without them, Python's operator precedence pulls the comparisons apart. The result is the pandas equivalent of SQL's WHERE clause.

PYTHON

```
mask = ipl['MatchNumber'] == 'Final'
new_df = ipl[mask]
```

PYTHON

```
ipl[(ipl['City'] == 'Kolkata') & (ipl['WinningTeam'] == 'Chennai Super Kings')]
movies[(movies['imdb_rating'] > 8.5) & (movies['vote_count'] > 10000)]
```

7.2 Deep dive on filtering

The notebook answers five real cricket and cinema questions with masks; trace each one.

`ipl[ipl['MatchNumber'] == 'Final'][['Season', 'WinningTeam']]` collapses the 'find all final winners' question into one expression: filter, then choose columns. `ipl[ipl['SuperOver'] == 'Y'].shape[0]` counts the super-over finishes — 14. `ipl[(ipl['City'] == 'Kolkata') & (ipl['WinningTeam'] == 'Chennai Super`

`kings'))].shape[0]` reports 5 wins. The toss question compares two columns at once: `ipl[ipl['TossWinner'] == ipl['WinningTeam']]` keeps rows where the toss-winner and the match-winner are the same team, and the cell computes that proportion as 51.47 percent.

The cinema filter is the strictest: `movies[(movies['imdb_rating'] > 8.5) & (movies['imdb_votes'] > 10000)].shape[0]` returns 0 — no bollywood movie meets both bars, which is itself a finding.

The Action-genre question shows how masks compose with string tools: `mask1 = movies['genres'].str.contains('Action')` flags genres containing the word Action, `mask2 = movies['imdb_rating'] > 7.5` sets the rating bar, and `movies[mask1 & mask2]` returns the intersection. A commented alternative splits genres and checks membership with `apply` — same goal, two routes. Storing masks in named variables keeps multi-condition filters readable.

PYTHON

```
# find all the final winners
mask = ipl['MatchNumber'] == 'Final'
new_df = ipl[mask]
new_df[['Season', 'WinningTeam']]

ipl[ipl['MatchNumber'] == 'Final'][['Season', 'WinningTeam']]
```

PYTHON

```
# how many super over finishes have occurred
ipl[ipl['SuperOver'] == 'Y'].shape[0]
```

8.1 Constant and derived columns

New columns arrive through assignment, exactly like adding a key to a dictionary.

Assigning a scalar to a brand-new column broadcasts that value to every row: the entire column instantly reads 'India'. This is the constant column — the fastest way to stamp a label onto a whole table.

The derived column computes new values from existing ones. Reading left to right: `movies['actors']` takes the actor list column, `.str` opens pandas' string accessor, `.split('|')` splits each row's text into a list at the pipe character, `.apply(lambda x: x[0])` takes the first element of each list, and the whole expression is assigned to a fresh column named 'lead actor'. The leading actor is now its own column, ready for its own filtering and counting.

Notice that `movies.dropna(inplace=True)` runs first in the notebook, removing the rows with missing values so the split-based derivation does not trip over empty text. Clean, then derive: that ordering is a real-world habit, not just a convenience.

PYTHON

```
movies['Country'] = 'India'
```

PYTHON

```
movies['lead actor'] = movies['actors'].str.split('|').apply(lambda x: x[0])
```

8.2 Deep dive on new columns

The three-column workflow in this section tells a complete story about feature engineering.

First, the constant: `movies['Country'] = 'India'` produces a column where every one of the 298 remaining movies shares the same value. After the earlier dropna and the head print, the Country column tiles the frame. Constants are trivial, but they matter when a dataset must carry a fixed attribute — the country, the currency, the dataset version.

Second, the derived: the lead-actor column parses the pipe-separated `actors` field with `.str.split('|')` and keeps the first name with `.apply(lambda x: x[0])`. The printed head shows Gully Boy, Yeh Hai India, Article 15, Aiyaary, and Raid with their respective lead actors extracted. The original actors column remains untouched; the new column is added and positioned at the right edge of the table.

Third, verification: `movies.info()` after the additions reports 19 columns (up from 18) and still holds the rows survived by dropna. The info chart is the receipt that the column was really created.

The general pattern — `df['new'] = expression` — applies to all three: a scalar broadcasts a constant, a Series aligns its rows by index, and any per-row computation through `apply` or the `.str` tools constructs values from your existing features. Feature engineering is simply this pattern repeated many times.

PYTHON

```
# completely new
movies['Country'] = 'India'
movies.head()
```

PYTHON

```
# from existing ones
movies.dropna(inplace=True)
```

9.1 astype and category

DataFrames often carry columns whose dtype is far fatter than the data requires. This section shows two ways to slim them down.

The integer match IDs are stored as int64 by default. `astype('int32')` downcasts them to a 32-bit integer, halving the bytes each ID occupies. The `info()` snapshot before and after makes the change visible: the ID row's dtype flips from int64 to int32.

Team names are the bigger win. A column like Team1 repeats the same handful of franchise names thousands of times, and storing each occurrence as a full text string wastes memory. The category dtype stores each unique name ONCE, in a small catalogue, and internally records each row as a compact code pointing at that catalogue. The printed `info()` reflects the change on the Team1 and Team2 rows.

The lesson generalises: assign `df['col'] = df['col'].astype(...)` to convert in place, and reach for category whenever a column holds many repeated strings — team names, cities, genres. The memory savings compound quickly on large data.

PYTHON

```
ipl['ID'] = ipl['ID'].astype('int32')
```

PYTHON

```
ipl['Team1'] = ipl['Team1'].astype('category')
ipl['Team2'] = ipl['Team2'].astype('category')
```

9.2 Deep dive on dtype

The comparison is best seen in the `info()` output, which prints a before-and-after report for the IPL frame.

Initially every column is either `int64` (ID) or `object` (everything else). Object columns are the pandas catch-all for text, but they are the slowest and heaviest dtype available. After the conversions, ID reads `int32`, and Team1 and Team2 read `category` — and the structure of a big notebook like this is exactly where the savings start to matter.

Why does category help so much? Think about the Team1 column: 950 IPL matches, but only a handful of distinct franchise names. A category column keeps each name once in a coding table and stores codes — tiny integers — in the 950 rows. The full names are recovered on demand, and operations like `value_counts` become blazingly fast because pandas can count codes instead of comparing strings.

Category also offers ordering semantics: categories have a natural order that pandas can sort by without alphabetical coercion, and you can even specify the order of categories when you build the dtype. Those details belong to a later session, however. For now, hold two actions in mind: `astype` changes a column's dtype to something narrower, and `category` is the special dtype for repeated labels. Laziness about dtypes is the most common cause of both inflated memory and slow pipelines on real datasets.

PYTHON

```
# astype
ipl.info()
```

PYTHON

```
ipl['ID'] = ipl['ID'].astype('int32')
```

9.3 value_counts for exploration

The section closes with the exploration tool that pairs perfectly with categorical columns.

`value_counts()` counts how many times each distinct value appears and returns a Series sorted descending. Which player won player-of-the-match most often? The method answers it instantly, and the same call works on TossDecision or Season.

The notebook's commented starter questions give you the flavour of real analysis:

- Which player won the most player-of-the-match awards in finals and qualifiers? Filter those rows first, then `value_counts`.
- Plot the toss-decision distribution: `ipl['TossDecision'].value_counts().plot(kind='bar')`.

- Count how many matches each team has played: `ipl['Team1'].value_counts() + ipl['Team2'].value_counts()` — add Team1 and Team2 counts so both home and away appearances are counted.

This is the same `value_counts` you met on Series, now pointed at DataFrame columns. Combined with `astype-category` columns, it becomes the fastest frequency analysis available in pandas: one call turns any categorical column into a sorted popularity report, and a plot call turns that report into a picture. Use it as your standard answer whenever a question begins with the words 'how many of each'.

PYTHON

```
ipl['Player_of_Match'].value_counts()
```

Important DataFrame & Series Methods

1.1 value_counts on Series and DataFrames

The simplest way to explore a categorical column is to count how often each distinct value appears, and `value_counts` does exactly that.

On a Series, `value_counts` returns a NEW Series whose index holds the unique values and whose values hold the counts, sorted in descending order by default. The output says 1 appears 3 times, 2 appears twice, and 3 once — the frequency table of the column.

On a DataFrame the behaviour extends to ROWS: `marks.value_counts()` counts how many times each full row combination repeats. In the demo frame of five rows, the row [80, 70, 14] appears twice and every other row once, which is exactly what the output reports.

The usual companions round out the method: `normalize=True` gives proportions instead of raw counts, `ascending=True` flips the sort, `dropna=False` includes missing values in the table, and a `bins=` argument can even bucket a numeric column into a miniature histogram. For a first look at any categorical column, `value_counts` is the number-one tool.

PYTHON

```
# value_counts
# sort_values
# rank
# sort_index
# set_index
# rename index -> rename
# reset_index
# unique & nunique
# isnull/notnull/hasnans
# dropna
# fillna
# drop_duplicates
# drop
# apply
# isin
# corr
# nlargest -> nsmallest
# insert
# copy
```

PYTHON

```
import numpy as np
import pandas as pd
```

1.2 value_counts in the wild

The notebook then points value_counts at real cricket data:

Read it as a pipeline. First select the rows whose MatchNumber is NOT a pure digit — that keeps exactly the finals, qualifiers, and eliminators, whose names are words. Among those high-pressure games, count the player-of-the-match awards. The output is a frequency table of the heroes of knockout matches: KA Pollard, F du Plessis, and SK Raina leading with three awards each, several bowlers with two, and a long tail of players with one.

Two further cells show valuecounts *feeding a plot and summing across columns*.

`ipl['TossDecision'].value_counts().plot(kind='pie')` renders the toss decisions as a pie chart in one line — pandas turns the count table straight into a picture. And the 'matches per team' question is answered by adding two count tables: `ipl['Team2'].value_counts() + ipl['Team1'].value_counts()` counts how often each team appears on either side of a match, and `sort_values(ascending=False)` puts the most-frequent franchise — Mumbai Indians — on top. Counting, plotting, and combining columns: valuecounts powers all three.

PYTHON

```
ipl[~ipl['MatchNumber'].str.isdigit()][ 'Player_of_Match' ].value_counts()
```

1.3 Deep dive — frequency analysis

The toy example is worth one more pass because it shows the full shape of the result.

`value_counts` returns a Series: the index is the set of unique input values, and the values are their counts. This is why you can chain a right-hand operation directly — `a.value_counts().plot(kind='bar')` plots each unique value on the x-axis with its count as the bar height, and `.sort_index()` reorders the bars by value rather than by frequency.

The dataframe variant counts identical full rows, which is the SQL-equivalent of a GROUP BY over every column in the frame. In the marks demo, two students scored exactly [80, 70, 14], so that combination earns a count of 2 while the other rows each stand alone.

Remember that `value_counts` by default EXCLUDES NaN and sorts descending. If a category seems to have vanished from the output, that is the missing-value exclusion, and `dropna=False` will bring it back as an explicit row. This one method—a count of every distinct value, already sorted — is the closest pandas comes to instant statistics for a categorical column, and nearly every 'how many of each' question in data analysis starts with a call to it.

PYTHON

```
a = pd.Series([1,1,1,2,2,3])
a.value_counts()
```

PYTHON

```
# value_counts(series and dataframe)

marks = pd.DataFrame([
    [100,80,10],
    [90,70,7],
    [120,100,14],
    [80,70,14],
    [80,70,14]
],columns=['iq','marks','package'])

marks
```

2.1 Sorting Series and DataFrames

Sorting transforms the familiar into something comparable. `sort_values` orders data by its VALUES, on a Series or on chosen columns of a DataFrame.

The Series demo sorts descending, so 89, 56, 14, 12, 1 fall into order, each keeping its original index label — the output still shows row 4 for 89 and row 0 for 12.

On a DataFrame you name the sort keys:

`sort_values('title_x')` reorders the whole movie table by title. With two keys you get a tie-breaker sort: first by year ascending, then within the same year by title descending — like sorting a contacts list by surname and then by first name. The `ascending` parameter accepts either a single boolean or a list matching the column list, and `na_position='first'` (seen with the students frame) forces missing values to the top.

By default `sort_values` returns a newly sorted copy. Add `inplace=True` and the original object is reordered permanently instead.

PYTHON

```
x = pd.Series([12, 14, 1, 56, 89])
x.sort_values(ascending=False)
```

PYTHON

```
movies.sort_values('title_x', ascending=False)
movies.sort_values(['year_of_release', 'title_x'], ascending=[True, False])
```

2.2 Sorting with `na_position` and `inplace`

The students example shows both parameters in one call:

The students frame deliberately contains four rows with NaN names. `na_position='first'` pushes those missing-name rows to the top of the result; `ascending=False` then orders the named students from z to a — rupesh before rishabh, then mrityunjay, and so on down to aditya. `inplace=True` makes the new order permanent, which the following cell confirms by printing the rearranged students table.

`na_position` has only two legal values, 'first' and 'last', and it addresses the classic question: where do missing keys go when sorting? By default they sink to the bottom; with 'first' they rise to the top. This matters whenever a column has gaps — a sort of the full frame would otherwise bury those rows where you least expect them.

The multi-column movie sort demonstrates the second parameter being a list:

Pandas sorts primarily by `yearofrelease` (ascending), and within every equal year it sorts by `title_x` (descending), so the newest-released titles appear first and within a year the titles run from z to a. Whether you sort one column or several, the key question is always the same: which column drives the order, and should missing values lead or trail?

PYTHON

```
x.sort_values(ascending=False)
```

PYTHON

```
movies = pd.read_csv('movies.csv')
movies.head(4)
```

3.1 Assigning ranks

Sorting reorders rows; ranking attaches position numbers. The `rank` method turns values into standings.

The batsman data holds each IPL batter's career runs. `batsman['batsman_run'].rank(ascending=False)` asks pandas to rank the runs column so that the HIGHEST run total receives rank 1 — the parameter name reads 'give rank 1 to the largest value'. The result is stored in a brand-new column, `batting_rank`.

When the sorted frame is printed, the rank column runs 1, 2, 3, 4, 5 down the page: V Kohli at 6634 runs sits at rank 1, S Dhawan at 6244 at rank 2, DA Warner at 5883 at rank 3, RG Sharma at 5881 at rank 4, SK Raina at 5536 at rank 5. A leaderboard in one expression.

Ties deserve care: by default rank uses the method='average', so tied values share the average of their positions — a pair tied for positions 6 and 7 would both receive 6.5. The `method` parameter offers 'min', 'max', 'first', and 'dense' for other tie policies, and `pct=True` converts ranks into percentages. When two batsmen share a total, decide the tie policy before you read anything else into the ranking column.

PYTHON

```
batsman['batting_rank'] = batsman['batsman_run'].rank(ascending=False)
batsman.sort_values('batting_rank')
```

PYTHON

```
marks = {
    'maths':67,
    'english':57,
    'science':89,
    'hindi':100
}

marks_series = pd.Series(marks)
marks_series
```

4.1 sortindex, setindex, reset_index

The index is the identity of a DataFrame, and three methods control how it is ordered, promoted, and demoted.

`sort_index` orders rows by their LABELS instead of their values. On the marks Series the labels are the subject names, so sorting descending puts science, maths, hindi, english in order. On the movies DataFrame the index is the 0-1628 RangeIndex, so sorting it descending effectively reverses the file's row order.

`set_index('batter')` SWAPS a column into the index — the batter names become row labels, so the frame can be looked up by player. `reset_index` undoes it, turning the current index back into an ordinary column and restoring the 0-to-n RangeIndex. The combined one-liner

`batsman.reset_index().set_index('batting_rank')` performs both at once: flatten the current multi-column index, then promote batting_rank to be the new row label. Two methods in one breath give you complete control over which column names the rows of your table.

PYTHON

```
marks_series.sort_index(ascending=False)
movies.sort_index(ascending=False)
```

PYTHON

```
batsman.set_index('batter', inplace=True) # column -> index
batsman.reset_index(inplace=True)        # index -> column
batsman.reset_index().set_index('batting_rank')
```

4.2 setindex and resetindex in action

The notebooks cells walk the index lifecycle in detail, and tracking the printing at each step is the correct way to understand it.

After `batsman.set_index('batter', inplace=True)`, the output stops showing a 0-to-n range on the left and instead shows batter names — A Ashish Reddy, A Badoni, A Chandila — with `batsmanrun` and `battingrank` to their right. The once-plain index has become a meaningful key.

`batsman.reset_index()` (with `inplace`) then runs in reverse: the batter names move back into a regular column and the range index returns, exactly as the first output shows. The following print of `batsman` re-shows the frame still with 'batter' as index because `reset_index` came after; read each printed table in the position where pandas printed it, and the sequence tells the full story.

The penultimate trick, `marks_series.reset_index()`, shows the Series-to-DataFrame conversion: a Series, when reset, produces a two-column DataFrame whose first column is named `index` and holds the old labels (maths, english, science, hindi), and whose second column holds the values. Finally, `movies.set_index('title_x', inplace=True)` from the rename section converts the movie title column into row labels so that each row can later be addressed by its title. The rhythm — promote with *setindex*, demote with *resetindex* — is one of the most common data-hygiene cycles in pandas.

PYTHON

```
marks_series.sort_index(ascending=False)
```

PYTHON

```
movies.sort_index(ascending=False)
```

5.1 rename, unique, nunique

Three quick methods modernise column names and summarise the distinct values inside them.

`rename` takes a dictionary of old-name to new-name pairs. With `columns=`, the keys are column names; with `index=`, the keys are row labels. The first call permanently shortens two header names to `imdb` and `link`; the second (without `inplace`) returns a renamed copy where two long movie titles become `Uri` and `Battalion`, leaving the original untouched. Rename is documentation you can run.

`unique()` returns the array of distinct values in order of first appearance — for this Series, six entries including one NaN. `nunique()` returns the COUNT of distinct values, and by default it SKIPS NaN — hence 5, one less than `len(unique())`. `nunique(dropna=False)` would count the missing value too. On a DataFrame, `nunique` returns per-column counts: how many distinct seasons are in `ipl`? The cells report 15 — fifteen IPL seasons in the matches file.

PYTHON

```
movies.rename(columns={'imdb_id':'imdb','poster_path':'link'},inplace=True)
```

PYTHON

```
movies.rename(index={'Uri: The Surgical Strike':'Uri','Battalion 609':'Battalion'})
```

6.1 isnull, notnull, hasnans

Before deciding what to DO with missing values, you have to find them. Three helpers make the detection explicit.

`isnull()` returns a boolean mask — True exactly where the value is missing. Wrapping it in square brackets filters the Series to just the missing rows; the output shows the four NaN-name students. `notnull()` is the logical opposite, and its mask keeps precisely the students who HAVE a name.

`hasnans` is a single-boolean question: does this Series contain any NaN at all? Here it answers True. On a DataFrame, `isnull()` and `notnull()` return an entire grid of True/False — every cell gets a verdict — which is the full data-quality portrait of the table at once.

The idioms compose into reports: `df.isnull().sum()` counts the missing values per column in one line, and `df[df['col'].isnull()]` extracts the offending rows for a closer look. `isna` is a full alias of `isnull` — both spell the same question. These tools are the gatekeepers of every cleaning pipeline: nothing gets dropped or filled until you have seen the holes clearly.

PYTHON

```
# isnull(series + dataframe)
students['name'][students['name'].isnull()]
```

PYTHON

```
# notnull(series + dataframe)
students['name'][students['name'].notnull()]
```

7.1 dropna and fillna

Once the holes are mapped, two strategies resolve them: delete the bad rows or patch the bad values.

`dropna` removes rows containing missing values. The `how` parameter is the policeman: `how='any'` drops a row if ANY value inside it is NaN (the strictest, largest effect), while `how='all'` drops a row only if ALL of its values are NaN (here, the one fully-empty student row). The `subset` parameter narrows the inspection to specific columns — `subset=['name']` drops a row only when the name is missing, and `subset=['name','college']` drops when EITHER of those two is missing. With `inplace=True` the deletions are permanent.

`fillna` replaces gaps instead of deleting them. A constant ('unknown') stamps a fixed label; a statistic (the package mean, 6.428571) fills numeric gaps with a representative value; and `method='bfill'` copies the NEXT valid value backwards into each hole (forward-fill with 'ffill' reverses the direction). Choose drop when the holes are few and unimportant, choose fill when the column matters — and let the data's story decide which fill value is honest.

PYTHON

```
# dropna(series + dataframe) -> how parameter -> works like or
students['name'].dropna()
```

PYTHON

```
students
```

8.1 drop_duplicates and duplicated

Duplicate detection mirrors the missing-value pairing: one method reports, one method removes.

On a Series, `drop_duplicates` keeps the first occurrence of each value and removes later repeats, returning 1, 2, 3, 4 in the positions of their first appearances.

On a DataFrame, `drop_duplicates` removes duplicate ROWS. `keep='last'` keeps the final occurrence of each repeated row — in the marks frame, the second [80, 70, 14] row survives and the first is dropped.

`keep='first'` (the default) and `keep=False` (remove every copy) complete the trio, and a `subset=` argument lets you judge duplicates on specific columns only.

The detection partner is `duplicated()`: it returns a boolean Series marking every repeat, and `.sum()` tallies them. The real-data example — `ipl['all_players'].apply(did_kohli_play)` combined with a City filter — shows the full flow: a user-defined function checks whether Kohli played, then `drop_duplicates(subset=['City', 'didkohli_play'], keep='first')` keeps the earliest Delhi match where he appeared. Duplicates, detected and removed, are a staple of every cleaning job.

PYTHON

```
temp = pd.Series([1,1,1,2,3,3,4,4])
temp.drop_duplicates()
```

PYTHON

```
marks.drop_duplicates(keep='last')
```

9.1 drop rows, drop columns

The precise counterpart to selecting is removing, served by the drop method.

On a Series, `drop(index=[0, 6])` removes the rows at labels 0 and 6. The demo temp Series holds [10, 2, 3, 16, 45, 78, 10], and dropping positions 0 and 6 leaves [2, 3, 16, 45, 78] — both copies of the 10 vanish, because those two positions hold the value 10.

On a DataFrame, `drop(columns=['branch', 'cgpa'])` removes two columns at once, and with `inplace=True` the trimmed table becomes permanent — the later print shows only name, college, and package. Drop rows by label with `index=`; after `set_index('name')`, the drop is by student names, so `drop(index=['nitish', 'aditya'])` removes exactly those players and leaves ankit, rupesh, and the rest. The `axis=1` shorthand `df.drop('col', axis=1)` means the same as `columns=`. Selecting keeps the data you want; dropping deletes what you do not — two senses of the same thought, and drop completes the grammar.

PYTHON

```
temp.drop(index=[0,6])
```

PYTHON

```
students
```

10.1 apply on Series and DataFrames

Built-in methods cover standard maths, but custom logic needs a door: `apply` runs YOUR function over values, rows, or columns.

On a Series, `apply(fn)` feeds every VALUE to the function and collects the results as a new Series. The sigmoid demo maps each number through the (admittedly parenthesised-questionably) sigmoid expression, and the output is a new Series small but transformed — the point being that a Python def can be plugged straight in.

The DataFrame form adds an axis:

The euclidean function reads two coordinate points from a ROW, computes the distance by the usual formula, and returns a single number. `apply(euclidean, axis=1)` hands pandas every row one at a time, and the returned numbers become the new distance column. `axis=1` = row-wise; `axis=0` = column-wise.

`apply` is the escape hatch of pandas: when no built-in fits, write a function and apply it. Its price is speed — a Python function call per element is slower than native vectorised ops — so use it for the genuinely custom cases and prefer named pandas methods everywhere else.

PYTHON

```
temp.apply(sigmoid)
```

PYTHON

```
points_df = pd.DataFrame(
    {
        '1st point':[(3,4),(-6,5),(0,0),(-10,1),(4,5)],
        '2nd point':[(-3,4),(0,0),(2,2),(10,10),(1,1)]
    }
)

points_df
```

11.1 isin filters by a list

`isin` answers one question for the whole column at once: is this value one of the values I care about?

The pattern is boolean masking with a membership test instead of a comparison. `df['col'].isin(a_list)` returns True wherever the cell's value appears in the list, and the surrounding brackets keep exactly those rows. It is pandas' version of SQL's IN clause.

On a Series the same call filters values directly: a batting-runs Series filtered by `vk.isin([49, 99])` keeps only the scores 49 and 99 — the near-misses. Written as an OR chain, the same filter needs `(vk == 49) | (vk == 99)`, which gets unwieldy the longer the list grows. `isin` swaps the whole chain for one word.

`isin` is also composable: combine its mask with other conditions using `&`, `|`, and `~`, and wrap with parentheses. `courses[~courses['course_id'].isin(regs['course_id'])]` — the tilde inverts the mask — finds every course with NO registration, a pattern that returns in the merging session as the anti-join. Whatever the whitelist — ids, names, categories — `isin` turns 'values belonging to a fixed set' into a clean filter.

PYTHON

```
df[df['col'].isin([...])]
```

12. corr — Correlation

12.1 corr measures relationships

When two numeric columns move together, correlation quantifies it. `df.corr()` computes the pairwise Pearson correlation matrix of every numeric column against every other.

The resulting table is symmetric: the diagonal is always 1.0 (each column correlates perfectly with itself), and the off-diagonal cells run from -1 to +1. A value near +1 means 'as one rises, the other rises' — strong positive relationship. A value near -1 means 'as one rises, the other falls' — strong negative relationship. A value near 0 means the two wiggle independently.

Correlation is the measuring tape of feature selection. In a dataset full of candidate features, the cells that approach 1 or -1 flag redundant pairs you can collapse; the cells near 0 flag independent information worth keeping. It is also the first warning system for mistakes: an unexpectedly perfect ± 1 between two columns usually means one is derived from the other.

The related cells in this notebook run `corr` on the available numeric columns and leave the result printed as the report; a heatmap visualisation is the usual follow-up and arrives in later plotting sessions. For now, remember the three anchor values — 1, -1, 0 — and the promise of `corr`: it tells you, in one table, which columns of your data are really saying the same thing.

13.1 Top and bottom N

Sorting then slicing to find the top few is so common that pandas packages it as two dedicated methods.

On a DataFrame, `nlargest(5, 'col')` returns the five rows with the largest values of column `col`, pre-sorted in descending order; `nsmallest(5, 'col')` returns the five smallest in ascending order. Each is exactly `df.sort_values('col', ascending=False).head(5)` — but specialised, so pandas can use faster partial-sorting paths.

On a Series the same methods take just a count: `series.nlargest(3)` returns the three largest values. Several columns may be passed to `DataFrame.nlargest` for tie-breaking, and NaN rows are excluded by default.

Use them for the classic reporting questions: who scored the most? Which courses cost the most? What are the top five player-of-the-match totals? `nlargest` gives the winners' podium in one line, and `nsmallest` answers the mirror-image question about the smallest values. Wherever a question phrases itself as 'top N' or 'bottom

N', these two methods are the direct translation.

PYTHON

```
df.nlargest(n, 'col')
df.nsmallest(n, 'col')
```

14.1 Insert a column anywhere

A normal assignment `df['col'] = values` always adds a column at the END of the frame. When the position matters, insert takes over.

The first argument, `loc`, is the integer position where the new column should land — 0 places it at the front, 3 places it fourth, and so on. The second argument names the column, and the third supplies its values as a Series, list, or scalar, as long as its length matches the frame.

Unlike the assignment syntax, insert always modifies the DataFrame in place and returns None; there is no inplace parameter because there is no copy-based alternative. It also protects you by default: attempting to insert a column name that already exists raises a ValueError unless you pass `allow_duplicates=True`.

Positional column placement is the tool of tidy-data construction: build a base frame with `df['col'] = values` and then slot an identifier or a computed field into the front with `insert(0, 'id', ...)` so the output's column order matches a required report format. Every other column keeps its position, and the new one sits exactly where you asked for it — something the append-at-the-end behaviour of assignment cannot do.

PYTHON

```
df.insert(loc, column, value)
```

15.1 copy before you modify

Everything in this session can mutate data, and the cleanest insurance policy against accidental edits is copy.

`df.copy()` returns a new DataFrame whose rows and columns are independent of the original. Any edit you make to subset — adding a column, filling NaN, dropping rows — leaves the source frame completely untouched. Without copy, a selection like `subset = df[['a', 'b']]` is only a VIEW onto the same underlying memory, and writes into it can leak back into `df`, usually accompanied by the notorious `SettingWithCopyWarning`.

The pattern to internalise:

folds into one place: whenever you are about to try something on a DataFrame — especially anything that writes — take a copy first. The identity check `df.copy() is df` returns False, confirming you hold a genuinely different object.

Copy also keeps pipelines stable: one master DataFrame, many copies processed in parallel without interference. The price of a copy is memory; the price of NOT copying can be hours of debugging a silently corrupted dataset. In this session, where `dropna`, `fillna`, `drop`, and `insert` all reshape frames, a working copy is the safest test bed of all.

PYTHON

```
subset = df.copy()
```

PYTHON

```
df_clean = df.copy()           # snapshot for the pipeline
df_clean['x'] = ...            # experiment freely
```

GroupBy Object in Pandas

1. Setup — Imports and Loading the Data

1.1 The code that prepares the session

Every analysis begins the same way — import the libraries and load the file.

`import pandas as pd` and `import numpy as np` bring in the two pillars of the course; numpy is imported because some GroupBy operations return numpy-style results. `pd.read_csv('/content/imdb-top-1000.csv')` reads the IMDb top-1000 films file (the path assumes the file exists in your Colab environment), and `movies.head()` shows the first five rows.

The head output defines the columns you will group by for the rest of the session: `Series_Title`, `Released_Year`, `Runtime`, `Genre`, `IMDB_Rating`, `Director`, `Star1`, `Star2`, `Star3`, `Star4`, `No_of_Votes`, `Gross`, and `Metascore`. Notice that `Genre` is a single value per movie (Action, Drama, Comedy, and so on) and `Star1` is the lead actor — both are ideal grouping keys. `Gross` is the box-office total, `No_of_Votes` the popularity proxy, and `IMDB_Rating` the score.

Hold one fact from the output as a reference point: The Shawshank Redemption heads the file with a 9.3 rating. When the session later asks for the highest-rated movie per genre, you will already know which answer to expect for Drama.

PYTHON

```
import pandas as pd
import numpy as np
```

PYTHON

```
movies = pd.read_csv('/content/imdb-top-1000.csv')
```

2. GroupBy — Split, Apply, Combine

2.1 Grouping is step zero

A single line — and nothing appears to happen, because nothing has happened yet.

`movies.groupby('Genre')` creates a GroupBy object: it has registered 'make a per-Genre group out of every row', but it has computed nothing. The object is lazy. Only when you hang an operation off it — `.sum()`, `.mean()`, `.agg()`, `.apply()` — does pandas actually split the frame, run the function on each group, and combine the results.

The idea can be captured as a slogan: make small groups, work on each group, join them back. This is the split-apply-combine strategy. When you write `movies.groupby('Genre')['Gross'].sum()`, pandas splits the frame by genre, applies `sum` to the Gross column within every group, and combines the totals into one Series whose index is the genre names — that is the combine step producing the final result.

Grouping keys become the output index. Since groups are named by their key values, the result of a group-aggregate is always 'answer indexed by category'. And because the GroupBy object is cheap to create, you can store it in a variable (`genres`) once and hang many different operations off it — which is exactly what the next several cells do.

Tip: compare with SQL — `SELECT genre, SUM(gross) FROM movies GROUP BY genre` — or with an Excel pivot table. Same idea, pandas spelling.

PYTHON

```
genres = movies.groupby('Genre')
```

PYTHON

```
genres = movies.groupby('Genre')
```

3. Built-in Aggregations

3.1 Aggregating a GroupBy

The simplest way to make a lazy GroupBy object do work is to call one of pandas' built-in aggregation methods on it.

`genres.std()` computes the standard deviation of EVERY numeric column separately within each group — Runtime, IMDBRating, Noof_Votes, Gross, Metascore — and returns a DataFrame whose rows are genres and whose columns are those statistics. Sum, mean, count, min, max, and median behave the same way: pick a function, apply it to every group.

The column selection trick `['Gross']` narrows the aggregation to one column.

`movies.groupby('Genre').sum()['Gross']` first sums everything, then keeps the Gross column; the shorter and faster equivalent `movies.groupby('Genre')['Gross'].sum()` selects the column first and only sums that. Both compile to the same answer.

Chaining makes leaderboards trivial: `movies.groupby('Genre')`

`['Gross'].sum().sort_values(ascending=False).head(3)` answers 'top three genres by total earnings' (Drama, then Action, then Comedy), and `movies.groupby('Genre')`

`['IMDB_Rating'].mean().sort_values(ascending=False).head(1)` finds the highest-rated genre (Western at 8.35). The `count()` idiom counts rows per group: `movies.groupby('Star1')['Series_Title'].count()` answers 'how many movies did each actor lead' — Tom Hanks at the top with twelve. GroupBy plus aggregate plus sort is the entire recipe for every 'top N by category' question.

PYTHON

```
# Applying builtin aggregation functions on groupby objects
genres.std()
```

PYTHON

```
# find the top 3 genres by total earning
movies.groupby('Genre').sum()['Gross'].sort_values(ascending=False).head(3)
```

4. GroupBy Attributes and Methods

4.1 Inspecting the groups themselves

Before asking what the groups contain, ask about the groups themselves. The GroupBy object exposes a small set of attributes and methods that describe its own structure.

`len(movies.groupby('Genre'))` returns 14 — the number of distinct groups — and `movies['Genre'].nunique()` independently confirms 14 distinct values in the column. `movies.groupby('Genre').size()` counts the rows in EVERY group, giving a full frequency table: Drama 289, Action 172, Comedy 155, Crime 107, and so on down to Thriller with a lonely 1. `size()` is exactly what `value_counts()` reports when called on the single grouping column.

`genres.nth(6)` returns the row at position 6 within each genre — the seventh movie in that genre's internal order — while the commented `first()` and `last()` would take the first or last row of each group. `genres.describe()` runs the whole statistical summary (count, mean, std, min, quartiles, max) for every numeric column, group by group, producing a large table of per-genre health statistics.

Tip: `size()` counts rows per group, `nunique()` counts distinct values per group per column, `nth(k)` grabs one row. Learn each with the notebook's real outputs.

4.2 Deep dive — one group at a time

Two GroupBy features deserve a closer look because they pop up constantly: `get_group` and `groups`.

`get_group('Fantasy')` returns the FULL DataFrame of one group — every movie whose Genre is Fantasy. The output shows exactly two rows: 'Das Cabinet des Dr. Caligari' (1920) and 'Nosferatu' (1922), both classic silent films. The equivalent filter `movies[movies['Genre'] == 'Fantasy']` produces the same two rows; `get_group` just gives the group's data by name. When you need to eyeball a single category before aggregating everyone, this is the tool.

`groups` is a dictionary mapping every group key to the list of its row indices: `{'Action': [2, 5, 8, 10, ...], 'Drama': [...], ...}`. It is a map of the whole grouping in one object — useful to check which rows belong where without touching the data itself. It also explains why the GroupBy object feels like a folder of smaller DataFrames: one sub-DataFrame per key, each reachable by name.

Two sampling helpers round out the toolkit. `genres.sample(2, replace=True)` picks two random rows from every genre (with replacement), handy for building a balanced random sample from each category. And `genres.nunique()` counts, per column, how many DISTINCT values appear inside each group — for Action, 172 distinct movie titles, 61 release years, 15 distinct ratings, 123 directors. When someone asks 'how varied are the groups?', `nunique` is the one-word answer.

PYTHON

```
# GroupBy Attributes and Methods
# find total number of groups -> len
# find items in each group -> size
# first()/last() -> nth item
# get_group -> vs filtering
# groups
# describe
# sample
# nunique
```

PYTHON

```
len(movies.groupby('Genre'))
```

5. agg — One Call, Many Aggregations

5.1 Per-column statistics in a single method

So far one aggregation at a time: sum, then mean, then min. Real reports rarely want just one. The `agg` method packages several aggregations into one call — the Swiss-army knife of group-wise reporting.

There are two syntaxes. A DICTIONARY picks functions per column: `{ 'Runtime': 'mean', ... }` says 'give me the mean Runtime per group', and a value that is itself a list, like `['min', 'max', 'mean']`, runs several functions on that one column. The dictionary syntax is the precise one — you choose exactly what happens to each column.

A LIST of function names applies the SAME list to every column: `genres.agg(['min', 'max', 'mean', 'sum'])` produces a wide DataFrame where each original column expands into four sub-columns. Function names are given as strings ('mean', 'sum', 'min', 'max', 'median', 'std'), but any callable — even `lambda x: x.max() - x.min()` — is accepted, so `agg` can also run your own per-group logic.

The notebook's cells combine both ideas: `genres.agg({'Runtime': ['min', 'mean'], 'IMDB_Rating': 'mean', 'No_of_Votes': ['sum', 'max'], 'Gross': 'sum', 'Metascore': 'min'})` gives a report card with exactly the statistics each column wants. When one aggregation per column feels limiting, reach for `agg`.

PYTHON

```
genres.agg({'Runtime': 'mean', 'Gross': ['min', 'max', 'mean']})
genres.agg(['min', 'max', 'mean', 'sum'])
```

5.2 Deep dive — reading agg's output

The shape of agg's result tells you which syntax you used. With the dictionary form, the output keeps one row per group (Action, Adventure, Animation, ...) and one column per requested statistic — Runtime mean, *IMDB_Rating mean*, *Noof_Votes sum*, Gross sum, Metascore min all appear side by side. Reading the Action row: 129.05 mean runtime, 7.95 mean rating, 72.28 million votes in total, 3.26e10 total gross, and a minimum Metascore of 33. Action is the genre that earned most at the box office among its 172 films.

With the list form, `genres.agg(['min', 'max', 'mean', 'sum'])`, every numeric column fans out into four sub-columns and the column header becomes a TWO-LEVEL MultiIndex — `Runtime` over its four functions, then `IMDB_Rating` over the same four, and so on. That is why the printed frame looks far wider than before: 5 numeric columns times 4 functions.

The mixed form produces both effects at once — the Runtime and NoofVotes headers become multi-level (two functions each), while IMDB_Rating, Gross, and Metascore stay single-level (one function each).

Two habits help you navigate these frames. Remember that the rows are always the group keys (the genres), and the columns are the aggregation results. And when you need to turn this summary back into a plain flat table, `reset_index()` flattens the group-key index into an ordinary column — a pattern you will use again with GroupBy results throughout the course.

PYTHON

```
# agg method
# passing dict
genres.agg(
    {
        'Runtime': 'mean',
        'IMDB_Rating': 'mean',
        'No_of_Votes': 'sum',
        'Gross': 'sum',
        'Metascore': 'min'
    }
)
```

PYTHON

```
# passing list
genres.agg(['min', 'max', 'mean', 'sum'])
```

6. Looping Over Groups

6.1 The (group, dataframe) pair

A GroupBy object is iterable: looping over it yields one tuple per group, `(group_name, group_data)`. The single most useful pattern for manual inspection:

Each iteration unpacks the group key (the genre, like 'Action') and the full DataFrame of that group's rows. This is the explicit form of split-apply-combine — you do the apply and combine yourself, by hand, instead of delegating to `sum()` or `mean()`. It is your debugging window onto what the groups actually contain.

Iteration is usually NOT the first tool you should reach for: built-in aggregations and `agg` are vectorised and far faster, and the session's `apply` method (next section) is more idiomatic for custom per-group logic. But looping wins in two situations: when you need to print or log something about each group, and when the group-specific logic is complex enough that forcing it into an `agg` dictionary or an `apply` function would obscure the code. Use the loop to explore and to build results manually, then move to the vectorised tool for production.

PYTHON

```
for group, data in genres:
    print(group, data.shape)
```

6.2 Deep dive — building a result manually

The notebook's loop demonstrates the manual combine step with a real goal:

An empty DataFrame is created with `columns=movies.columns` to reserve the same column names. Inside the loop, for every genre, `data['IMDB_Rating'].max()` finds the genre's highest rating, the boolean mask `data['IMDB_Rating'] == ...` keeps only the movie(s) at that rating, and `append` stacks that winner onto the growing answer frame. The final `df` is the highest-rated movie of every genre — The Dark Knight for Action, Interstellar for Adventure, Spirited Away for Animation.

Two things to notice. First, the loop-style combine is explicit: you build the accumulator yourself instead of letting pandas do it. `df.append` is the pre-1.4 spelling of `pd.concat`, and appending inside a loop is fine for small passes (modern pandas prefers `pd.concat` on a list of frames). Second, when a genre's top rating is shared, the mask keeps EVERY tie — the same logic a `groupby + max` would flatten. For strictly one winner per genre, `drop_duplicates(subset='Genre')` or `.idxmax()`-based selection would be the next refinement. The loop remains the most readable way to see exactly what the combine step does.

PYTHON

```
# Looping on groups
df = pd.DataFrame(columns=movies.columns)
for group, data in genres:
    df = df.append(data[data['IMDB_Rating'] == data['IMDB_Rating'].max()])

df
```

PYTHON

```
df = pd.DataFrame(columns=movies.columns)
for group, data in genres:
    df = df.append(data[data['IMDB_Rating'] == data['IMDB_Rating'].max()])

df
```

7. apply — Custom Functions

7.1 The custom escape hatch

Every aggregation so far used a built-in: sum, mean, or min. But split-apply-combine also needs CUSTOM logic — a statistic that does not exist, or a transformation that ranks movies inside their genre before combining. That door is `apply`.

`apply(fn)` passes each group's sub-DataFrame (or sub-Series) to the function and collects whatever it returns. When the function returns one value per group, the result is a Series indexed by group key — an aggregation. When the function returns a whole DataFrame (for example a ranked copy of the group), the results are stacked back into one big DataFrame — a transformation.

The notebook starts with `genres.apply(min)` — the built-in `min` run per group, which returns the 'smallest' row of each group. Then it moves to named functions: `foo` counts titles, `rank_movie` ranks inside the genre, and `normal` normalises ratings per group. This is the real 'apply' of split-apply-combine: arbitrary Python, executed per group, results combined. It is slower than built-ins — every group costs a Python function call — so prefer vectorised aggregations whenever the built-in expresses your question, and keep `apply` for the genuinely custom cases.

PYTHON

```
genres.apply(min)           # built-in function applied to each group

def foo(group):
    return group['Series_Title'].str.startswith('A').sum()
genres.apply(foo)           # a function you defined

def rank_movie(group):
    group['genre_rank'] = group['IMDB_Rating'].rank(ascending=False)
    return group
genres.apply(rank_movie)    # a function that transforms the group itself
```

7.2 Deep dive — the three named functions

The workbook defines three functions and applies them, so read each one inside-out.

`foo` counts, per genre, how many titles begin with A. `group['Series_Title'].str.startswith('A')` returns a boolean Series, and `.sum()` counts the True values — because True counts as 1. Applied, the result is a Series of integer counts indexed by genre: Drama 21, Comedy 14, Action 10, and 0 for the small genres Family, Fantasy, and Mystery.

`rank_movie` adds a column. Inside the group, `rank(ascending=False)` gives the best-rated movie rank 1; every movie of that genre carries its own *genrerank*, so the returned group now has an extra column. Because the function returns a whole DataFrame, `genres.apply(rank_movie)` returns the ENTIRE movies table with the new *genrerank* column — a transformation, not a reduction.

`normal` min-max scales the rating within each genre: the genre's best movie becomes 1.0, its worst 0.0, and the rest spread between. This is normalisation done per group — exactly the situation where split-apply-combine shines, because the reference values (min and max) differ from genre to genre.

The pattern — define a function that accepts a group, apply it, combine — is the full power of `GroupBy`. Vectorised built-ins cover the standard cases; `apply` covers everything else.

PYTHON

```
# split (apply) combine
# apply -> builtin function

genres.apply(min)
```

PYTHON

```
# find number of movies starting with A for each group

def foo(group):
    return group['Series_Title'].str.startswith('A').sum()
```

8. GroupBy on Multiple Columns

8.1 Two keys, nested groups

Nothing limits you to one grouping column. Pass a list and pandas groups by their combination.

Now every pair (director, lead actor) is one group. 'Christopher Nolan + Leonardo DiCaprio' is a group, 'Christopher Nolan + Matthew McConaughey' is another; the group key becomes a TUPLE, and the result of any aggregation gets a MultiIndex on those two levels — usually named after the grouped columns.

`duo['Gross'].sum()` sums the box-office revenue for every director-actor pair, and the sort answers the natural follow-up: which pairing earned the most? The output shows Akira Kurosawa / Toshirô Mifune heading the list with roughly \$3 billion of combined gross — a legendary partnership quantified. The `.head(1)` variant keeps exactly the winner.

Aggregations and attributes work with tuples just as with single keys: `duo.size()` counts movies per pair, `duo.get_group(('Aamir Khan', 'Amole Gupte'))` pulls the one Taare Zameen Par row, and `duo.agg(['min', 'max', 'mean'])` reports all three statistics for every pair. Because the result is a MultiIndex, retrieving a particular pair later uses the tuple in `.loc`, like `result.loc(('Aamir Khan', 'Amole Gupte'))`.

PYTHON

```
duo = movies.groupby(['Director', 'Star1'])
```

PYTHON

```
duo['Gross'].sum().sort_values(ascending=False)
```

8.2 Deep dive — cross-column questions

Grouping on multiple columns answers questions a single key cannot.

This chain asks for the actor-genre combination with the best average Metascore. Broken down: group by (Star1, Genre); take the mean Metascore within each pair; `reset_index()` flattens the MultiIndex back into plain columns; sort descending; keep the top row — Ellar Coltrane / Drama at a perfect 100. The `reset_index` call is the key habit: it turns an indexed Series into a normal DataFrame so sorting and printing behave like a flat table.

The multi-key frame can also be aggregated broadly. `duo.agg(['min', 'max', 'mean'])` produces a DataFrame with a THREE-level column structure — director / actor on the rows, statistic function under each column — manageable at a glance and perfect for scanning extremes across every pair.

Two practical notes. First, group keys are sorted alphabetically in the result index unless you say otherwise. Second, grouping on more columns creates finer and finer groups: the more keys you add, the fewer movies usually land in each group, so results get noisier. Use one key for broad summaries, two for cross-section analytics like the director-actor combos here, and more only when the data is dense.

PYTHON

```
# groupby on multiple cols
duo = movies.groupby(['Director', 'Star1'])
duo
# size
duo.size()
# get_group
duo.get_group(('Aamir Khan', 'Amole Gupte'))
```

PYTHON

```
# find the most earning actor->director combo
duo['Gross'].sum().sort_values(ascending=False).head(1)
```

9. Exercise — Real Cricket Analysis

9.1 The deliveries dataset

The session closes by moving from movies to cricket — a ball-by-ball log of IPL deliveries.

Every row of `deliveries.csv` is a single ball bowled: `match_id` identifies the game, `inning` says whether it is the first or second innings, `batting_team` and `bowling_team` name the sides, `over` and `ball` count position in the innings, `batsman`, `non_striker`, and `bowler` name the three players involved, and `batsman_runs` records the runs scored off that delivery. `ipl.shape` reports (179078, 21) — over 179 thousand balls across 21 columns, a comfortably big dataset for groupby to chew on.

The questions that follow are all the same shape: group players, aggregate their column, sort, and read off the top. These are not toy queries — they are the exact analysis real cricket dashboards run:

- Top 10 batsmen by total runs — groupby batsman, sum batsman_runs, sort descending.
- Most sixes — filter to balls with `batsman_runs == 6` first, then count per batsman.
- Best at the death — filter to the last overs, then filter fours and sixes, then count.
- A batsman's record against every team — group the batsman's rows by bowling_team.

Before solving the cells, predict each answer on paper; the outputs at the bottom are the sanity check. The solutions also build a reusable `highest(batsman)` function — the course's standard move of turning a one-off query into a function.

PYTHON

```
ipl = pd.read_csv('/content/deliveries.csv')
ipl.head()
```

9.2 Getting the data in

The two cells that follow load the deliveries file and confirm its size.

Read the CSV into a DataFrame named `ipl`, then show the first five rows — the head tells you the exact column spellings you will group and filter on: `batsman`, `bowler`, `batsman_runs`, `over`, `inning`, `match_id`.

Then:

returns `(179078, 21)`: 179,078 deliveries across 21 columns. Keep two columns in mind for the questions ahead — `batsman_runs` (the runs off each ball, the value you will sum) and `over` (which over the ball was bowled, the death-over filter).

Tip: re-run `ipl.head()` now and then; whenever a GroupBy output looks odd, dip back into the raw rows to sanity-check the grouping keys.

PYTHON

```
ipl = pd.read_csv('/content/deliveries.csv')
ipl.head()
```

PYTHON

```
ipl.shape
```

9.3 Deep dive — the exercise solutions

The solution cells exercise every pattern from this session, so trace each one.

Top ten run-getters — already seen, and the output numbers double as trivia: V Kohli 5434 and SK Raina 5415, separated by nineteen runs.

Sixes leader. Notice the refinement compared with the earlier code: `.index[0]` extracts just the winning batsman's name instead of printing the whole sorted Series — Chris Gayle.

Death-over boundaries: filter to overs 16+, keep fours and sixes via `|`, count per batsman — MS Dhoni.

A single player against every bowling team. The groupby is by *bowlingteam*; *summing runs per opponent and calling resetindex* flattens the MultiIndex into a tidy two-column table of his record team by team. (Chennai Super Kings and Delhi Daredevils sit near the top.)

The reusable function: filter to the batsman's rows, sum runs PER MATCH, take the maximum single-match score. `highest('DA Warner')` returns 126 — one function, one pattern.

PYTHON

```
# find the top 10 batsman in terms of runs
ipl.groupby('batsman')['batsman_runs'].sum().sort_values(ascending=False).head(10)
```

PYTHON

```
# find the batsman with max no of sixes
six = ipl[ipl['batsman_runs'] == 6]

six.groupby('batsman')['batsman'].count().sort_values(ascending=False).head(1).index[0]
```

Merging, Joining, Concatenating

1. Setup — Loading the Toy Data

1.1 Five small tables and two big ones

Before touching any merge, the notebook loads the frames you will combine.

Two families of data exist. The first is a miniature course portal: `courses` (`courseid`, `coursename`, price — python at 2499, C++ at 1299, Numpy at 699), `students` (`studentid`, `name`, `partner` — 28 students), and two monthly registration files `nov` and `dec`, each holding only `studentid` / `course_id` pairs. The preview of `dec` shows that shape: every row is one enrolment — student 3 took course 5, student 16 took course 7, student 12 took courses 10 and 1.

That four-table setup is the classic relational database in miniature: students and courses are the 'fact' tables, and the registration files are the 'link' tables that say who took what. The price lives on courses, the names live on students, and the enrolments live in `nov` and `dec` — which is precisely why you must JOIN tables to answer anything interesting. No single file contains revenue; revenue only appears when registrations meet prices.

The second family is two heavy cricket files — `matches.csv` (one row per IPL match, with `id`, `season`, `venue`, `teams`, and `results`) and `deliveries.csv` (one row per ball bowled, with `match_id` linking back to the matches) — saved for the case study at the end.

Look at the keys now: `course_id` in `courses` and in the registration tables, `student_id` in `students` and `registrations`, and `id` in `matches` versus `match_id` in `deliveries`. Those shared columns are exactly the join keys the rest of the session relies on.

PYTHON

```
import pandas as pd
import numpy as np
```

PYTHON

```
courses = pd.read_csv('/content/courses.csv')
students = pd.read_csv('/content/students.csv')
nov = pd.read_csv('/content/reg-month1.csv')
dec = pd.read_csv('/content/reg-month2.csv')

matches = pd.read_csv('/content/matches.csv')
delivery = pd.read_csv('/content/deliveries.csv')
```

2. concat — Stacking Tables

2.1 Pure assembly, no matching

concat is the assembler: it stacks DataFrames and does NOT try to match anything. With `axis=0` (the default), the rows of dec sit underneath the rows of nov — registration month 1, then month 2 — making one combined `regs` frame of every enrolment of both months. `ignore_index=True` renumbers the result 0, 1, 2, ... instead of keeping two sets of repeated indices; without it, the concat result would repeat the original row numbers for each block, which breaks filtering and debugging almost instantly.

There is a legacy alias, `nov.append(dec, ignore_index=True)`, that behaves identically — and is deprecated. Modern pandas prefers `pd.concat`. Both appear in the notebook so that you recognise old code when you meet it.

The default concat is an OUTER union: if the two frames have different columns, the result contains ALL columns from both, and NaN fills the cells where one frame had no such column. concat is the tool for stacking month files, appending year folders, or widening across columns (`axis=1`) — but it never, ever joins on keys.

PYTHON

```
regs = pd.concat([nov, dec], ignore_index=True)
```

2.2 Deep dive — keys and horizontal stacking

concat has two extra superpowers worth seeing in the outputs.

The `keys=` argument labels each block with a name. The result `multi` becomes a MultiIndex DataFrame whose top level is the source ('Nov' or 'Dec') — so the origin of every row is recorded, not lost.

`multi.loc[('Dec', 4)]` uses the tuple ('Dec', 4) to fetch the rows of the December block that came from original index 4: student 14, course 9. If you later need the source as a real column instead of an index level, `reset_index()` promotes it.

`axis=1` stacks SIDEWAYS — columns, not rows. The result is a wide frame with *studentid* and *courseid* twice (once from nov, once from dec), aligned by original position. The printed headers repeat: `student_id course_id student_id course_id`. Different lengths or orderings produce NaN padding on the shorter frame, so horizontal concat is normally used only with compatible row counts.

The rule to remember: concat gives you MORE ROWS or MORE COLUMNS, never more information per row. For that you need merge — the next section.

PYTHON

```
# pd.concat
# df.concat
# ignore_index
# df.append
# mullitindex -> fetch using iloc
# concat dataframes horizontally
```

PYTHON

```
regs = pd.concat([nov,dec],ignore_index=True)
regs
```

3. merge — SQL-Style Joins

3.1 Joining rows on a key

merge is the relational JOIN: it combines two tables by matching a shared column, bringing columns from both frames into a single result.

The grammar is `left_frame.merge(right_frame, how=..., on=...)`. `on` names the join key — the column that must exist in both frames. `how` chooses the join type:

- `inner`: only keys present in BOTH tables (the common core).
- `left`: every row of the LEFT frame, plus matches from the right; unmatched right columns become NaN.
- `right`: every row of the RIGHT frame, plus matches from the left.
- `outer`: the union of everything, NaN where the other side has no match.

So `students.merge(regs, how='inner', on='student_id')` joins every registration to its student record — the result adds name and partner columns to each registration row, keeping only students who enrolled.

`courses.merge(regs, how='left', on='course_id')` keeps ALL courses — including those with zero registrations; registration columns fall out as NaN for the empty ones.

If the two keys have different names, use `left_on` and `right_on` instead of `on`. And mind many-to-many joins: if a key repeats in both tables, the result explodes into every combination (a cartesian product) — a classic source of over-counted rows.

Mental hook: choose the base = left frame. 'Keep everything from courses' is a left join with courses on the left.

PYTHON

```
students.merge(regs, how='inner', on='student_id')
courses.merge(regs, how='left', on='course_id')
```

3.2 Deep dive — four joins, four shapes

Read the notebook's four merge outputs and their shapes tell you everything.

The INNER join, `students.merge(regs, how='inner', on='student_id')`, shows *studentid*, *name*, *partner*, *courseid*. Only the students who actually registered appear, each repeated once per course: Kailash Harjo and his registrations, then Esha Butala, and so on.

The LEFT join swaps perspective: `courses.merge(regs, how='left', on='course_id')` prints every course with its registrations. Python (2499) has many rows because many students registered for it; courses nobody took appear once with NaN in the *student_id* column — the left join is the classic 'keep the master list' pattern.

The RIGHT join is the mirror image. The notebook first ADDS students 26, 27, and 28 (Nitish, Ankit, Rahul) by concatenating a small frame onto `students`, then runs `students.merge(regs, how='right', on='student_id')`. Because the right frame *regs* drives the output, every registration appears, and the students who registered for nothing show up with NaN *course_id*. `regs.merge(students, how='left', on='student_id')` produces the identical result with the frames swapped — always possible, and a good sanity check that your join direction is right.

The OUTER join, `students.merge(regs, how='outer', on='student_id').tail(10)`, shows the union: the bottom rows carry some students with NaN *course* columns where the other side had no match.

Each join answers a different question. Inner: who is linked? Left/right: keep one side complete. Outer: keep everything, gaps and all.

PYTHON

```
# inner join
students.merge(regs, how='inner', on='student_id')
```

PYTHON

```
# Left join
courses.merge(regs, how='left', on='course_id')
```

4. Merge + GroupBy Patterns

4.1 Revenue from two tables

Merging is not the goal — answering questions is. The session's middle section now stitches merge onto the grouping skills you already have, in the classic pattern `merge -> groupby -> sum`.

This is revenue in one line. First merge the registrations with the course table so each enrolment gains its course's price; then `['price'].sum()` totals the money earned — 154,247 in the notebook. The merge is what created the price column to begin with; no single input file contained it.

Month-by-month revenue merges the two tricks. The `concat` with `keys=['Nov', 'Dec']` records the origin the moment the files are combined; `resetindex` turns the *MultiIndex* level into a plain column (named *level0*); and the merge attaches prices before `groupby('level_0')['price'].sum()` totals November (89,175) and

December (65,072) separately. The combine-first, merge-second, group-third order is deliberate — every step adds exactly the information the next step needs.

PYTHON

```
regs.merge(courses, how='inner', on='course_id')['price'].sum()
```

PYTHON

```
temp_df = pd.concat([nov, dec], keys=['Nov', 'Dec']).reset_index()
temp_df.merge(courses, on='course_id').groupby('level_0')['price'].sum()
```

4.2 Deep dive — the join problem set

The next cells run a small test suite of join questions. Trace each one and the pattern becomes automatic.

The three-table chain. Start from registrations, join students for the names, join courses for the names and prices, then keep the three report columns — the entire customer bill printed as one table. The key column survives every join so the chain can keep going.

'Who enrolled in both months?' is a set-intersection question that needs no merge at all: `np.intersect1d` returns the IDs present in both files, and `isin` filters the students table down to them.

'Which courses got zero enrolments?' is the anti-join: `setdiff1d` (or the pandas version `~courses['course_id'].isin(regs['course_id'])`) finds IDs in courses but not in registrations — the answer: Numpy and C++.

The SELF-JOIN: merging students with itself pairs each student's partner ID with the partner's name — *name_x is the student, name_y their partner*. The merged columns carry the `_x` / `_y` suffixes, which is how pandas distinguishes the two 'name' columns.

The top-N cells finish the batch: group registrations by (student_id, name) and count for 'most enrolments', or join prices and sum for 'most money spent' — Chhavi Lachman tops both lists.

Tip: revenue and ranking questions are almost always merge-then-group; membership questions (who/what is IN a set) are `setdiff1d` / `intersect1d` or `isin`.

PYTHON

```
# 1. find total revenue generated
total = regs.merge(courses, how='inner', on='course_id')['price'].sum()
total
```

PYTHON

```
# 2. find month by month revenue
temp_df = pd.concat([nov, dec], keys=['Nov', 'Dec']).reset_index()
temp_df.merge(courses, on='course_id').groupby('level_0')['price'].sum()
```

5. merge — Syntax Variants

5.1 Two spellings, one join

merge can be called either on a DataFrame or on the pandas module, and the two are interchangeable.

`df.merge(other, ...)` and `pd.merge(df, other, ...)` compute the identical result; the method form reads more naturally in a chain, while the module form is useful when you are building the join from variables. Newer pandas also offers `df.join()` for index-based merges and the *lefton* / *righton* spelling for mismatched key names, as the IPL case study will use.

The important vocabulary you must not lose: `how` (inner/left/right/outer), `on` (shared key), the `_x` / `_y` suffixes when both frames bring the same column name, and the cardinality warning — when keys repeat, rows multiply. Merging one student record against five registrations yields five rows for that student; that is expected for a many-to-one join, and it is exactly why the notebooks' top-N groupby counts registered users correctly.

Before moving on, re-run the alternate-syntax cell and confirm the output is byte-identical to the earlier inner join. Same tool, same result, two spellings — keep whichever you find clearer.

PYTHON

```
# Alternate syntax for merge
# students.merge(regs)

pd.merge(students, regs, how='inner', on='student_id')
```

PYTHON

```
students.merge(regs, how='inner', on='student_id')
pd.merge(students, regs, how='inner', on='student_id')
```

6. Case Study — IPL Merges

6.1 Two files become one

The session closes by merging two real cricket files whose key columns have DIFFERENT names — the situation `left_on` and `right_on` were made for.

delivery's per-ball table identifies its game by `match_id`; matches' summary table calls the same thing `id`. Since the names differ, `on` cannot be used — the merge declares both: `left_on='match_id'` in the deliveries frame, `right_on='id'` in the matches frame. Every ball then gains its match's columns: season, city, venue, teams, winner.

The merged `temp_df` now answers questions neither file could answer alone. Stadium sixes ratio: count sixes per venue from the six-filtered merged frame (`six_df.groupby('venue')['venue'].count()`) and divide by the matches-per-venue counts (`matches['venue'].value_counts()`) — the ratio gives the six-happiest grounds, Holkar Stadium on top at 17.6 sixes per match. Per-season orange cap: `temp_df.groupby(['season', 'batsman'])['batsman_runs'].sum()` — one groupby, both files' information.

Tip: always check that key dtypes match before merging — a string '12' and an int 12 will silently fail to join.

PYTHON

```
temp_df = delivery.merge(matches, left_on='match_id', right_on='id')
```

6.2 Deep dive — reading the merge pipeline

Walk the cells in the order the notebook runs them.

`matches` and `delivery` are printed raw first, so you can stare at the two key columns — `id` against `match_id` — and at the venue/season information you are about to gain per ball. Then the one-line merge creates `temp_df`, the 179k-row working frame; and `six_df = temp_df[temp_df['batsman_runs'] == 6]` keeps only the sixes.

The stadium ratio joins two countings: `num_sixes = six_df.groupby('venue')['venue'].count()` counts sixes per ground, `num_matches = matches['venue'].value_counts()` counts games per ground, and `(num_sixes / num_matches).sort_values(ascending=False).head(10)` ranks the ratio. Aligning the two Series works because both are indexed by venue — the same label-alignment magic you met with Series arithmetic. Holkar Cricket Stadium's 17.6 means its matches average more than 17 sixes, an extreme ground for hitting.

The final block computes the orange cap for every season:

Group by (season, batsman) and sum the runs; *resetindex flattens the MultiIndex; sort runs descending so each season's best is on top; dropduplicates(season, keep='first')* keeps exactly that best; and the final sort orders the winners by season — SE Marsh 616 in 2008, ML Hayden 572 in 2009, SR Tendulkar 618 in 2010, CH Gayle 2011-2012 with 608 and 733, and so on. A single pipeline, built from groupby + reset + drop_duplicates + sort, full of both files' data.

PYTHON

```
# IPL Problems

# find top 3 stadiums with highest sixes/match ratio
# find orange cap holder of all the seasons
```

PYTHON

```
matches
```

Multindex — Series and DataFrames (Session 21)

1. Multindex Series — Hierarchical Indexing

A Series is one-dimensional and a DataFrame two-dimensional — but the *index itself* can have depth. When rows need two or more meaningful keys, you give the index multiple levels.

Series is 1D and DataFrames are 2D objects

- But why?
- And what exactly is index?

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

MultIndex — Hierarchical indexing

- More than one level of index → MultiIndex
- `from_tuples()` — a list of tuples → multiple levels
- A Series can also have a multi-level index
- `s['cse']` — access from the first level

PYTHON

```
pd.MultiIndex.from_tuples([('cse',2019), ('cse',2020), ('ece',2019), ('ece',2020)])
s = pd.Series([], index=multi_index)
```

1.1 Why a plain index is not enough

The first cell tries a naive approach: it builds a Series whose index entries are Python *tuples* like `('cse', 2019)`. Pandas accepts this happily and prints `(cse, 2019) → 1, (cse, 2020) → 2, ...` — the label of each item is a single tuple object. The moment you ask `a['cse']` you hit the wall: there is no level called `'cse'`, only a bag of whole tuples, so the lookup utterly fails. The problem is fundamental — a Flat index has one label per row and cannot represent the *grouping structure* that spans `('cse', 2019)` through `('cse', 2022)`.

The solution is the **MultiIndex**, also called hierarchical indexing: an index with two (or more) separate levels, computed from the same tuples. Build it with `pd.MultiIndex.from_tuples(...)` — a list of `(level0, level1)` pairs — and attach it to a Series. Suddenly the hierarchy is real: `s['cse']` returns the whole `cse` sub-series (2019 through 2022), and `s[('cse', 2019)]` plucks a single value. The index now behaves like a folder tree — open the top folder `cse` and see all its years — which is exactly the structure statistical software calls nested or hierarchical. `from_product` (`[['cse', 'ece'], [2019, ..., 2022]]`) builds the full Cartesian grid in one call.

PYTHON

```
# can we have multiple index? Let's try
index_val = [('cse',2019),('cse',2020),('cse',2021),('cse',2022),('ece',2019),('ece',2020),('ece',2021),('ece',2022)]
a = pd.Series([1,2,3,4,5,6,7,8],index=index_val)
a
```

PYTHON

```
# The problem?
a['cse']
```

Deep Dive : MultiIndex Series — hierarchical indexing

- Problem: plain index + duplicate keys ambiguous
- Solution: **MultiIndex** — 2+ levels (tuples)
- `fromtuples` / `fromproduct` / `from_arrays` — creators
- levels — each hierarchy axis separate
- Select top-level → sub-Series of below levels
- Selection: `s['cse']` , `s[( 'cse', 2019)]`
- Levels power different aggregation depth
- Think of 2D table folded into Series
- (unstack shows the table view)
- SQL GROUP BY style hierarchy in index

PYTHON

```
# how to create multiindex object
# 1. pd.MultiIndex.from_tuples()
index_val = [('cse', 2019), ('cse', 2020), ('cse', 2021), ('cse', 2022), ('ece', 2019), ('ece', 2020), ('ece', 2021), ('ece', 2022)]
multiindex = pd.MultiIndex.from_tuples(index_val)
multiindex.levels[1]
# 2. pd.MultiIndex.from_product()
pd.MultiIndex.from_product([['cse', 'ece'], [2019, 2020, 2021, 2022]])
```

PYTHON

```
# creating a series with multiindex object
s = pd.Series([1,2,3,4,5,6,7,8], index=multiindex)
s
```

2. stack / unstack — Reshaping Levels

The same data can live as a deep Series or as a wide grid. `unstack` and `stack` are the two doors between those rooms.

stack / unstack

- `unstack()` — move a level from the index to the columns
- `stack()` — move columns back into the index
- Series ⇌ DataFrame conversion
- Turns MultiIndex tables into something pivot-like

PYTHON

```
s.unstack() # second level → columns
temp.stack() # columns → back to index
```

Deep Dive : stack / unstack

- unstack — moves inner level to columns → 2D table
- stack — reverse, shove columns back into index
- MultiIndex Series ⇌ DataFrame conversion
- Visualize any hierarchy as grid
- pivot-like restructuring (from_tuples long→wide)
- default: innermost level; level=N param
- pair: display grid, analyze index
- Fast re-layout of same data
- broad DW skill (long vs wide understanding)

PYTHON

```
temp = s.unstack() # second Level → COLUMNS (Index→Columns)
temp # DataFrame grid: rows cse/ece, cols 2019-2022
temp.stack() # columns → back to inner index level
```

2.1 Moving levels between the index and the columns

A MultiIndex Series is, visually, a tall folded table: the two levels are printed one inside the other (`cse 2019 → 1`, `cse 2020 → 2`, ...). `unstack()` unfolds it: it takes one level of the index — by default the innermost one, here the year — and pushes it into the *columns*. The result is a genuine 2-D grid (a DataFrame): rows `cse / ece`, columns `2019 – 2022`, and the values (1...8) as cells. The Series that was 1-D in spirit is revealed as a pivot-like table.

`stack()` is the exact reverse: it pulls a level back out of the columns and re-inserts it as the inner level of the index, returning space to the original tall MultiIndex Series. Run `temp.stack()` and you recover the eight labelled rows verbatim.

Why does this matter? These two verbs let you display data in whatever orientation suits the job — a grid for your eyes, a deep Series for computation. The pair `Series ⇌ DataFrame` answering the same question 'where does this level physically live?', and they power the Semi-hierarchical workflows you will use again with `pivot_table`. Later sessions rely on exactly this conversation between index-levels and column-levels.

PYTHON

```
# unstack
temp = s.unstack()
temp
```

PYTHON

```
# stack
temp.stack()
```

3. MultiIndex DataFrames

A MultiIndex can live not only in the row axis but also in the columns — or in both at once — producing two-dimensional hierarchies.

MultiIndex DataFrame

- Both the index and the columns can be multi-level (branch_df3)
- `loc[('cse',2022)]` — access a row with a tuple
- Columns can have levels too — `df['delhi']['students']`
- `.stack().stack()` — levels from the columns into the index

PYTHON

```
branch_df1 = pd.DataFrame(..., index=multi_index)
branch_df2 = pd.DataFrame(..., columns=multi_index)
```

Deep Dive : MultiIndex DataFrame

- Rows MultiIndex — groupby-like nesting in index
- Columns multiindex — categories × subcategories
- both — full 2D hierarchy
- index cols vary per level
- axis selection deep via loc/tuples
- Transpose — rows/cols levels flip
- swaplevel — switch level order
- Pandas max granularity: 2D; extra dims = hier
- Read as: outer key → inner key → values
- Multi-dim data in 2D tables

PYTHON

```
branch_df1 = pd.DataFrame([[1,2],[3,4],[5,6],[7,8],[9,10],[11,12]],
    index=pd.MultiIndex.from_product([('cse','ece'),(2019,2020,2021)]),
    columns=['students','package'])

branch_df2 = pd.DataFrame(
    index=['cse','ece'], columns=pd.MultiIndex.from_product([('delhi','hyderabad'),(2019,2021)]),
    data=[... ]) # MultiIndex COLUMNS
```

3.1 Rows, columns, and both

Building `branch_df1`, a DataFrame with the `multiindex` (branch × year) as its row index and plain columns `avg_package` / `students`, you get hoverable rows: `('cse', 2019)` and friends label eight rows whose cells hold numbers like 1–16. Selecting a single column via `branch_df1['students']` still returns a MultiIndex Series. The follow-up question the notebook poses — *are columns really different from the index?* — is answered by flipping the construction: `branch_df2` uses plain years (2019–2022) as rows but builds its *columns* from `pd.MultiIndex.from_product([['delhi', 'mumbai'], ['avg_package', 'students']])`. Now the header itself is two levels, printing `delhi/avg_package`, `delhi/students`, `mumbai/avg_package`, `mumbai/students`. `branch_df2.loc[2019]` returns a horizontal Series whose labels are `(delhi, avg_package)` etc.

`branch_df3` puts both hierarchies into play: multi-level rows *and* multi-level columns. Because Pandas tops out at two visual axes, any extra dimension — a third category, a city, a season — must be folded into these levels. The mental model is 'outer key → inner key → value': read `cse → 2019 → avg_package → delhi → 1`. And since an axis is just a level stack, operations like `.stack().stack()` (columns → index twice) can completely re-lodge a structure.

PYTHON

```
branch_df1 = pd.DataFrame(
    [
        [1,2],
        [3,4],
        [5,6],
        [7,8],
        [9,10],
        [11,12],
        [13,14],
        [15,16],
    ],
    index = multiindex,
    columns = ['avg_package', 'students']
)

branch_df1
```

PYTHON

```
branch_df1['students']
```

Stacking and Unstacking

PYTHON

```
branch_df3.stack().stack()
```

4. Working with MultiIndex DataFrames

Having built the hierarchies, the payoff is precise, flexible selection. This section drills the complete query vocabulary: head/tail, shape, info, duplicated, isnull, loc, iloc, sorting, transpose, and swaplevel.

MultilIndex operations

- `loc` tuple-based rows/cols
- `iloc` position-based
- `sort_index(level=...)` — specific level per sort
- `transpose` / `swaplevel` — axis manipulation

PYTHON

```
branch_df3.loc[('cse', 2022)]           # single row
branch_df3.loc[('cse', 2019):('ece', 2020):2] # slice rows
branch_df3.iloc[0:5:2]                  # position
branch_df3.transpose()                  # swap dims
branch_df3.swaplevel(axis=1)             # level order
```

4.1 loc, iloc, sorting, and axis surgery

The routine housekeeping works unchanged: `.head()`, `.shape` (8 rows, 4 columns here), `.info()` (which shows the MultilIndex levels and the tuple column names), `.duplicated()`, and `.isnull()` all behave normally on a hierarchical frame. What changes is *selection*.

`loc` with a tuple reaches a precise row: `branch_df3.loc[('cse', 2022)]` returns the four cells of that one (branch, year) combo as a Series. Slicing also understands level boundaries: `branch_df3.loc[('cse', 2019):('ece', 2020):2]` walks the two-level index — cse-2019, cse-2021, ece-2019 — inclusive and step-picked. `iloc` ignores labels entirely and slices by pure position (`branch_df3.iloc[0:5:2]`, `iloc[:, 1:3]`, `iloc[[0, 4], [1, 2]]`), perfect when you know the physical layout. The columns' own hierarchy responds to chaining: `branch_df3['delhi']['students']` dives level by level.

Three reorganisers round out the toolkit. `sort_index` orders by level — `ascending=[False, True]` sorts the branch level descending while years ascend, and `level=0` targets a single level. `transpose()` flips rows and columns wholesale, swapping which hierarchy is where. `swaplevel(axis=1)` keeps data but reorders the column levels — turning `delhi/avg_package` into `avg_package/delhi`. Between `loc`, `iloc`, and these axis verbs, any view of a nested table is one expression away.

PYTHON

```
# head and tail
branch_df3.head()
# shape
branch_df3.shape
# info
branch_df3.info()
# duplicated -> isnull
branch_df3.duplicated()
branch_df3.isnull()
```

Deep Dive : MultilIndex — loc querying

- loc tuples — full/partial level selection
- Partial (`['cse']`) → all sublevel rows

- Slicing tuples — inclusive group boundaries
- col hierarchy — chain/double brackets
- `iloc` — unaffected positional regardless of levels
- `sort_index` first for clean slices
- Get level values: `.unique()` on index `get_level_values`
- Query semantics — two axes both levels
- Selection = mastery layer
- Useful subsetting explores full nest

PYTHON

```
# Extracting rows single
branch_df3.loc[('cse', 2022)]
```

PYTHON

```
# multiple
branch_df3.loc[('cse', 2019):('ece', 2020):2]
```

5. Reshaping — Wide vs Long, and melt

This section confronts the eternal shape question. **Wide format** gives each data point one row with many attribute columns; **long (tidy) format** gives each point many rows — one per attribute/value pair. Machines, `groupby`, and statistical modelling prefer long; readers prefer wide.

Long Vs Wide Data

Name	Height	Weight
John	160	67
Christopher	182	78

Name	Attribute	Value
John	Height	160
John	Weight	67
Christopher	Height	182
Christopher	Weight	78

Wide format is where we have a single row for every data point with multiple columns to hold the values of various attributes.

Long format is where, for each data point we have as many rows as the number of attributes and each row contains the value of a particular attribute for a given data point.

Wide → Long (melt)

- `melt` — wide table → long format

- `id_vars` — the columns that stay fixed (i.e. the identifier)
- Each value gets its own row, and a variable-name column is created
- Real example: COVID time-series → long for plotting

PYTHON

```
pd.DataFrame({'cse':[120]}).melt() # col → row
df.melt(id_vars=[...], var_name='date', value_name='count')
```

Deep Dive : melt — wide → long

- melt — pivot wide table to TIDY long format
- Columns → rows (variable column + value column)
- `id_vars` — keep fixed identity cols
- `varname/valuenam` — rename melted cols
- Year-wide data → date × place tidy rows
- Coupled with merge/groupby analyses
- Real EDAs: COVID case tables
- Long format = machine ready (plotting/groupby)
- Compose: wide presentable ⇌ long computational
- essential data gymnastics

PYTHON

```
pd.DataFrame({'cse':[120], 'ece':[100], 'mech':[80]}).melt()
#   variable  value
# 0      cse    120
# 1      ece    100
# 2     mech     80

death.melt(id_vars=['Province/State', 'Country/Region', 'Lat', 'Long'],
           var_name='date', value_name='cases')
```

5.1 melt — converting wide to long

`melt()` pours a wide table into long/tidy form by turning selected columns into rows. Without extra arguments, `pd.DataFrame({'cse':[120]}).melt()` produces two columns: `variable` (the original column name `cse`) and `value` (the number `120`). Add a branching example with `var_name='branch'`, `value_name='num_students'` and the one-column frame becomes three rows: branch `cse/ece/mech` with their counts — every column is now a data row, and the column *name* became a variable's value.

The year-wide school example shows the pattern at full power: a frame with `branch` plus columns `2020`, `2021`, `2022` melts via `id_vars=['branch']` into nine tidy rows `(branch, year, students)`. `id_vars` lists the columns that stay put as identifiers; every other column is collapsed into the melting variable. `var_name` and `value_name` rename the two produced columns.

The real-world payoff is the COVID-19 case/death tables: each has a date *column* for every day of the pandemic. Melting `death` and `confirm` with `id_vars=['Province/State', 'Country/Region', 'Lat', 'Long']`, `var_name='date'`, and `value_name='num_cases'/'num_deaths'` reshapes 100+ date columns into a tidy `date` × `place` table, which then merges cleanly on the five key columns into one analysis-ready dataset. Where wide is presentable, long is *computable*.

PYTHON

```
# melt -> simple example branch
# wide to long
pd.DataFrame({'cse':[120]}).melt()
```

PYTHON

```
# melt -> branch with year
pd.DataFrame({'cse':[120], 'ece':[100], 'mech':[50]}).melt(var_name='branch', value_name='num_students')
```

6. Pivot Tables

A pivot table takes column-wise (usually long-ish) data and groups it into a two-dimensional summary table — rows from one category, columns from another, cells from an aggregation of a third. Excel calls it a pivot; Pandas calls it `pivot_table` — effectively a one-stop `groupby` + `unstack`.

Pivot Table

The pivot table takes simple column-wise data as input, and groups the entries into a two-dimensional table that provides a multidimensional summarization of the data.

PYTHON

```
import numpy as np
import pandas as pd
import seaborn as sns
```

6.1 pivot_table — a summary cross-tab

The tips dataset is loaded from the `seaborn` library and gives each restaurant bill a `total_bill`, `tip`, `sex`, `smoker`, `day`, `time`, and `size`. First recall the groupby baseline: `df.groupby('sex')[['total_bill']].mean()` reports Male ~20.7 vs Female ~18.1 — one column of results. Grouping on *two* columns and unstacking, `df.groupby(['sex', 'smoker'])[['total_bill']].mean().unstack()`, produces a grid whose rows are sex and whose columns are smoker Yes/No. `df.pivot_table(index='sex', columns='smoker', values='total_bill')` reproduces that exact grid directly: `index` names the row keys, `columns` the column keys, and `values` the quantity being aggregated. The default aggregation is the **mean**, so the cells match the grouped means.

`aggfunc` swaps in other reductions: `aggfunc='std'` shows the spread of bills per sex/smoker cell, and a dict form (`{'size':'mean', 'tip':'max', 'total_bill':'sum'}`) lets different columns use different functions in one call. Multi-level keys are allowed too — index `['sex', 'smoker']`, columns `['day', 'time']` — building

genuinely multi-dimensional summaries. `margins=True` appends grand total row and column ('All'), and `fill_value=0` pad densifies empty cells. When a business wants 'months × categories' or 'region × channel' style reports, `pivot_table` produces them in a single expression.

PYTHON

```
df = sns.load_dataset('tips')
df.head()
```

PYTHON

```
df.groupby('sex')[['total_bill']].mean()
```

Pivot tables

- `pivot_table` — a shortcut for `groupby+unstack`
- `index` → rows, `columns` → cols, `values` → cell values
- `aggfunc` — mean/sum/std (default mean)
- `margins=True` — total row/col
- `fill_value=0` — missing → 0

PYTHON

```
df.pivot_table(index='sex', columns='smoker', values='total_bill')
df.pivot_table(index='sex', columns='smoker', aggfunc='std')
df.pivot_table(index=['sex', 'smoker'], columns=['day', 'time'], aggfunc='sum', margins=True)
```

Deep Dive : pivot_table — Excel pivot in pandas

- `pivot_table(index, columns, values, aggfunc)` — agg cross-tab
- `index` → rows, `columns` → column labels, `values` → cell values
- `aggfunc` default mean — sum/count/custom
- `margins` — grand totals row/col
- Multi lists — richer cross-sections
- vs `groupby().unstack()` — same; `pivot_table` all-in-one agg
- Use: 2D summary reports instantly
- Business reporting: months×categories (expense)
- `.dt.month_name()` prep dates for pivots
- plot the pivot result — heatmap/drawing

PYTHON

```
df.pivot_table(index='sex', columns='smoker', values='total_bill', aggfunc='mean')
df.pivot_table(index=['sex', 'smoker'], columns=['day'], values='total_bill') # multi
df.pivot_table(index='sex', columns='smoker', values='total_bill',
                aggfunc=np.sum, margins=True) # totals row/col
```

6.2 Aggregation functions in pivot tables

The `aggfunc` parameter decides what the cell values actually *mean*. The default is `mean` — each cell holds the average of the `values` column within that (index, column) combo. The example with `aggfunc='std'` on `total_bill` shows standard deviation instead: Male smokers' bills have a spread of ~9.9, Male non-smokers ~8.7, and so on. The difference matters dramatically for reporting: a mean hides volatility, a std exposes it, a sum tells you absolute volume, and `count` reveals sample sizes. You can pass any string Pandas understands ('sum', 'mean', 'median', 'std', 'count', 'max', 'min') or even a NumPy function such as `np.sum`. When values are a list-worthy mixture, the dict form lets each value-column use its own function, all in a single pivot. The notebook also demonstrates that omitting `values` pivots **all numeric columns at once**, returning a DataFrame with a two-level column header — the original column name stacked on top of the pivot categories.

PYTHON

```
# aggfunc
df.pivot_table(index='sex', columns='smoker', values='total_bill', aggfunc='std')
```

PYTHON

```
# all cols together
df.pivot_table(index='sex', columns='smoker')['size']
```

6.3 Case study — expense data pivoted and plotted

The final sequence turns a personal expense CSV into a set of time-vs-category charts. The frame is inspected (`value_counts()` shows Food dominating), `df.info()` reveals `Date` is still text, and `pd.to_datetime` upgrades it to `datetime64`. A `month` column is then derived with `df['Date'].dt.month_name()`. Three pivots follow, each `index='month'`, and each with a different `columns` category: `Category`, `Income/Expense`, and `Account`, with `values='INR'`, `aggfunc='sum'`, and `fill_value=0` so missing month/category combos plot as zero rather than gaps. Each pivot result is piped straight into `.plot()`, and the month names on the x-axis sort alphabetically by string, not chronologically. The lesson is end-to-end: read text dates, parse them, extract a time label, pivot (month × category, summed), and visualise the summary — a template that transfers directly to sales, finance, and operations dashboards.

PYTHON

```
# plotting graphs
df = pd.read_csv('/content/expense_data.csv')
```

PYTHON

```
df.head()
```

Vectorized String Operations in Pandas (Session 22)

1. Vectorized Operations

Vectorization is the reason Pandas can chew through hundreds of thousands of strings without breaking a sweat: one operation applied to the whole column at once, in compiled code, instead of a Python `for` loop touching one element at a time.

Vectorized String Operations in Pandas

Topic: 07.07 · **Level:** ● Intermediate · **Type:** THEORY + CODE

Vectorized operations

- Vectorized = one operation on every element, no loop
- Numpy/pandas core speed
- Direct Python string operations fail on None/NaN

PYTHON

```
a = np.array([1, 2, 3, 4])
a * 4           # elementwise → [4,8,12,16]
```

Deep Dive : Vectorized ops — pandas solves python's string slowness

- Vectorized: op applied to WHOLE array at once (compiled C)
- Python loop: per-element, slow + error-prone
- **Pandas** `.str`: missing values (None/NaN) auto-ignored → no crash
- `.str` = vectorized string methods on Series
- Chaining: `s.str.lower().str.strip()`...
- NaN propagation handled for you
- This is why pandas over raw python for text columns

PYTHON

```
a = np.array([1,2,3,4]); a * 4           # fast elementwise – vectorized

s = ['cat', 'mat', None, 'rat', 'sitting', 'batting'] # plain python list
# [x.upper() for x in s] -> None.upper() crashes TypeError!
```

PYTHON

```
s2 = pd.Series(['cat', None, 'rat'])
s2.str.upper() # NaN stays NaN, rest uppercase
```

1.1 What 'vectorized' really means

The NumPy example makes the idea concrete. `a = np.array([1,2,3,4])` then `a * 4` returns `array([4, 8, 12, 16])` — every element multiplied in a single expression, with the loop hidden inside C-level machine code. Compare that with a Python list comprehension; both produce the same numbers, but the NumPy version runs orders of magnitude faster on large arrays and the code is shorter and more readable. This is the same philosophy Pandas inherits for tabular and textual work.

The vulnerability appears the moment text holds anything unexpected. The cell constructs `s = ['cat', 'mat', None, 'rat']` and tries `[i.startswith('c') for i in s]`. The list comprehension walks element-by-element; when it reaches `None`, calling `.startswith` on it raises a `TypeError` — Python objects are *objects* and `None` has no string methods. In the real world missing values are everywhere: blank cells, `None`, `NaN`, and empty strings all arrive from CSVs. A technique that explodes on the first missing value is unusable at scale. The cure arrives in the next section: Pandas' `.str` accessor performs the same logical operation but is deliberately built to tolerate missing data, so a column with a few holes still processes safely in one go.

PYTHON

```
# What are vectorized operations
a = np.array([1,2,3,4])
a * 4
```

PYTHON

```
# problem in vectorized operations in vanilla python
s = ['cat', 'mat', None, 'rat']

[i.startswith('c') for i in s]
```

2. The .str Accessor

The `.str` accessor is Pandas' doorway to every vectorised string method. Write `series.str.method(...)` and the method runs across the whole column at once.

AI/ML relevance

Shapes, dtypes, vectorization, indexing, and clean tabular ops are critical for reliable data pipelines.

.str accessor

- `.str` — accesses string methods vectorized
- Missing values (None/NaN) are skipped automatically
- Much faster and safer than Python loops
- Every string method works with `.str`

PYTHON

```
s = pd.Series(['cat', 'mat', None, 'rat'])
s.str.upper()    # nan/None handled automatically
```

2.1 String methods that never crash on missing data

`pd.Series(['cat', 'mat', None, 'rat'])` followed by `.str.startswith('c')` returns a boolean Series: `True` for `cat`, `False` for `mat` and `rat`, and for the `None` element Pandas returns `None` — no exception, no cascade failure. The same is true for `s.str.upper()`, which uppercases everything present and keeps missing entries missing. This is the single most practical detail in the session: with vanilla Python, one stray `None` kills your loop; with `.str`, missing values ride through quietly and propagate as `None` / `NaN` instead of crashing.

The magical property is that the accessor gives you *every* standard string method in vectorised form — `upper`, `lower`, `capitalize`, `title`, `strip`, `split`, `replace`, `contains`, `startswith`, `endswith`, `isdigit`, and regex-capable ones — all usable on the literal strings as if they lived in one giant C loop. Being vectorised, they are fast; being Pandas-native, they compose with the rest of the ecosystem: filter using the resulting boolean mask, chain accessors (`s.str.lower().str.strip()`), and never write a Python loop for text again. After this warm-up, the notebook moves to a real column — Titanic passenger names — where the power becomes obvious.

PYTHON

```
# How pandas solves this issue?

s = pd.Series(['cat', 'mat', None, 'rat'])
# string accessor
s.str.startswith('c')

# fast and optimized
```

PYTHON

```
# import titanic
df = pd.read_csv('/content/titanic.csv')
df['Name']
```

3. Common String Methods

The Titanic `Name` column follows `Lastname, Title. Firstnames`. It is perfect raw material for practising case conversion, splitting, and slicing.

Common string methods

- `lower/upper/capitalize/title` — case uniform
- `split(',')` → list; `.str.get(i)` → element
- `.str.strip()` — remove whitespace
- Chain: `split` → `get` → `strip` → `split`

PYTHON

```
df['Name'].str.upper()           # case
df['Name'].str.lower()
df['Name'].str.capitalize()
df['Name'].str.title()
df['lastname'] = df['Name'].str.split(',').str.get(0) # split
```

Deep Dive : Common string methods

- case: upper/lower/title/capitalize — uniform text
- split(',') → Series of LISTS
- .str.get(i) → pick i-th element
- .str.strip() — remove leading/trailing spaces
- split with expand=True + n=1 → DataFrame multiple cols
- chain accessors — pipeline one-shot
- .str.len(), .str.contains('x'), .str.replace()
- titanic Name parsing: lastname/title/firstname extraction
- All vectorized, handle NaN
- Cleaning: normalize whitespace & case before matching

PYTHON

```
df['Name'].str.upper() / .lower() / .title() / .capitalize()
df['lastname'] = df['Name'].str.split(',').str.get(0)
df[['title', 'firstname']] = df['Name'].str.split(',') \
    .str.get(1).str.strip() \
    .str.split(' ', n=1, expand=True)
```

3.1 Upper, lower, capitalize, title, and split

Case normalisation is the bread-and-butter of cleaning. `df['Name'].str.upper()` makes every character uppercase, `.str.lower()` the reverse, `.str.capitalize()` capitalises only the first letter of each string, and `.str.title()` capitalises the first letter of every word. They matter because 'john', 'John', and 'JOHN' should match as the same person when you filter or group — but only after you standardise the case.

Length and whitespace are next. `df['Name'][df['Name'].str.len() == 82].values[0]` finds the single longest name (82 characters) and returns it — `.str.len()` counts characters element-wise. The bare Python `" nitish ".strip()` demonstrates how `strip` removes surrounding spaces; `df['Name'].str.strip()` applies it to every name, guarding against ragged whitespace left by CSVs.

The headline trick is splitting a Titanic name into parts. `df['Name'].str.split(',')` turns each name into a list, e.g. `Braund, Mr. Owen Harris` → `['Braund', ' Mr. Owen Harris']`. Follow with `.str.get(0)` to keep element 0 of each list — that is the surname, stored into `df['lastname']`. Then the second element is stripped and re-split on spaces with `n=1, expand=True`, fanning one string into two DataFrame columns `title` and `firstname` in a single assignment. `value_counts()` on the extracted titles immediately reveals the passenger mix — the power of chaining accessors.

PYTHON

```
# Common Functions
# Lower/upper/capitalize/title
df['Name'].str.upper()
df['Name'].str.capitalize()
df['Name'].str.title()
# Len
df['Name'][df['Name'].str.len() == 82].values[0]
# strip
"          nitish          ".strip()
df['Name'].str.strip()
```

PYTHON

```
# split -> get
df['lastname'] = df['Name'].str.split(',').str.get(0)
df.head()
```

4. Cleaning Titles and Filtering Text

Theory turns to action: standardise the messy `title` values, then use text conditions as row filters.

Functional tools

map/filter apply functions; reduce aggregates. Often paired with lambdas.

replace & filtering

- `replace(old, new)` — substring replace
- `startswith/endswith` — boolean filter
- `contains(regex)` — pattern search (regex support)
- String slicing `[::-1]` also works

PYTHON

```
df['title'] = df['title'].str.replace('Ms.', 'Miss.')
df[df['firstname'].str.endswith('A')] # filter rows
df[df['firstname'].str.startswith(...)]
df['Name'].str[::-1] # reverse slice
```

Deep Dive : replace & filtering strings

- `replace(old, new)` — string substitute (exact substring by default, `regex=True` option)
- `startswith / endswith` — prefix/suffix filter → bool mask → `df[mask]`
- combine: `df[df['name'].str.contains('Mr.', na=False)]`
- `na=False` keeps rows where NaN doesn't get dropped by comparison
- filter = the main string use in analysis (name, city, id patterns)
- pattern matching both edge-based and contains-based

- output keeps full df, only filters rows
- value_counts on cleaned titles — insight after cleanup
- boolean masks compose with & | ~

PYTHON

```
df['title'] = df['title'].str.replace('Ms.', 'Miss.') # exact string swap
df[df['firstname'].str.endswith('A')] # filter rows
df[df['firstname'].str.startswith('William')] # prefix rows
```

4.1 replace, startswith, endswith, isdigit

Two Titanic quirks get fixed first: `Ms.` and `Mlle.` are the same person-title as `Miss.`.

`df['title'].str.replace('Ms.', 'Miss.')` swaps the substring everywhere, and repeating it for `Mlle.` merges the counts — check `value_counts()` afterwards and `Miss.` has grown from 182 to 185. By default `replace` treats the search text as a **regular expression**, which is why Pandas emits a `FutureWarning` promising the default will flip to plain-text in a future release; for literal swaps you can pass `regex=False`.

Filtering rows by their text is the second trick. `df[df['firstname'].str.endswith('A')]` keeps only passengers whose first name ends in 'A' — the boolean Series from `.endswith` becomes a mask that selects whole rows. `.str.isdigit()` similarly checks whether the name is made entirely of digits; on beginners' data it returns an empty frame here, but on dirty data it is the fastest way to find rows where numbers got typed into a text column. The general pattern — `df[df['col'].str.<condition>()]` — is one of the most repeated lines in Pandas analysis because it composes: prefix checks with `startswith`, suffix checks with `endswith`, and substring checks with `contains`. Every result keeps the full DataFrame intact and simply filters rows.

PYTHON

```
# replace
df['title'] = df['title'].str.replace('Ms.', 'Miss.')
df['title'] = df['title'].str.replace('Mlle.', 'Miss.')
```

PYTHON

```
df['title'].value_counts()
```

5. Regular Expressions with .str

When `startswith` and `contains` are not expressive enough, regular expressions step in: a tiny pattern language that describes whole classes of text.

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Regex with .str

- `contains` — regex pattern match

- `case=False` — case-insensitive
- Filtering = boolean mask
- Common in data cleaning (irregular text)

PYTHON

```
df[df['firstname'].str.contains('John', case=False)]
```

Deep Dive : Regex with .str

- `str.contains(pattern, regex=True)` — substring or regex test
- `flags=re.I` — case-insensitive
- alternation `|`, wildcards `.`, anchors `^` `$`, `[a-z]` classes
- `str[::-1]` — python string slicing applied elementwise
- `str.findall/extract` for capture groups (complex)
- `regex` = pattern languages for text matching
- Filter by patterns — phone, email, code columns
- escape specials, test carefully
- `regex` slows down — simple substring first via `regex=False`
- Every text col query pattern starts here

PYTHON

```
# contains -> search substring or pattern
df[df['firstname'].str.contains('john', regex=True)] # case sensitive
df[df['firstname'].str.contains('john|John', regex=True)] # alternation
df[df['firstname'].str.contains('.*john.*', flags=re.I, regex=True)] # ignorecase whole string
df['Name'].str[::-1] # reverse each name (slicing on strings!)
```

5.1 Regex filtering in a single line

`.str.contains('john', case=False)` is the workhorse: it tests every first name for the substring 'john' ignoring case, so both 'John' and 'john' match, and returns a boolean mask you feed straight into `df[...]`. Because `contains` supports regex by default, the pattern can grow arbitrarily powerful: `'john|John'` matches either spelling via alternation, and anchors like `^` and `$` pin the pattern to the start/end of the string.

The lastname filter demonstrates real regex muscle: `df[df['lastname'].str.contains('^[^aeiouAEIOU].+[^aeiouAEIOU]$')]` keeps only surnames that begin *and* end with a consonant — `^[^aeiou...]` matches a leading non-vowel, `+` any middle, and `[^aeiou...]$` a trailing non-vowel — parsing the pattern visually is the skill to practise.

Two closing notes. First, if you only need a literal substring, pass `regex=False`; plain-text search is dramatically faster than pattern matching on millions of rows. Second, remember `.str` is a general accessor: `df['Name'].str[::-1]` slices every string from right to left, returning each name reversed — standard Python

string slicing applied element-wise. Between `contains`, `case=False`, anchored patterns, and slicing, every text column in a dataset becomes searchable and filterable in one expressive line.

PYTHON

```
# applying regex
# contains
# search john -> both case
df[df['firstname'].str.contains('john',case=False)]
# find Lastnames with start and end char vowel
df[df['lastname'].str.contains('^[^aeiouAEIOU].+[^aeiouAEIOU]$')]
```

PYTHON

```
# slicing
df['Name'].str[::-1]
```

Datetime in Pandas — Timestamp, DatetimeIndex, and the .dt Accessor (Session 22)

1. The Timestamp Object

A timestamp references a particular moment in time, e.g. Oct 24th, 2022 at 7:00pm. Everything in date analytics begins at this single point.

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Timestamp Object

Time stamps reference particular moments in time (e.g., Oct 24th, 2022 at 7:00pm)

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Creating Timestamp objects

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Deep Dive : Timestamp objects

- Timestamp = single moment in time (enhanced datetime)
- Parses MANY flexible string formats automatically
- AM/PM, ordinal ('5th'), text months all work

- Attributes: year, month, day, hour, minute, second, microsecond
- Construct from datetime, np.datetime64, string, epoch
- **Why pandas not datetime?:** performance + timezone + vector ops
 - datetime is convenient but slow in loops
 - pandas ops vectorized
- Foundation of all time series work
- `.to_pydatetime()` conversion back
- comparisons supported like datetime

PYTHON

```
type(pd.Timestamp('2023/1/5'))           # pandas._libs.tslibs.timestamps.Timestamp
pd.Timestamp('2023-1-5') / '2023, 1, 5' / '2023' / '5th January 2023' / '5th January 2023 9:21AM'
pd.Timestamp(dt.datetime(2023,1,5))      # from python datetime
x.year; x.month; x.day; x.hour; x.minute; x.second  # attribute access
```

1.1 Constructing timestamps from almost any input

`type(pd.Timestamp('2023/1/5'))` returns `pandas._libs.tslibs.timestamps.Timestamp` — a dedicated class, not a plain `datetime`. The constructor is famously forgiving about formats: `'2023-1-5'` and `'2023, 1, 5'` both parse to `Timestamp('2023-01-05 00:00:00')`; `'2023'` alone still yields midnight of January 1st; the texty `'5th January 2023'` works; and `'5th January 2023 9:21AM'` carries the morning hour. The commented-out `'2023/1/5/9/21'` shows the rare input the parser refuses.

You can equally seed a Timestamp from a Python `datetime` object: `dt.datetime(2023, 1, 5, 9, 21, 56)` becomes a Timestamp with the full time preserved. The parsed object then answers attribute questions directly — `x.year` → 2023, `x.month` → 1, `x.day` → 5, and so on through `.hour`, `.minute`, `.second`.

Why choose Timestamp over `datetime`? Three reasons pile up at scale: *performance* (vectorised operations instead of Python loops), *uniform storage* (a compact 64-bit representation), and *ecosystem reach* (timestamps feed directly into DatetimeIndex, resampling, and plotting). Comparing timestamps, converting with `.to_pydatetime()`, and doing calendar arithmetic all come free. A single moment, parsed flexibly, answering component questions — that is the Timestamp.

Try creating timestamps in the announced formats in the cells below, then inspect their attributes.

PYTHON

```
# creating a timestamp
type(pd.Timestamp('2023/1/5'))
```

PYTHON

```
# variations
pd.Timestamp('2023-1-5')
pd.Timestamp('2023, 1, 5')
```

2. Why Python's datetime Is Not Enough — numpy.datetime64

Python already ships a `datetime` module, so why does Pandas bother inventing its own time machinery? The honest answer is performance and memory: convenient loops are slow, but uniform arrays fly.

Deep Dive : numpy datetime64 & why vectorized time

- `np.datetime64` — numpy's time dtype (uniform, fast)
- Addition with ints = days shift vectorized
- Whole arrays shifted together — no python loop
- Reason: performance + uniform memory
- pandas builds on numpy → its own Timestamp layer
- pandas DatetimeIndex ops similarly vectorized
- Yearly/monthly/day arithmetic natural
- 64-bit int backing — compact memory
- This is performance foundation of DateTime in pandas
- syntax wise datetime is very convenient
- But the performance takes a hit while working with huge data. List vs Numpy Array
- The weaknesses of Python's datetime format inspired the NumPy team to add a set of native time series data type to NumPy.
- The datetime64 dtype encodes dates as 64-bit integers, and thus allows arrays of dates to be represented very compactly.

PYTHON

```
date = np.array('2015-07-04', dtype=np.datetime64)
date + np.arange(12)      # all 12 next days at once!

# uniform dtype → fast arithmetic
# python datetime List would need a loop
```

2.1 datetime64 — dates as compact integers

The starting provocation is `date = np.array('2015-07-04', dtype=np.datetime64)`. With the `D` unit this single-cell array stores a date as one 64-bit integer — how many days since the epoch — which is why `datetime64` arrays represent dates so compactly and why arithmetic is so fast: adding `np.arange(12)` to the date performs a vectorised integer addition, returning the twelve following days in a single expression.

The bullet-point cells spell the argument out. Python's `datetime` is *syntax-friendly* but performance takes a hit on large data — a Python list of datetime objects stores pointers and objects, and any operation must call methods per element. NumPy's native `datetime64` dtype encodes dates uniformly, so whole arrays shift at C speed. Pandas then fuses the two worlds in its own `Timestamp` class: the ease of Python's `datetime` plus the efficient storage and vectorised interface of `numpy.datetime64`. From a group of these Timestamps, Pandas builds a **DatetimeIndex**, which the next section shows being used to index a Series or DataFrame. Remember the one-line mental model: `datetime` is ergonomic, `datetime64` is fast, `Timestamp` is both.

- Because of the uniform type in NumPy datetime64 arrays, this type of operation can be accomplished much more quickly than if we were working directly with Python's datetime objects, especially as arrays get large
- Pandas Timestamp object combines the ease-of-use of python datetime with the efficient storage and vectorized interface of numpy.datetime64
- From a group of these Timestamp objects, Pandas can construct a DatetimeIndex that can be used to index data in a Series or DataFrame

PYTHON

```
import numpy as np
date = np.array('2015-07-04', dtype=np.datetime64)
date
```

PYTHON

```
date + np.arange(12)
```

3. DatetimeIndex — a Collection of Timestamps

Take the Timestamp idea and multiply it: a DatetimeIndex is simply an ordered collection of Pandas timestamps, ready to serve as row labels.

AI/ML relevance

Shapes, dtypes, vectorization, indexing, and clean tabular ops are critical for reliable data pipelines.

DatetimeIndex Object

A collection of pandas timestamp

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Deep Dive : DatetimeIndex

- DatetimeIndex = collection of Timestamps → used as INDEX
- Built from strings / datetime objects / Timestamps
- Series indexed by dates → time-indexed data
- Enables time-slicing, resampling, plotting with dates on x-axis
- can infer index from parseable column
- date_sorting matters (chronological indexing)
- `.index` on time-indexed df gives DatetimeIndex
- Advance slicing: `yt.loc['2022-12']` partial-indexing works
- Time-based lookup instant after sort

PYTHON

```
pd.DatetimeIndex(['2023/1/1', '2022/1/1', '2021/1/1'])
pd.DatetimeIndex([dt.datetime(2023,1,1), dt.datetime(2020,1,1)])
pd.Series([1,2,3], index=pd.DatetimeIndex([pd.Timestamp(2023,1,1), ...]))
```

3.1 Building and using a datetime index

Construct one from strings — `pd.DatetimeIndex(['2023/1/1', '2022/1/1', '2021/1/1'])` — from Python `datetime` objects, or from Timestamps, and the result is a `pandas.core.indexes.datetimes.DatetimeIndex`. Crucially the dates need not be sorted the moment you build them; Pandas keeps them in your order until you sort explicitly.

The payoff appears when the `DatetimeIndex` becomes the *index* of a Series: `pd.Series([1, 2, 3], index=dt_index)` labels each value with a date. A time-indexed Series or DataFrame is what unlocks the whole feature set used later in this module — slicing by year/month (`yt.loc['2022']`), resampling, and automatic date axes on plots. Because the index is internally ordered and hashable, lookups are fast and unambiguous, and the `.index` of any time-indexed frame is itself a `DatetimeIndex`.

Practical notes from these cells: pass a list of strings, datetime objects, or Timestamps interchangeably; store the index in a variable (`dt_index`) so you can reuse it; and remember the pattern for the real world — read a CSV, parse the date column with `to_datetime`, then `set_index` so time becomes the axis. Ordering and clean dtypes make every later time operation possible.

PYTHON

```
# from strings
type(pd.DatetimeIndex(['2023/1/1', '2022/1/1', '2021/1/1']))
```

PYTHON

```
# using python datetime object
pd.DatetimeIndex([dt.datetime(2023,1,1), dt.datetime(2022,1,1), dt.datetime(2021,1,1)])
```

4. date_range — Generating Date Sequences

Rather than typing dates by hand, ask Pandas to produce a whole calendar for you with `pd.date_range(...)`.

Deep Dive : date_range — generate date sequences

- freq codes:
 - D daily, B business, W weekly, H hourly
 - M month-end, MS month-start, A year-end
 - combos: '2D', '3H', 'W-MON', 'BM'
- start/end OR periods — both ways
- Creates `DatetimeIndex` — template for data gaps
- Calendar-aware calendars (business days skip weekends)
- Filling experiment dates

- Common: daily series a month, sub-hourly
- `resample/asfreq` rely on freq logic
- ISO calendar/weekday support — freq detailed
- Creating dates without data — simulation, joins

PYTHON

```
pd.date_range(start='2023/1/5', end='2023/2/28')
pd.date_range(start='2023/1/5', end='2023/2/28', freq='B')    # business days
pd.date_range(start='2023/1/5', end='2023/2/28', freq='H')    # hourly
pd.date_range(start='2023/1/5', end='2023/2/28', freq='MS')   # month start
pd.date_range(start='2023/1/5', periods=25)                  # by count
```

4.1 Frequencies you should know

`pd.date_range(start='2023/1/5', end='2023/2/28', freq='3D')` walks from start to end in three-day steps, returning a `DatetimeIndex`. Change the `freq` and the rhythm of the walk changes completely:

- `'D'` — every calendar day; `'3D'` — every third day.
- `'B'` — business days only, skipping weekends (Jan 6→Jan 9 in the sample output).
- `'W-THU'` — once per week on the chosen weekday (Thursday in the example).
- `'6H'` — every six hours, stamping each day at 00:00, 06:00, 12:00, 18:00.
- `'M'` — month-end dates (2023-01-31, 2023-02-28); `'MS'` — month starts.
- `'A'` — year-end (2023-12-31, 2024-12-31), the annual rule `A-DEC`.

Instead of an end date you can specify `periods`: `pd.date_range(start='2023/1/5', periods=25, freq='M')` generates exactly 25 month-ends, useful when you want a template of fixed length regardless of calendar bounds.

Where do people use this? Generating experiment dates, expanding a sparse set of observed days into a full reference calendar, simulating time series, and constructing the index grids that `resample` / `asfreq` rely on.

`freq='B'` already answers 'what dates are working days?' and `freq` strings compose with numbers, so `'2W-MON'` and `'BM'` stretch the vocabulary further.

PYTHON

```
# generate daily dates in a given range
pd.date_range(start='2023/1/5', end='2023/2/28', freq='3D')
```

PYTHON

```
# alternate days in a given range
pd.date_range(start='2023/1/5', end='2023/2/28', freq='3D')
```

5. to_datetime — Converting String Columns

`date_range` creates dates; `to_datetime` rescues dates already sitting in your data as strings.

to_datetime function

converts an existing objects to pandas timestamp/datetimeindex object

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Deep Dive : to_datetime — string → Timestamp column

- to_datetime — CONVERT existing series/df of strings to datetime
- errors='coerce' → unparseable become NaT (missing)
- errors='raise' default — crash on bad
- apply to df col: df['Date'] = pd.to_datetime(df['Date'])
- format= arg for custom formats (speed + correctness)
- yearfirst/dayfirst for ambiguous dates
- After conversion: dtypes datetime64[ns], info shows
- Then .dt accessor, plotting, resampling available
- Weekly pipeline: read CSV → to_datetime → index → analyze
- Datetime col = 80% of real datasets 'date' column

PYTHON

```
s = pd.Series(['2023/1/1', '2022/1/1', '2021/1/1'])
pd.to_datetime(s)                                # parse whole column
pd.to_datetime(s, errors='coerce')                # bad dates → NaT (not crash)
```

5.1 Parsing whole columns safely

`pd.to_datetime(s)` takes a Series of date-like strings — `['2023/1/1', '2022/1/1', '2021/1/1']` — and returns a datetime series, and because it is now a real datetime you can immediately hang the `.dt` accessor off it: `.dt.day_name()` prints `Sunday`, `Saturday`, `Friday`. No loop, no exceptions.

The killer feature for dirty data is `errors='coerce'`. Feed it `['2023/1/1', '2022/1/1', '2021/130/1']` — note the impossible day 130 — and instead of raising, Pandas marks that element as `NaT` (Pandas' missing timestamp), letting the rest of the column parse. The default `errors='raise'` would abort the program. This mirrors `pd.to_numeric(..., errors='coerce')`: one call to sanitise a messy column into a professional dtype.

The expense notebook then applies the real-world recipe: `df['Date'] = pd.to_datetime(df['Date'])` upgrades the `Date` column's dtype in place, and `df.info()` confirms `datetime64[ns]` in the Dtype column. From that moment everything becomes possible: `.dt` extractions (next section), resampling, plotting with time axes, and chronological sorting. If a dataset has a 'date' column, roughly 80% of the time its first cleaning step is exactly this line.

PYTHON

```
# simple series example

s = pd.Series(['2023/1/1', '2022/1/1', '2021/1/1'])
pd.to_datetime(s).dt.day_name()
```

PYTHON

```
# with errors

s = pd.Series(['2023/1/1', '2022/1/1', '2021/130/1'])
pd.to_datetime(s, errors='coerce').dt.month_name()
```

6. The .dt Accessor

A Series of datetimes has its own accessor, `.dt`, just as strings have `.str`. It broadcasts date-property extraction across every row.

dt accessor

Accessor object for datetimelike properties of the Series values.

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

6.1 Extracting date parts and plotting time series

The first cell shows boolean flags straight off the accessor: `df['Date'].dt.is_quarter_start` returns a Series of `True` / `False` telling you which days open a quarter. The same family — `is_month_start`, `is_month_end`, `is_quarter_end`, plus `.weekday`, `.quarter`, `.hour` — turns the calendar into filterable conditions.

Plotting a raw time series is as simple as `plt.plot(df['Date'], df['INR'])`: matplotlib accepts the datetime column directly and spaces the x-axis chronologically. That alone beats the scramble of string axes.

The grouping cells then build the real dashboard. `df['day_name'] = df['Date'].dt.day_name()` labels every expense with its weekday name (`Monday` ...), and `df.groupby('day_name')['INR'].mean().plot(kind='bar')` shows you the average spend per weekday — immediately revealing, say, that weekends cost more. Mirror it at monthly granularity: `df['month_name'] = df['Date'].dt.month_name()` followed by `df.groupby('month_name')['INR'].sum().plot(kind='bar')` summarises where the month's money goes. Filtering closes the loop: `df[df['Date'].dt.is_month_end]` returns only transactions recorded on the last day of a month — handy for payroll, subscription-lapse, or bill analysis. Pattern to internalise: parse → extract labels with `.dt` → `groupby` a derived label → plot the aggregate.

PYTHON

```
df['Date'].dt.is_quarter_start
```

PYTHON

```
# plot graph
import matplotlib.pyplot as plt
plt.plot(df['Date'], df['INR'])
```

Deep Dive : dt accessor + date plots

- .dt — accessor for datetime props (like .str for strings)
- dayname/monthname — easiest grouping labels
- ismonthstart/end, isquarterstart/end — period flags
- Extract: hour, weekday, quarter via .dt
- groupby extracted name → trend bars (avg per weekday)
- filter flags: month-end transactions
- expense analysis use-case driven
- datetime col → valuable insights here
- plot direct: plt.plot(df['Date'], df['INR'])
- Careful: timezone, locale names

PYTHON

```
# day name wise bar chart/month wise bar chart

df['day_name'] = df['Date'].dt.day_name()
```

PYTHON

```
df.head()
```

Time Series Analysis — Case Study

1. Recap — Date & Time Fundamentals

Before diving into new concepts, this section replays the four date-time tools you have already met. Together they form the vocabulary of everything that follows.

PYTHON

```
import pandas as pd
import numpy as np
```

1.1 Timestamps, DatetimeIndex, datarange, todatetime

`pd.Timestamp('6th jan 2023 8:10')` builds a single moment in time — a **Timestamp** — automatically understanding strings like `'2023-1-5'`, `'5th January 2023'`, or `'2023, 1, 5'`. A `Timestamp` behaves like a superior `datetime` object: it exposes `.year`, `.month`, `.day`, and supports comparison and arithmetic.

A **DatetimeIndex** is a collection of timestamps. Placing one on a Series or DataFrame turns ordinary rows into time-stamped rows, which later enables slicing by date. `pd.date_range(start='2023-1-6', end='2023-1-31', freq='D')` manufactures a whole range of dates in one shot; the `freq` argument controls the spacing ('D' daily, 'B' business days, 'M' month-end, and so on). Finally, `pd.to_datetime()` converts a column of date-looking strings into real `datetime64` values, after which we can use the `.dt` accessor — for example `pd.to_datetime(s).dt.day_name()` prints the weekday name for every element.

These four tools are the foundation: a Timestamp is one point, a DatetimeIndex is many points as labels, `date_range` fabricates points, and `to_datetime` converts messy text into points. Run the code cell to see each one in action.

PYTHON

```
# till now
# Timestamp
pd.Timestamp('6th jan 2023 8:10')
# DatetimeIndex -> df and series index
pd.DatetimeIndex([pd.Timestamp('6th jan 2023 8:10'),pd.Timestamp('7th jan 2023 8:10'),pd.Timestamp('8th jan 2023 8:10')])
# date_range()
pd.date_range(start='2023-1-6',end='2023-1-31',freq='D')
# to_datetime()
s = pd.Series(['2023/1/6','2023/1/7','2023/1/7'])
pd.to_datetime(s).dt.day_name()
```

2. Timedelta — Duration Between Dates

The second big idea of the module: while a Timestamp answers '*when?*', a **Timedelta** answers '*how long?*'. It is a duration, not a moment on the calendar.

Timedelta Object

Represents a duration, the difference between two dates or times.

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Timedelta — duration

- Timedelta = **difference** between 2 dates/times (duration)
- Or standalone `pd.Timedelta(days, hours, ...)`
- Arithmetic: `Timestamp + Timedelta` → a new Timestamp
- `date_range - Timedelta` → shifted dates

PYTHON

```
t1 = pd.Timestamp('6th Jan 2023 8:20:14')
t2 = pd.Timestamp('7th Jan 2023 10:00:00')
t2 - t1                                # Timedelta object
pd.Timedelta(days=2, hours=10, minutes=35)
```

Deep Dive : Timedelta — duration between dates

- Timedelta = DURATION (difference), Timestamp = a MOMENT
- subtract Timestamps → Timedelta
- Arithmetic with Timedelta shifts dates
- .dt.days / .total_seconds() — extract duration size
- Real: order→delivery time = *deliverydate* - *orderdate* column
- Rolling: avg delivery time, late order detection
- As dtype: timedelta64[ns] — vector-friendly
- Compare durations directly
- Useful for SLA/completion time metrics

PYTHON

```
t1 = pd.Timestamp('6th Jan 2023 8:20:14'); t2 = pd.Timestamp('9th Jan 2023 6:00:00')
t2 - t1                                # → Timedelta 2 days 21:39:46
pd.Timedelta(days=2, hours=10, minutes=35) # standalone duration
pd.Timestamp('6th jan 2023') + pd.Timedelta(days=2, hours=10, minutes=35)

df['delivery_time_period'] = df['delivery_date'] - df['order_date']
df['delivery_time_period'].dt.days      # integer days
```

2.1 Constructing and using Timedelta objects

A Timedelta represents a span of time such as `2 days 10:35:00`. There are two natural ways to obtain one. First, **subtract one Timestamp from another**: `t2 - t1` yields the elapsed duration between the two moments, exactly like measuring a journey between a start and an end time. Second, you can build a standalone duration with `pd.Timedelta(days=2, hours=10, minutes=35)`, which is handy when you know the length in advance.

Timedeltas participate in arithmetic. Adding a Timedelta to a Timestamp (`pd.Timestamp('6th jan 2023') + pd.Timedelta(days=2, hours=10, minutes=35)`) shifts the moment forward and returns a new Timestamp; subtracting shifts it backwards. You can also subtract a Timedelta from an entire `DatetimeIndex`, for example a `date_range`, which slides every date by the same amount at once — a fully vectorised calendar operation.

These durations can also live inside data structures: subtract two datetime columns element-wise and you get a Timedelta column, which you will exploit in the next cell's delivery-time problem. When printed, a Timedelta reads naturally as `Timedelta('20 days 01:39:46')`, and you can later extract pieces of it with `.dt.days` or `.dt.total_seconds()`.

PYTHON

```
# create using Timestamp objects
t1 = pd.Timestamp('6th Jan 2023 8:20:14')
t2 = pd.Timestamp('26th Jan 2023 10:00:00')

t2 - t1
```

PYTHON

```
# standalone creation
pd.Timedelta(days=2, hours=10, minutes=35)
```

2.2 Real-world example — computing delivery time

Real business data loves this trick. Here a CSV records `order_date` and `delivery_date` for thousands of orders. Both arrive as strings (`5/24/98`), so the first step is to parse them with `pd.to_datetime`. Only after both columns are true datetimes can you subtract them row by row: `df['delivery_date'] - df['order_date']` produces a **Timedelta column** whose value in every row is the number of days the order took to arrive.

That computed column is a genuine business metric — the delivery time — and it is fully vectorised, meaning Pandas calculates all rows simultaneously in compiled code rather than looping in Python. You can inspect its mean (`df['delivery_time_period'].mean()`) returns something like `Timedelta('1217 days 22:53:53')` in this toy data), or later bucket it into fast/slow categories, detect late orders, and report a team's average SLA. The `.dt` accessor similarly lets you pull out `.days` as integers for further analysis. This pattern — parse, convert, subtract, aggregate — is the heart of many logistics and e-commerce analytics pipelines.

PYTHON

```
# real life example
df = pd.read_csv('deliveries.csv')
df.head()
```

PYTHON

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['delivery_date'] = pd.to_datetime(df['delivery_date'])
```

Real-life — delivery time

- Difference of 2 datetime cols → Timedelta col
- Business metric — delivery time
- Use `.dt.days` to extract it

PYTHON

```
df['delivery_time_period'] = df['delivery_date'] - df['order_date']

df['delivery_time_period'].mean()
```

PYTHON

```
df['delivery_date'] - df['order_date'] # duration per row
df['delivery_time_period'] = ...
```

3. Time-Series Data & the DatetimeIndex

A time series is a data set that tracks a sample over time. What makes it special is the ordering: rows mean something because of *when* they happened, so the date must become the primary axis of the table.

Time series

A time series is a data set that tracks a sample over time. In particular, a time series allows one to see what factors influence certain variables from period to period. Time series analysis can be useful to see how a given asset, security, or economic variable changes over time.

Examples

- Financial Data (Company stocks)
- Natural Data (Rainfall measurement)
- Event Data (Covid)
- Medical Data (Heart rate monitoring)

Types of Operations done on Time Series

- Time Series Analysis
- Time Series Forecasting

Time series data

- Time series = data ordered in time
- Setting a Datetime column as the index unlocks datetime elegance
- `loc['2021-12-30']` — specific date
- Partial indexing: `yt.loc['2022']`, `yt.loc['2022-12']`
- `.index.month_name()` etc — labels

PYTHON

```
google = pd.read_csv('google.csv')
google['Date'] = pd.to_datetime(google['Date'])
google.set_index('Date', inplace=True)
```

3.1 Loading time-series data

Two realistic datasets drive this section. `google.csv` holds daily stock prices — Open, High, Low, Close, Adjusted Close, and Volume — from August 2004 to May 2022. The remaining files (`subscribers.csv`, `views.csv`, `watch-time.csv`, `comments.csv`) describe a YouTube creator channel day by day since April 2019. The YouTube pieces are joined together on the shared `Date` column with a chain of `merge()` calls, producing one wide daily table of Subscribers, Views, Watch time (hours), and Comments added.

The moment of transformation is `pd.to_datetime(...)` on the `Date` column followed by `set_index('Date', inplace=True)`. After that the label of every row is a Timestamp and the DataFrame becomes a true time series: `yt.info()` shows a `datetime64[ns]` index, head/tail confirm chronological order, and all the date-powered features that follow — slicing, resampling, and automatic time axes on plots — switch on automatically. The principle is worth repeating: set a meaningful index early, ideally right after you load the data.

PYTHON

```
google = pd.read_csv('google.csv')
google.head()
```

PYTHON

```
google.tail()
```

3.2 Partial indexing and slicing with dates

With a DatetimeIndex in place, lookup becomes elegant. `google.loc['2021-12-30']` fetches that exact trading day, returning its six price columns as a Series. More remarkably, Pandas understands **partial indexing**: `yt.loc['2022']` returns every row in the year 2022, and `yt.loc['2022-12']` every row in December 2022 — you never write a range, the library infers the window from a prefix of the label.

Slicing with `.loc` is inclusive on both ends and can carry a step, so `yt.loc['2022-12-15':'2023-1-1':2]` walks every second day across the year boundary. Because every label is a real Timestamp you can also derive brand-new columns from the index itself: `.index.month_name()`, `.index.day_name()`, and `.index.quarter` fabricate `'December'`, `'Wednesday'`, and `2` for each row. Try `google[x]`. Isin-style lookups even support `pd.date_range` + `pd.DateOffset(years=1)` to pull the same calendar date across many years — a neat answer to the challenge of 'every January 6th'. Every technique presupposes a sorted, real datetime index.

Deep Dive : DatetimeIndex — partial indexing & slicing

- Setting Date as index unlocks date-powered lookup
- Partial index ('2022-12', '2022') — pandas figures range automatically
- `.loc` slices inclusive on dates like labels
- derived from index: `.index.monthname()/dayname()`
- challenges: exact-date-every-year needs day/month extraction
- Frequency accessories: `index.year`, `index.quarter`
- Fast range queries with sorted index
- Chronological plotting axis labels
- Principle: set meaningful index early

PYTHON

```
# fetch a specific date
google.loc['2021-12-30']
```

PYTHON

```
# partial indexing -> select a particular year/month
yt.loc['2022-12']
yt.loc['2022']
```

4. Visualizing Time Series

Numbers in a table are hard to read; a line across time tells the story instantly. Once the index is a `DatetimeIndex`, `plot()` does the rest.

Visualization

- Datetime index → plot automatically sorted x-axis
- Monthly/quarterly trend — groupby + mean
- Helps to see yearly/month-wise patterns

PYTHON

```
google['Close'].plot()
yt.groupby('month_name')['Subscribers'].mean().plot(kind='bar')
yt.groupby('quarter')['Close'].mean().plot(kind='bar')
```

Deep Dive : Time series visualization

- After datetime index → `.plot()` time axis automatic
- Single col / subset window plots
- groupby month_name/year/quarter → trend bars
- Aggregation before plotting = smoother trends
- Multiple cols: `yt[['Subscribers','Views',...]].plot()`
- Layout: `df.plot(subplots=True, layout=(3,2), sharex=False)`
- Longer range zoom via loc slices
- Visual pattern: each trends, seasonality, spikes
- Bar for categorical time, line for continuous
- default figure size adjustable

PYTHON

```
google['Close'].plot() # line over time
yt['Subscribers'].plot()
yt.loc['2022-12']['Subscribers'].plot() # single month window
yt.groupby('month_name')['Subscribers'].mean().plot(kind='bar')
google.groupby('quarter')['Close'].mean().plot(kind='bar')
```

4.1 Plotting lines and bars

A single column followed by `.plot()` — `google['Close'].plot()` — draws a continuous line with time automatically ordered along the x-axis, revealing the long growth arc of the stock plus its volatile daily noise. The same works for `yt['Subscribers']`, and slicing first (`yt.loc['2022-12']['Subscribers'].plot()`) zooms into a one-month window for a close-up.

For several series at once, `subset_yt.plot(subplots=True, layout=(2, 2), sharex=False, sharey=False, ...)` renders four separate panels (Subscribers, Views, Watch time, Comments) on one figure so wildly different scales do not distort each other; `figsize`, `linewidth`, and `legend` control the cosmetics.

The real analytical payoff comes from **aggregating before plotting**. `yt.groupby('month_name')['Subscribers'].mean().plot(kind='bar')` shows the average subscriber gain for each month name across all years — instantly exposing seasonality. The identical trick on `weekday_name` reveals which weekdays perform best, and `google.groupby('quarter')['Close'].mean().plot(kind='bar')` summarises a decade of stock prices into four tidy bars. The rule of thumb: line plots for continuous time, bar plots after grouping by a categorical time label such as a month or weekday name.

PYTHON

```
# viz a single col
google['Close'].plot()
```

PYTHON

```
yt['Subscribers'].plot()
```

5. Frequency and asfreq

The spacing between dates — the **frequency** — is a property of any time series, and Pandas provides tools to inspect it and force your data onto a regular grid.

AI/ML relevance

Shapes, dtypes, vectorization, indexing, and clean tabular ops are critical for reliable data pipelines.

PYTHON

```
# frequency
google.index
```

5.1 Reading the frequency of a DatetimeIndex

Before you can convert or resample a time series, it helps to know the current frequency of the index.

`google.index` is a `DatetimeIndex`; dumping it to the output shows each trading date. Notice that the dates are not evenly spaced — weekends and holidays are missing because the stock market is closed. Pandas represents this as a frequency string. A simple daily calendar would be `'D'`, a business-day calendar `'B'`, and here the gaps mean the index is **irregular** with respect to a pure calendar.

The `asfreq()` method then forces the index onto a fixed frequency grid. For example `google.asfreq('6H')` upsamples the daily data to 6-hour intervals, creating new timestamps that have no observed values. Because those new slots are empty, you must tell Pandas how to fill them: `method='bfill'` (back-fill) copies the *next* known value backwards, while `method='ffill'` would copy the *previous* value forward. `asfreq` only rearranges the layout of existing values — it never aggregates or smooths them. That is the key difference from `resample`, which groups values into periods and then applies a reduction like `mean()` or `sum()`. Think of `asfreq` as a ruler that forces your data onto a regular grid, and of the filling method as the rule that decides which neighbour fills each gap.

Deep Dive : Frequency & asfreq

- freq = time spacing between observations
- asfreq — CONVERT index frequency (no aggregation)
- higher freq (D→6H) = upsample — gaps created → filling method needed
- lowest freq (6H→D) = down
- method: bfill (back fill — next), ffill (forward — previous)
- Creates regular grid for missing
- Compare same date across time enabling
- Aligns irregular data to uniform steps
- asfreq vs resample — asfreq layout only; resample does aggregate
- Padding = where values appear duplicated continuous assumption

PYTHON

```
google.index          # check current frequency (eg 'D')
google.asfreq('6H')   # upsample frequent Higher freq
google.asfreq('6H', method='bfill') # fill missing by backfill
```

5.2 asfreq — forcing data onto a regular grid

`asfreq('6H', method='bfill')` on the Google frame produces a new `DatetimeIndex` spaced every six hours — a much higher frequency than the original daily trading dates. This is **upsampling**: you manufacture new timestamps that fall between the observed ones. Of course no price was ever recorded at those intermediate instants, so the values at the new slots must be borrowed from neighbours. `method='bfill'` copies the next known value backwards to fill the gap (also called back-filling), while `method='ffill'` would copy the previous value forwards.

It is important to distinguish `asfreq` from what comes next. `asfreq` only *relabels* the index onto a fixed grid and optionally pads the holes; it never summarizes. Because it does not aggregate, you still see the raw closing prices repeated along each day. If instead you need daily → monthly summaries, or hourly → daily totals, you trade `asfreq` for `resample`, which genuinely groups rows into periods and reduces them. In a pipeline you will often use `asfreq` to build a clean regular calendar (fills required for joins, monthly reporting templates, and comparison grids) and `resample` to derive period-level aggregates.

PYTHON

```
# asfreq
google.asfreq('6H',method='bfill')
```

6. Resampling — Changing Data Frequency

Resampling changes the frequency of the observations themselves, and unlike `asfreq` it always involves an aggregation step for coarser periods.

Resampling

Resampling involves changing the frequency of your time series observations.

Two types of resampling are:

Upsampling: Where you increase the frequency of the samples, such as from minutes to seconds.

Downsampling: Where you decrease the frequency of the samples, such as from days to months.

- **Downsampling** — higher freq → lower (sum/mean)
- **Upsampling** — lower → higher (interpolate)
- `resample('Y')` daily → yearly
- `interpolate` — gaps smooth fill

PYTHON

```
yt['Subscribers'].resample('Y').mean() # down - aggregate
google['Close'].resample('12H').interpolate(method='spline', order=2) # up
```

Deep Dive : Resampling

- `resample` = `groupby` for time — CHANGE FREQUENCY + AGGREGATE
- Downsample (high→low freq): mean/sum/max per period (smooth trend)
- Upsample (low→high freq): new points → interpolate fills
 - linear, `spline(order)`, `ffill/bfill`
- Rules: D/M/Y/Q/H; 'M' month-end, 'Y' year-end
- Aggregations per period like `groupby`
- Patterns: quarterly report, daily → weekly sales
- Rolling mean equivalent but period based
- Requires datetime index (sorted)
- Contrast: `asfreq` layout only; `resample` + aggregate

PYTHON

```
yt['Subscribers'].resample('Y').mean()      # DOWNSAMPLE → YEAR aggregate
google['Close'].resample('12H')            # UPSAMPLE → 12-hour grid
yt['Subscribers'].resample('M').sum()
# downsampling: plot smoother monthly/yearly trend
# upsampling: missing points via interpolate()
google['Close'].resample('12H').interpolate(method='spline', order=2)
```

6.1 Downsampling and upsampling

Think of `resample` as **groupby for time**. It slices the series into consecutive periods (months, years, hours) and then applies a reduction to each slice. **Downsampling** moves from a higher frequency to a lower one — daily to yearly, say — and compresses many rows into one summary:

`yt['Subscribers'].resample('Y').mean()` averages all the daily subscriber counts within each calendar year, and `.plot()` on the result overlays a smooth yearly trend on top of the raw daily curve so the long-term direction becomes visible. `'M'` (month-end) and `'Y'` (year-end) are the everyday rules; `'W'`, `'Q'`, and `'H'` extend the menu.

Upsampling is the reverse movement — daily to 12-hourly — and it creates more rows than exist, with no observed values at the new points. Those missing values cannot be summed away; they must be manufactured. `google['Close'].resample('12H').interpolate(method='spline', order=2)` fills the newly created gaps with a smooth spline curve fitted through the neighbouring prices. Ordinary `linear`, or a simple forward/backward fill, are the cheaper alternatives. In one sentence: downsampling aggregates *real* data into summaries, while upsampling invents plausible in-between points so that uneven or sparse series can share a common clock with other data.

PYTHON

```
# Downsampling
yt['Subscribers'].plot()
yt['Subscribers'].resample('Y').mean().plot()
```

PYTHON

```
# Upsampling
google['Close'].resample('12H').interpolate(method='spline', order=2).plot()
```

7. Rolling Window — Smoothing the Signal

Raw time-series data is often disgustingly noisy: a daily subscriber count jumps up and down for no good reason. Rolling windows are the classic cure.

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Rolling Window(Smoothing)

Time series data in original format can be quite volatile, especially on smaller aggregation levels. The concept of rolling, or moving averages is a useful technique for smoothing time series data.

- Keep taking the mean over a window of data
- To smooth a volatile series
- Makes the trend visible

PYTHON

```
yt['Subscribers'].rolling(30).mean() # 30-day avg
```

Deep Dive : Rolling window — smoothing

- rolling(n) — rolling WINDOW of last n observations then agg
- rolling mean = moving average — SMOOTHS noise, shows trend
- Volatile daily → clean mAvg curve
- window= n (rows) OR offset ('30D' time-based)
- agg: mean/max/std/min/sum
- Rolling std — volatility measure
- First n-1 → NaN (small window initials)
- The classic time-series smoothing tool
- vs resample: rolling = moving window; resample = buckets periods

PYTHON

```
yt['Subscribers'].rolling(30).mean() # 30-day MOVING average
yt['Subscribers'].rolling(30).mean().plot(title='rolling')
yt['Subscribers'].rolling(30).max() / .std()
```

7.1 Moving averages and exponential smoothing

`.rolling(30)` slides a window of the last 30 observations across the series, and `.mean()` computes the mean of the values currently inside the window at every step. The result is the 30-day **moving average**: each smoothed point replaces the volatile daily spike with the average of the trailing month, so the underlying trend emerges while transients get ironed out. The first 29 rows are `NaN` because the window has not yet collected enough history — a harmless and expected behaviour.

Because `rolling` returns a windowed object, any reduction works: `.max()`, `.min()`, `.std()` (a rolling volatility), `.sum()`, and more. Windows can also be time-based, e.g. `.rolling('30D')`, when you have irregularly spaced timestamps. The `ewm(30).mean()` line adds a second smoother — the **exponentially weighted moving average** — which weights the most recent observations more heavily than ancient ones, producing a tail that reacts faster and never waits a full window before it starts. Compare the two curves in the plot: rolling gives a clean moving average, `ewm` gives a quickly adapting, memory-weighted average. If `resample` buckets time into fixed periods, `rolling` is a window that *moves* one step at a time.

PYTHON

```
# Rolling window
yt['Subscribers'].rolling(30).mean().plot(title='rolling')
yt['Subscribers'].ewm(30).mean().plot(title='ewm')
```

8. Shift — Creating Lag Features

Shifting does not change your measurement frequency at all; it moves the *values* relative to the rows, which is the foundation of feature engineering and of detecting change.

Shifting

The `shift()` function in Pandas is used to, well, shift the entire series up or down by the desired number of periods.

Pandas fundamentals

Labeled axes, alignment, indexing, grouping, missing data, I/O. Critical for EDA/preprocessing.

Shift

- `shift(n)` — shifts the values by *n* positions
- Lag features (prev day values)
- Login analysis: prev login time minus current
- A tool for feature engineering

PYTHON

```
yt['Subscribers'].shift(1) # one step back
```

Deep Dive : Shift — time lag

- `shift(k)` — move values *k* steps (creates LAG column)
- `shift(1)` = previous value; `shift(-1)` = next value
- First row `shift(1)` → NaN (nothing before)
- Compute differences: current - shifted = per-step change
- Login time analysis: gap between consecutive logins
- Feature engineering: yesterday's value as predictor
- % change, momentum detections
- Align by position, not date
- lag features = forecasting bread & butter
- You can shift per group too

PYTHON

```
yt['Subscribers'].shift(1)      # PREVIOUS value at current row
yt['Subscribers'].shift(-1)     # NEXT value (negative)
user_df['shifted'] = user_df['login_time'].shift(1)
(user_df['login_time'] - user_df['shifted']) # time BETWEEN logins
```

8.1 shift() and the gap-between-events trick

`shift(n)` slides the entire series up or down by `n` positions, keeping the index untouched. `yt['Subscribers'].shift(1)` places the *previous* day's value in the current row: row at date D now shows what was measured at D−1. Because nothing existed before the first row, that first slot receives `NaN`. A negative shift does the opposite — `shift(-1)` places the *next* day's value in the current row, and the last row becomes `NaN`.

Why care? A shifted column is a **lag feature**: yesterday's subscriber count is often a strong predictor of today's, so lagged columns feed forecasting models. The same machinery solves the login-gap problem. A login log is read with `header=None`, the useful columns are renamed `user_id` and `login_time`, and one user's rows are filtered out. After converting `login_time` to real datetimes, `user_df['shifted'] = user_df['login_time'].shift(1)` lines each login up with the login *before* it. Subtracting the two columns then measures the time *between* consecutive logins, and taking the mean gives the user's average idle gap — a rich behavioural metric built from three simple operations: parse, shift, subtract. Watch for the `SettingWithCopyWarning`; it is only a caution, not an error.

PYTHON

```
# shift
yt['Subscribers']
```

PYTHON

```
yt['Subscribers'].shift(-1)
```

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- Series and DataFrame internals
- selection, indexing and alignment
- missing data and type handling
- groupby, aggregation and reshaping
- merges, joins and time series indexes

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

1.1 IMPORTING PANDAS

```
import numpy as np
import pandas as pd
```

PYTHON

2.1 THE LABELED ARRAY

```
country = ['India', 'Pakistan', 'USA', 'Nepal', 'Srilanka']
pd.Series(country)
```

PYTHON

2.2 DTYPE INFERENCE ON CREATION

```
# string
country = ['India', 'Pakistan', 'USA', 'Nepal', 'Srilanka']

pd.Series(country)
```

PYTHON

2.2 DTYPE INFERENCE ON CREATION

```
# integers
runs = [13, 24, 56, 78, 100]

runs_ser = pd.Series(runs)
```

PYTHON

2.2 DTYPE INFERENCE ON CREATION

```
pd.Series(['India', 'Pakistan', 'USA'])
pd.Series([13, 24, 56, 78, 100])
```

PYTHON

3. CUSTOM INDEX AND NAME

```
# custom index
marks = [67,57,89,100]
subjects = ['maths','english','science','hindi']

pd.Series(marks,index=subjects)
```

PYTHON

3. CUSTOM INDEX AND NAME

```
# setting a name
marks = pd.Series(marks,index=subjects,name='Nitish ke marks')
marks
```

Glossary

Timestamp

a single point in time, parsing strings like ``5th January 2023 9:21AM`` and exposing `.

inner

only keys present in BOTH tables (the common core).

left

every row of the LEFT frame, plus matches from the right; unmatched right columns become NaN.

right

every row of the RIGHT frame, plus matches from the left.

outer

the union of everything, NaN where the other side has no match.

Wide format

where we have a single row for every data point with multiple columns to hold the values of various attributes.

Long format

where, for each data point we have as many rows as the number of attributes and each row contains the value of a particular attribute for a given data point.

DatetimeIndex

an array of timestamps good enough to sit on a Series or DataFrame, unlocking time-powered lookup.

Upsampling

Where you increase the frequency of the samples, such as from minutes to seconds.

Downsampling

Where you decrease the frequency of the samples, such as from days to months.

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

☐

1. Introducing Pandas

☐

1.1 Importing pandas

☐

2. Series from Lists

☐

2.1 The labeled array

☐

2.2 Dtype inference on creation

☐

3. Custom Index and Name

☐

4.1 Dict keys become the index

☐

5.1 Introspecting a Series

☐

5.2 Reading attributes in practice

☐

6.1 *readcsv, squeeze, and indexcol*

☐

7. Copy versus Views

☐

7.2 Making safe copies

☐ 8.1 head, tail, sample, and value_counts☐ 8.2 sortvalues and sortindex☐ 9.1 count, sum, mean, median, mode, std, var☐ 9.2 describe and the statistical toolkit☐ 10.1 Positional and label access☐ 10.2 Fancy indexing and step slices☐ 11.1 Assignment by position and label☐ 11.2 Adding new labels☐ 12. Copy and Views in Practice☐ 13.1 len, type, list, dict, membership